Esquema de Base de Datos y Servicios de Latch

Steven Cuadra

Alfredo Montalvan

Caleb Tardencilla

Aris Gavarrete

Facultad de Ingeniería y Arquitectura, Universidad Americana (UAM)

CNU0525: Programación de Aplicaciones de Escritorio, Grupo 2

José A. Durán García

26 de noviembre de 2026

Resumen

Latch organiza los datos del alquiler mediante veintitrés tablas PostgreSQL y servicios de aplicación que operan dentro del escritorio JavaFX. El modelo conserva contratos, cargos, pagos parciales, recibos, garantías y mantenimiento, con referencias explícitas a los requisitos del producto y los flujos suministrados. La propuesta utiliza UUID como identidad técnica y un contador transaccional separado para los recibos, con bloqueo de fila y confirmación conjunta del pago, sus aplicaciones y la auditoría. Los importes y las tasas mantienen precisión decimal, mientras las vistas calculan deuda, crédito y remanentes sin totales editables redundantes. La persistencia distribuye entidades de negocio en Hibernate y seguridad, bitácora y agregados en JDBC, sobre una misma conexión cuando participan en una operación financiera. La integración externa conserva ExchangeRate-API como referencia informativa y exige autorización por fecha para convertir un cobro entre NIO y USD. El documento incluye diccionario, restricciones, consultas nombradas, doce contratos de servicio, diez fuentes de reportes y dos consultas complementarias, respaldo y migraciones, con scripts completos y datos sintéticos. Las verificaciones históricas de la Tabla 55 quedan pendientes de reejecución tras modificar los scripts; las comprobaciones sobre esta revisión quedan pendientes de reejecución. Sus resultados no acreditan una aplicación terminada, pruebas con usuarios o una integración HTTP autenticada. La planificación asigna responsables y capacidad estimada hasta el cierre académico de referencia.

Palabras clave: PostgreSQL, Java, persistencia, concurrencia, recibos, servicios, Latch

Esquema de Base de Datos y Servicios de Latch

Introducción

Latch almacenará las obligaciones y los movimientos del alquiler en PostgreSQL 18.6, con una capa de servicios dentro de la aplicación JavaFX. El esquema conecta unidades, inquilinos, contratos, cargos y pagos mediante claves y restricciones explícitas. Su audiencia comprende a los cuatro integrantes y al docente de Programación de Aplicaciones de Escritorio. El documento fija qué conserva cada registro, cómo se confirma una operación y de dónde procede cada reporte. El término backend designa esa combinación de base de datos y servicios locales, sin añadir un servidor de aplicación ni una interfaz web al alcance académico.

El PRD aporta requisitos RF, reglas RN y metas RNF (Cuadra et al., 2026c). Los flujos determinan validaciones y confirmación, especialmente FLU-24 a FLU-27 (Cuadra et al., 2026b). El TRD establece JDBC, Hibernate y las fronteras de transacción (Cuadra et al., 2026d). El Brief fija pantallas, mensajes y catálogos visibles (Cuadra et al., 2026a). El Plan es la fuente única de calendario, tareas TAR, casos CP, RACI y revisión cruzada (Cuadra et al., 2026f). Las decisiones de coherencia aplicadas a este Esquema prevalecen cuando los documentos de apoyo conservan una redacción anterior.

La investigación documental permite formular requisitos verificables sin afirmar prácticas observadas en propietarios de Managua. Lemma et al. (2020) revisaron intervenciones de calidad y uso de datos en sistemas sanitarios, con factores técnicos y de capacitación. Esa evidencia pertenece a otro sector y no acredita beneficios de Latch ni hábitos locales. El equipo adopta como decisión de diseño la conciliación de saldos, la trazabilidad y la revisión de registros. El escenario de alquiler combina contratos NIO o USD y cobros en otra moneda; los ejemplos y las personas de prueba son sintéticos y no proceden de expedientes privados. El

antecedente de Balzarini y Boyd (2021), citado en el PRD, se identifica por el año del número final; su versión en línea anticipada de 2020 se distingue en el Apéndice F.

Supuestos y Alcance de la Evidencia

La fecha base de planificación es el 22 de octubre de 2026 y la consulta documental corresponde al 23 de octubre de 2026. El 26 de noviembre de 2026 funciona como entrega interna de documentos y versión demostrable; su uso en portada no confirma el calendario institucional. El contexto aportado sitúa la evaluación entre esa fecha y el 1 de diciembre. Cada integrante dispone de una capacidad supuesta de doce horas por semana. Las asignaciones y los hitos describen una propuesta del proyecto, sin representar acuerdos celebrados, horas trabajadas ni aprobación del docente sobre decisiones técnicas.

La Tabla 55 conserva las comprobaciones del ensayo anterior como pendientes de reejecución tras los cambios de DDL y scripts. Sus resultados anteriores no acreditan esta revisión. Toda evidencia nueva queda pendiente de reejecución con entorno, scripts y alcance explícitos. Las pruebas de aplicación JavaFX, JasperReports e integración HTTP requieren su implementación. El resumen académico orienta RA1 a RA5; el syllabus original no forma parte de las fuentes del proyecto.

La Tabla 1 presenta decisiones Conservadas y Precisiones del Esquema.

Tabla 1

Decisiones Conservadas y Precisiones del Esquema

Tema	Decisión Adoptada	Fundamento
Persistencia	PostgreSQL 18.6; pgJDBC 42.7.14; Hibernate 7.4.6.Final; Jakarta Persistence 3.2.	ADR-04, ADR-06 y ADR-10; versiones consultadas en fuentes oficiales.
Precisión	NUMERIC(19,2) para dinero y NUMERIC(19,8) para tasa; HALF_EVEN.	RN-32 y TRD recibido prevalecen sobre 18,2 y 18,6 sugeridos en la base común. No se reduce precisión histórica.

Tema	Decisión Adoptada	Fundamento
Identidad	UUID preasignado en Java; gen_random_uuid() para SQL.	Conserva TRD y referencias opacas de preferencias. Se descarta IDENTITY como clave del dominio; el recibo usa contador aparte.
Reserva de unidad	Índice único parcial para Vigente, Por vencer y Vencido.	RN-01 ampliada. Es más estricto que excluir intervalos: solo una reserva activa, incluso con intervalos futuros distintos. Los futuros sucesores quedan Borrador.
Nombres físicos	consecutivo_recibo y esquema_migracion; rol de ejecución latch_runtime.	Nombres físicos canónicos para TRD, Flujo y Esquema; latch_owner conserva la propiedad y latch_runtime la ejecución.
Recuperación	Puerta con fila bloqueada, compartida para escrituras y exclusiva para respaldo.	La decisión histórica de control de respaldo se concreta en un bloqueo de fila equivalente, RN-29. La conexión coordinadora permanece abierta durante la copia.
Presentación	Claro y Oscuro Aurora UI; preferencias locales por UUID.	ADR-18, RN-35 a RN-38. El tema, efectos, degradados y desenfoques no agregan tablas al dominio.

Versiones y Límites de Compatibilidad

La política de PostgreSQL identifica 18.6 como revisión vigente de la rama 18, con soporte hasta el 14 de noviembre de 2030 (PostgreSQL Global Development Group [PGDG], s. f.-t). La descarga oficial de pgJDBC publica 42.7.14 para Java 8 o posterior, compatible con el JDK 21 adoptado (PostgreSQL JDBC Driver, s. f.-a). Hibernate enumera 7.4.6.Final y admite Java 21 con Jakarta Persistence 3.2; la serie consultada ya publica revisiones posteriores (Hibernate, s. f.-a). Latch conserva la versión fijada por el TRD para mantener reproducibilidad durante la entrega y somete actualizaciones a revisión del equipo.

Jaspersoft publica el lanzamiento de JasperReports 7.0.8 y su repositorio permite revisar el POM etiquetado, en lugar de suponer la existencia de esa versión (Jaspersoft, 2026; Jaspersoft, s. f.). Latch mantiene esa generación para Library y sus plantillas, y transmite DTO o filas JDBC al reporte. La integración no usará el adaptador Hibernate de JasperReports para escribir el

dominio. Esta elaboración verifica publicación y diseño de integración, pero no certifica una compilación conjunta de todas las dependencias Java. El montaje final registrará la revisión de Maven, las bibliotecas resueltas y las pruebas de exportación ejecutadas.

Principios y Convenciones de Modelado

El esquema latch usa nombres en español, snake_case, minúsculas y caracteres sin tildes para objetos SQL. El vocabulario coincide con el proyecto y reduce traducciones entre reglas, servicios y diccionario. Los valores visibles conservan tildes y espacios de los catálogos, como Depósito bancario o Por vencer. Las claves primarias llevan pk_, las referencias fk_, las unicidades uq_, las comprobaciones ck_ y los índices de consulta ix_. Cada nombre incluye tabla y propósito cuando corresponde. El Apéndice A contiene el DDL completo y el Apéndice B reúne las vistas, funciones y el procedimiento de validación financiera. Los discriminadores internos aplicacion_pago.origen y movimiento_deposito.tipo se almacenan en ASCII: Inicial/Credito/Garantia y Aplicacion/Devolucion, respectivamente; sus etiquetas de presentación conservan las tildes que correspondan.

La base se crea con codificación UTF8 y locale C, elección determinista disponible en el entorno PostgreSQL utilizado. Ese orden no pretende reproducir la clasificación lingüística del español; los listados que necesiten orden natural usarán Collator de Java sobre una selección acotada. Las búsquedas por cuenta admiten únicamente ASCII y aplican lower(nombre_usuario), por lo que su unicidad no depende de reglas de acentuación. PostgreSQL permite definir collation por base, columna o expresión; el diseño evita imponer una collation ICU cuya disponibilidad no se haya comprobado en la instalación destino (PGDG, s. f.-a).

Los importes usan BigDecimal y NUMERIC(19,2); las tasas NIO por USD usan NUMERIC(19,8), de acuerdo con RN-32 del PRD. PostgreSQL distingue tipos exactos de tipos

binarios aproximados, por lo que Latch excluye float, double y money de los cálculos financieros (PGDG, s. f.-k). La conversión redondea el total una vez con HALF_EVEN y distribuye cantidades ya expresadas en centavos contractuales. Si una proporción produce residuo, el último cargo seleccionado recibe el ajuste sin superar su saldo. El crédito conserva cualquier remanente, siempre en la moneda del mismo contrato y con origen en el pago.

Las fechas de contrato, período, vencimiento y pago son DATE y se mapean a LocalDate. Los instantes de auditoría son TIMESTAMP WITH TIME ZONE, abreviado TIMESTAMPTZ, y se mapean a Instant; pgJDBC puede transportarlos como OffsetDateTime y convertirlos a Instant en el adaptador. La base conserva el instante y la interfaz aplica America/Managua para presentación. La función fn_fin_corte define el límite exclusivo del día local, de modo que el corte incluye sus veinticuatro horas. PostgreSQL diferencia estos tipos y no conserva una etiqueta de zona original dentro de TIMESTAMPTZ (PGDG, s. f.-h). El instante y la fecha operativa financiera proceden de now() de la BD, con fecha calculada en America/Managua. Si la diferencia absoluta con el reloj local supera cinco minutos, el servicio bloquea nuevas operaciones financieras con RelojDesfasadoException (ERR-057). El bloqueo se mantiene hasta corregir el desfase y no interrumpe una transacción financiera ya iniciada; la fecha efectiva ingresada se valida por separado (ERR-057 / MSG-080).

La Tabla 2 presenta convenciones de Datos y Auditoría.

Tabla 2

Convenciones de Datos y Auditoría

Elemento	Convención	Responsabilidad
id	UUID no nulo y PK; Java lo preasigna antes de persistir.	Hibernate no usa IDENTITY ni una secuencia para claves del dominio.
version	BIGINT no nulo, inicial 0, @Version en entidades editables.	JDBC aplica control optimista a usuario. Los parámetros y otras filas

Elemento	Convención	Responsabilidad
		históricas no llevan version; parametro_sistema admite solo INSERT.
creado_en / creado_por	Instante de now() de la BD y FK al usuario; actor nulo en catálogos, migración o alta inicial. fn_auditar: Actor NULL solo en M01 ACCESO_FALLIDO/ACCESO_BLOQUEADO, M05 MORA_AUTOMATICA/REVISION_VENCIMIENTO/INCIDENTE_H2 con detalle.origen='H2', o M09 REFERENCIA_ACTUALIZADA/REFERENCIA_EN_CUARENTENA/INCIDENTE_H3 con detalle.origen='H3'. autorizacion_tasa exige actor humano.	El servicio exige actor en operaciones de negocio. Los accesos fallidos guardan únicamente el SHA-256 truncado del nombre normalizado, nunca el nombre ingresado.
Baja lógica	activo/activa o estado Inactiva; sin DELETE de ejecución.	RN-11: comprobar contratos activos y unidades dependientes bajo bloqueo.
Textos	VARCHAR con límites del formulario; TEXT para hash PHC sin truncamiento.	Validación no vacía, longitud y normalización en servicios; CHECK de valores cerrados en base.
JSONB	Instantánea del recibo, parámetros tipados y detalle auditado.	Solo contenido estructurado del dominio; sin contraseñas, claves API ni rutas absolutas privadas.
Catálogos	Cada catálogo compartido tiene enum Java y AttributeConverter con etiqueta exacta; CHECK y rol protegen la representación SQL.	No editables por usuarios; cambios mediante migración revisada.
Ausencia de saldo mutable	Deuda, crédito y remanente se calculan desde movimientos.	Evita dos valores contables que deban sincronizarse; el bloqueo del contrato protege sumas.

Catálogos y Significado de los Estados

Los catálogos cerrados forman parte del contrato de datos y no constituyen parámetros que el Propietario pueda renombrar. La tabla rol conserva los dos valores asignables a usuarios; las otras listas usan CHECK para reducir tablas de consulta sin atributos propios. Un cambio de valor visible exige migración, convertidor y pruebas de compatibilidad con los datos previos.

Cargo y DepósitoGarantía no almacenan un estado financiero editable: las vistas lo derivan con la precedencia de RN-24 y RN-26. El pago sí conserva Registrado o Anulado, porque la anulación registra un acontecimiento administrativo y sus responsables. Todo catálogo compartido se representa mediante enum Java y AttributeConverter, sin cadenas sueltas en las entidades. EstadoUnidad contiene solo los tres valores persistidos; EstadoUnidadVisible añade OCUPADA para UnidadDto.

La Tabla 3 presenta catálogos Cerrados Compartidos.

Tabla 3

Catálogos Cerrados Compartidos

Catálogo	Valores Exactos
Monedas	NIO (C$), USD (US$)
Roles	Propietario, Asistente
Contrato	Borrador, Vigente, Por vencer, Vencido, Terminado, Cancelado
Unidad	Persistido: Disponible, En mantenimiento, Inactiva. Visible: añade Ocupada, derivada en v_ocupacion y UnidadDto.
Cargo	Pendiente, Parcial, Pagado, Vencido, Anulado
Pago	Registrado, Anulado
Solicitud de mantenimiento	Abierta, En proceso, Resuelta, Cancelada
Depósito	Retenido, Aplicado, Devuelto parcial, Devuelto
Tipo de cargo	Renta, Mora, Reparación, Servicio, Otro
Método de pago	Efectivo, Transferencia, Depósito bancario, Otro
Tipo de unidad	Casa, Apartamento, Cuarto, Local comercial

El estado Anulado prevalece sobre los demás estados de cargo, aunque su importe original permanezca para trazabilidad. Un saldo igual a cero produce Pagado; con saldo y vencimiento anterior al corte, el resultado es Vencido. Si todavía no vence, un abono produce Parcial y su ausencia produce Pendiente. La garantía tiene Retenido mientras conserve remanente sin devolución, y Devuelto parcial cuando exista devolución y aún quede remanente.

Agotada con alguna devolución, pasa a Devuelto; agotada solo con aplicaciones, pasa a Aplicado. La garantía de importe cero permanece Retenido como convención explícita, sin salida financiera disponible.

Claves, Borrado Lógico y Preferencias

Cada entidad recibe UUID para conservar la identidad opaca usada por el TRD y por las preferencias personales del escritorio. La aplicación lo asigna al construir el agregado y la base tiene gen_random_uuid() como respaldo para herramientas SQL autorizadas. Esa clave puede conocerse antes de persistir y no expresa orden de negocio. El número del recibo es BIGINT y procede de una fila bloqueada dentro de la misma transacción del pago. El diseño permite huecos entre UUID sin significado contable; la continuidad exigida por RN-07 corresponde únicamente a los números de recibos confirmados en la base activa.

Las bajas no borran filas referenciadas por contratos o movimientos y las claves foráneas usan ON DELETE RESTRICT. El Propietario desactiva una unidad o un inquilino solo después de comprobar la ausencia de contratos activos bajo el mismo bloqueo que emplea la activación. Las preferencias Claro, Oscuro y sin efectos pertenecen al almacén local del escritorio y se identifican por UUID de usuario después del acceso. La última selección local rige antes de autenticar. Un fallo de lectura usa Claro; un fallo de guardado conserva el cambio durante la sesión y comunica la incidencia sin bloquear operaciones financieras.

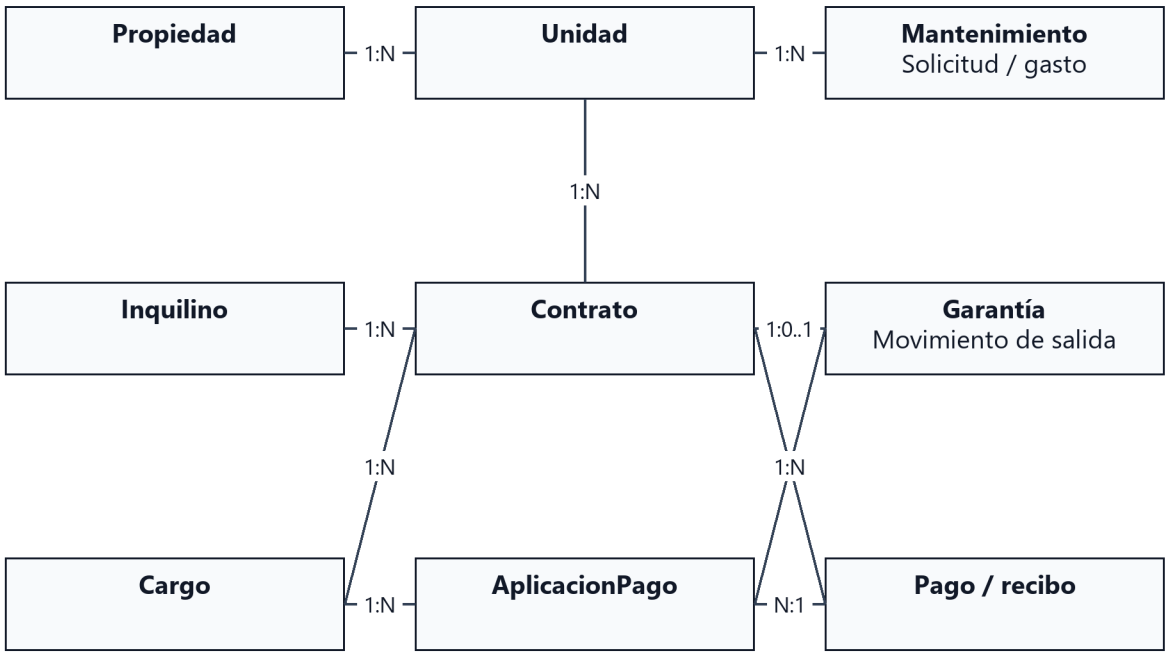
Modelo Conceptual

La unidad es el espacio que puede alquilarse y el contrato conserva las condiciones pactadas con un inquilino. Cada contrato origina cargos, recibe pagos y mantiene una garantía separada de la renta. Las aplicaciones resuelven la relación entre el dinero de origen y la obligación pagada, incluso cuando el pago deja crédito. Un recibo identifica un pago y congela el

contenido emitido; su estado administrativo depende de la anulación del pago. La Figura 1 agrupa estas relaciones sin confundir un gasto del propietario con un cargo trasladado al inquilino, pues RN-23 exige una decisión explícita para repercutirlo.

Figura 1

Relaciones Conceptuales del Dominio



Contrato origina cargos y recibe pagos; cada aplicación conserva su fuente.
Garantía aplicada: movimiento y aplicación equivalentes del mismo contrato.
Leyenda: 1:N, uno a muchos; 0..1, relación opcional única.
Una unidad admite una sola reserva activa. Un pago genera un recibo.

Una propiedad agrupa unidades y permite consolidar gastos, ocupación e ingresos por moneda. La solicitud de mantenimiento describe el trabajo y el gasto registra su costo confirmado; un cargo Reparación requiere otro registro con concepto autorizado. Los adjuntos identifican un solo propietario lógico entre propiedad, unidad, inquilino, contrato y versión de parámetro, con huella y ruta interna. El logotipo pertenece al registro de parametro_sistema cuya

clave es emisor. Usuario aporta el actor de creación y la bitácora reúne los eventos de una correlación. La tasa informativa permanece separada de la autorización de cobro, de modo que una actualización HTTP no reescribe pagos ni altera los documentos emitidos con anterioridad.

Modelo Lógico

El modelo lógico usa relaciones de uno a muchos mediante claves foráneas y resuelve los vínculos financieros mediante AplicacionPago. Pago y Recibo tienen una relación uno a uno, protegida por unicidad del pago en el recibo y por la validación de integridad antes del commit. Contrato y DepositoGarantia admiten una garantía por contrato, creada al activarlo. Los registros opcionales, como un contrato predecesor o la referencia informativa de una autorización, usan claves nulas con significado explícito. La tabla de relaciones incluye los vínculos de auditoría porque también deben conservar existencia y restricciones al borrar usuarios históricos.

La Tabla 4 presenta relaciones y Cardinalidades del Modelo.

Tabla 4

Relaciones y Cardinalidades del Modelo

Entidad Referida	Referencia en Entidad Dependiente	Cardinalidad	Regla
rol	usuario.rol_id	1 a N	RN-13
usuario	usuario.creado_por	1 a 0..N	RN-14
usuario	propiedad.creado_por	1 a 0..N	RN-14
propiedad	unidad.propiedad_id	1 a N	RN-11
usuario	unidad.creado_por	1 a 0..N	RN-14
usuario	inquilino.creado_por	1 a 0..N	RN-14
unidad	contrato.unidad_id	1 a N	RN-01
inquilino	contrato.inquilino_id	1 a N	RN-18
contrato	contrato.predecesor_id	1 a 0..N	RN-19
usuario	contrato.creado_por	1 a 0..N	RN-14
contrato	cargo.contrato_id	1 a N	RN-04

Entidad Referida	Referencia en Entidad Dependiente	Cardinalidad	Regla
cargo	cargo.cargo_origen_id	1 a 0..N	RN-05
usuario	cargo.anulado_por	1 a 0..N	RN-13
usuario	cargo.creado_por	1 a 0..N	RN-14
usuario	tipo_cambio.creado_por	1 a 0..N	RN-14
tipo_cambio	autorizacion_tasa.referencia_id	1 a 0..N	RN-22
usuario	autorizacion_tasa.creado_por	1 a 0..N	RN-14
contrato	pago.contrato_id	1 a N	RN-06
autorizacion_tasa	pago.autorizacion_id	1 a 0..N	RN-22
usuario	pago.anulado_por	1 a 0..N	RN-08
usuario	pago.creado_por	1 a 0..N	RN-14
contrato	deposito_garantia.contrato_id	1 a N	RN-10
usuario	deposito_garantia.creado_por	1 a 0..N	RN-14
deposito_garantia	movimiento_deposito.deposito_id	1 a N	RN-26
cargo	movimiento_deposito.cargo_id	1 a 0..N	RN-10
usuario	movimiento_deposito.creado_por	1 a 0..N	RN-14
pago	aplicacion_pago.pago_id	1 a 0..N	RN-06
movimiento_deposito	aplicacion_pago.movimiento_id	1 a 0..N	RN-10
cargo	aplicacion_pago.cargo_id	1 a N	RN-06
usuario	aplicacion_pago.revertida_por	1 a 0..N	RN-31
usuario	aplicacion_pago.creado_por	1 a 0..N	RN-14
pago	recibo.pago_id	1 a N	RN-07
usuario	recibo.creado_por	1 a 0..N	RN-14
unidad	solicitud_mantenimiento.unidad_id	1 a N	RN-23
usuario	solicitud_mantenimiento.creado_por	1 a 0..N	RN-14
propiedad	gasto_propiedad.propiedad_id	1 a N	RN-23
solicitud_mantenimiento	gasto_propiedad.solicitud_id	1 a 0..N	RN-23
usuario	gasto_propiedad.creado_por	1 a 0..N	RN-14
usuario	parametro_sistema.creado_por	1 a 0..N	RN-14
usuario	bitacora.creado_por	1 a 0..N	RN-14

Entidad Referida	Referencia en Entidad Dependiente	Cardinalidad	Regla
propiedad	documento_adjunto.propiedad_id	1 a 0..N	RN-25
unidad	documento_adjunto.unidad_id	1 a 0..N	RN-25
inquilino	documento_adjunto.inquilino_id	1 a 0..N	RN-25
contrato	documento_adjunto.contrato_id	1 a 0..N	RN-25
usuario	documento_adjunto.creado_por	1 a 0..N	RN-14
usuario	esquema_migracion.creado_por	1 a 0..N	RN-14
parametro_sistema	documento_adjunto.parametro_id	1 a 0..N	RN-25; logotipo de la versión emisor

Nota. La FK opcional permite 0..1 padre por fila dependiente; el padre admite 0..N dependientes. Las unicidades recibo.pago_id y deposito_garantia.contrato_id reducen N a 0..1.

Normalización y Dependencias

La primera forma normal separa cada aplicación de pago y cada movimiento de garantía en una fila, en vez de concatenar cargos o deducciones en un campo. Las claves técnicas identifican la fila completa y las unicidades expresan dependencias naturales adicionales. La segunda forma normal evita que atributos del contrato dependan de una parte de una asociación entre pago y cargo. AplicacionPago conserva el importe aplicado y su procedencia, mientras la moneda corresponde al contrato. La tercera forma normal separa rol, propiedad e inquilino de sus referencias y elimina copias editables de sus nombres dentro de cada movimiento financiero.

La tasa_usada y la instantánea del recibo son desnormalizaciones deliberadas de hechos históricos. La autorización conserva un valor por fecha, pero el pago también guarda la tasa aplicada para explicar su conversión sin depender de una edición posterior. El recibo conserva los datos visibles del emisor y del inquilino en el momento de emitirlo, aunque sus fichas cambien después. Estas copias son inmutables y tienen una finalidad documental distinta de un

dato maestro editable. Deuda, crédito y remanente no se almacenan como totales acumulados; sus consultas calculan el resultado desde los movimientos vigentes y sus reversiones.

La Tabla 5 presenta dependencias Funcionales y Desnormalizaciones.

Tabla 5

Dependencias Funcionales y Desnormalizaciones

Relación	Dependencia	Decisión
Usuario	id determina nombre, rol y credencial; lower(nombre_usuario) también identifica.	Rol se consulta por FK y no se copia como texto editable.
Unidad	(propiedad_id,codigo) determina unidad.	Canon de referencia no sustituye canon contractual.
Renta	(contrato_id,periodo) determina cargo Renta, incluso anulado.	Índice parcial conserva identidad de facturación histórica.
Recibo	numero y pago_id identifican por separado el recibo.	Instantánea JSONB inmutable para reimpresión.
Pago	solicitud_id determina resultado idempotente y solicitud_hash.	Congela monto original, equivalente y tasa.
Saldos	Movimiento confirmado y aplicación determinan abonos.	Vistas derivadas; sin columna de saldo actual editable.
Autorización	fecha_efectiva determina una autorización de cobro.	Sin corrección destructiva: una vez usada, no se reemplaza. Una autorización errónea bloquea cobros de esa fecha hasta resolver los pagos afectados mediante un procedimiento de migración autorizado.
Parámetro	(clave,vigente_desde) determina el registro histórico y su valor.	Solo INSERT; vigente = última fila por clave con vigente_desde ≤ now(); sin version.

Crédito, Garantía y Estados Derivados

Cada pago registrado dispone de un crédito igual al monto convertido menos la suma de sus aplicaciones no revertidas. La vista v_credito_pago conserva ese origen, por lo que un contrato con varios pagos mantiene tantos remanentes como pagos que los produjeron. H1 los aplica al generar el siguiente cargo con orden de fecha e identificador, bajo bloqueo del contrato

y de los pagos afectados. La aplicación posterior se identifica como Credito y no genera otro recibo. Al anular el pago de origen, el servicio exige revertir antes esas aplicaciones posteriores, con motivo, usuario e instante que expliquen la devolución del saldo.

La garantía no participa en el cálculo de crédito del pago ni en ingresos por renta al recibirse. Una salida de tipo Aplicacion referencia un cargo del mismo contrato y tiene una AplicacionPago equivalente de origen Garantia. Una Devolucion no referencia cargo y documenta el concepto de la salida al inquilino. El servicio comprueba que la suma de salidas no supera el importe inicial y que la liquidación pertenece al cierre autorizado por el Propietario. El movimiento de garantía no consume el consecutivo comercial de pagos; su comprobante usa su UUID como referencia administrativa separada para evitar una clasificación incorrecta del ingreso.

Modelo Físico y Diccionario de Datos

El modelo físico contiene veintitrés tablas, diez vistas y siete rutinas, seis funciones y un procedimiento. Las tablas adicionales a las entidades mínimas resuelven autorización de tasa, exclusión de mantenimiento, presupuesto HTTP y control de migraciones. El diccionario resumido identifica su finalidad y el Apéndice D detalla cada columna con tipo, nulabilidad, clave, valor predeterminado, restricciones y regla. Las figuras por dominio usan los mismos nombres físicos del DDL y muestran claves y referencias principales. Las columnas comunes de creación y versión se documentan completas en el diccionario para evitar abreviaturas que oculten su presencia en las tablas.

La Tabla 6 presenta inventario de Entidades Físicas.

Tabla 6

Inventario de Entidades Físicas

Tabla	Finalidad y Regla	Persistencia
rol	Roles funcionales inalterables de la aplicación. RN-13	JDBC
usuario	Identidad de acceso; no representa al inquilino. RN-15	JDBC
propiedad	Agrupa unidades y gastos del propietario. RN-11	Hibernate
unidad	Espacio alquilable individual. RN-01	Hibernate
inquilino	Persona sintética en demostración; no inicia sesión. RN-18	Hibernate
contrato	Condiciones históricas del alquiler y reserva de unidad. RN-01	Hibernate
cargo	Obligación en moneda contractual; estado financiero derivado. RN-04	Hibernate
tipo_cambio	Versiones de caché informativa; nunca cambian un pago histórico. RN-22	Hibernate
autorizacion_tasa	Tasa de cobro por fecha; creada solo por Propietario. RN-09	Hibernate
pago	Ingreso de renta; importes y tasa quedan congelados. RN-06	Hibernate
deposito_garantia	Garantía inicial; estado y remanente derivados de movimientos. RN-10	Hibernate
movimiento_deposito	Aplicación o devolución de garantía, sin consumo de consecutivo de pago. RN-26	Hibernate
aplicacion_pago	Abono a un cargo procedente de pago o garantía. RN-06	Hibernate
consecutivo_recibo	Única fila de control transaccional de recibos. RN-07	JDBC
recibo	Instantánea inmutable de emisión; anulación se consulta en Pago. RN-08	Hibernate
solicitud_mantenimiento	Incidencia y ciclo operativo de mantenimiento. RN-27	Hibernate
gasto_propiedad	Egreso del propietario; no crea cargo al inquilino. RN-23	Hibernate
parametro_sistema	Parámetros administrativos con historial de solo inserción; vigente = última fila por clave con vigente_desde ≤ now(). RN-13	JDBC
bitacora	Registro de eventos sin UPDATE ni DELETE para	JDBC

Tabla	Finalidad y Regla	Persistencia
	ejecución. RN-14	
documento_adjunto	Metadatos de archivo controlado; un único propietario lógico. RN-25	Hibernate
puerta_mantenimiento	Fila de exclusión para escrituras y respaldo de adjuntos. RN-29	JDBC
control_fx	Concesión y presupuesto HTTP compartidos en LAN. RN-22	JDBC
esquema_migracion	Versiones SQL aplicadas y suma de comprobación. RN-29	JDBC

Las Figuras 2 a 5 distribuyen el esquema en cuatro dominios para conservar legibilidad en papel carta. Los recuadros identifican la clave primaria y las referencias sin repetir todos los atributos descriptivos. Las referencias hacia otro dominio conservan su nombre de destino y se consultan con la tabla lógica de relaciones. El modelo evita referencias polimórficas sin FK en DocumentoAdjunto: cinco columnas opcionales (propiedad_id, unidad_id, inquilino_id, contrato_id y parametro_id) y una comprobación num_nonnulls exigen un solo propietario. La relación genérica entidad_id de Bitacora es una excepción deliberada para registrar también acciones sobre objetos que no nacieron por un intento fallido o sobre operaciones administrativas. La Figura 4 se actualizó conforme a las decisiones canónicas; las Figuras 2, 3 y 5 conservan sus imágenes originales revisadas. El DDL y el diccionario corregidos son la fuente de verdad.

Figura 2

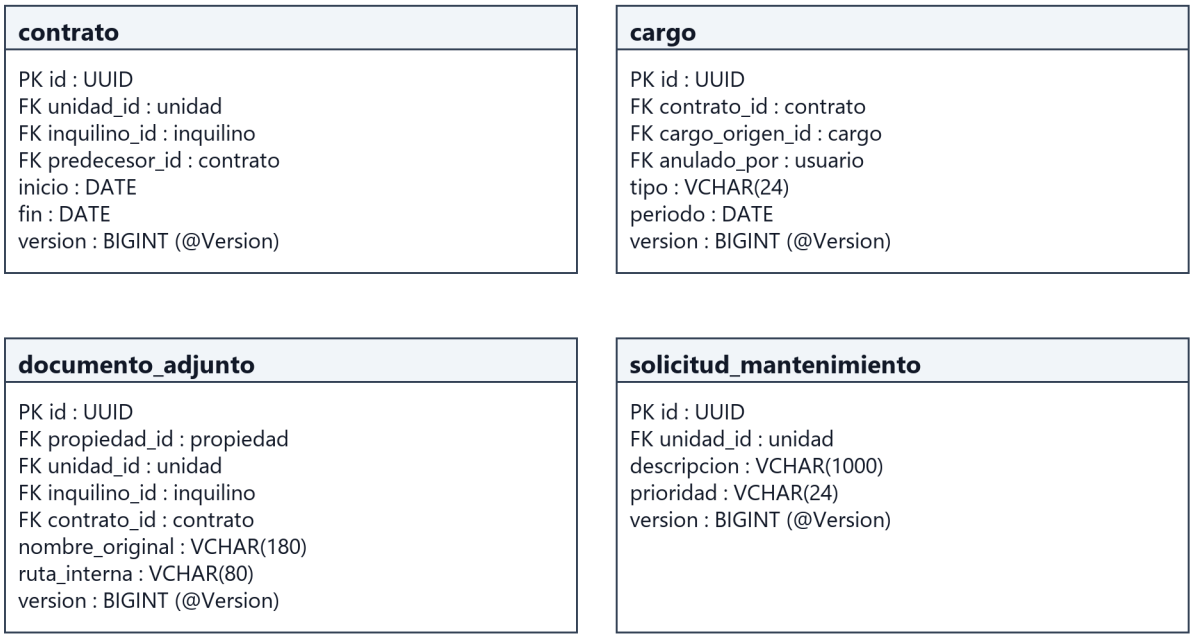
Modelo Físico de Identidad e Inventario



Leyenda: PK, clave primaria; FK, referencia N:1 salvo unicidad en DDL.
Los destinos fuera del dominio se identifican por nombre de tabla.

Figura 3

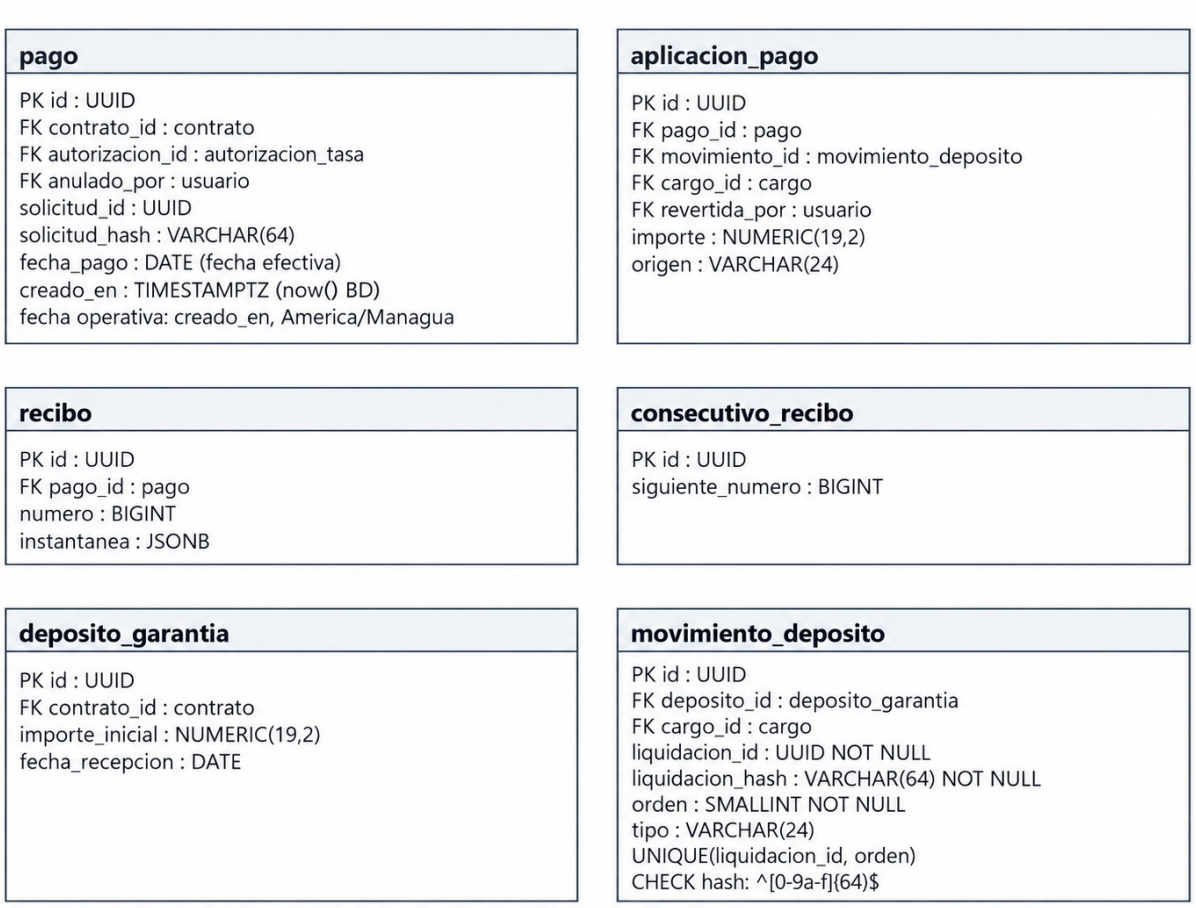
Modelo Físico de Contratación y Expedientes



Leyenda: PK, clave primaria; FK, referencia N:1 salvo unicidad en DDL.
Los destinos fuera del dominio se identifican por nombre de tabla.

Figura 4

Modelo Físico de Pagos y Garantías



Leyenda: PK, clave primaria; FK, referencia N:1 salvo unicidad en DDL.
Los destinos fuera del dominio se identifican por nombre de tabla.

Figura 5

Modelo Físico de Integración y Operación



Leyenda: PK, clave primaria; FK, referencia N:1 salvo unicidad en DDL.
Los destinos fuera del dominio se identifican por nombre de tabla.

Reglas de Integridad y Restricciones

La base protege existencia, dominio, claves y unicidades mediante restricciones declarativas, mientras los servicios verifican permisos, transiciones y ecuaciones que combinan varias filas. PostgreSQL advierte que CHECK no es el mecanismo para consultar otras filas o tablas; por ello, las sumas de aplicaciones requieren bloqueo y validación antes del commit (PGDG, s. f.-b). El procedimiento sp_verificar_integridad aporta una comprobación central invocable sobre la conexión transaccional. Su uso es obligatorio después del flush financiero y

antes de confirmar, pero no sustituye al control de permisos del servicio ni convierte una credencial SQL compartida en aislamiento entre usuarios del sistema.

La Tabla 7 presenta restricciones de Integridad de Mayor Impacto.

Tabla 7

Restricciones de Integridad de Mayor Impacto

Nombre	Tipo y Tabla	Expresión	RN
uq_unidad_codigo	UNIQUE; unidad	UNIQUE(propiedad_id,codigo)	RN-16
ck_contrato_duracion	CHECK; contrato	fin >= (inicio + INTERVAL '1 month')::date	RN-02
uq_autorizacion_fecha	UNIQUE; autorizacion_tasa	UNIQUE(fecha_efectiva)	RN-22
uq_pago_solicitud	UNIQUE; pago	UNIQUE(solicitud_id)	RN-21
ck_pago_tasa	CHECK; pago	(tasa_usada IS NULL) = (autorizacion_id IS NULL)	RN-09
ck_pago_anulacion	CHECK; pago	(estado='Registrado' AND anulado_en IS NULL AND anulado_por IS NULL AND motivo_anulacion IS NULL) OR (estado='Anulado' AND anulado_en IS NOT NULL AND anulado_por IS NOT NULL AND length(trim(motivo_anulacion))>0)	RN-08
uq_deposito_contrato	UNIQUE; deposito_garantia	UNIQUE(contrato_id)	RN-10
ck_movimiento_destino	CHECK; movimiento_deposito	(tipo='Aplicacion' AND cargo_id IS NOT NULL AND metodo IS NULL) OR (tipo='Devolucion' AND cargo_id IS NULL AND metodo IS NOT NULL)	RN-10
ck_aplicacion_fuente	CHECK; aplicacion_pago	(pago_id IS NOT NULL AND movimiento_id IS NULL AND origen IN ('Inicial','Credito')) OR (pago_id IS NULL AND movimiento_id IS NOT NULL AND origen='Garantia')	RN-06
uq_aplicacion_movimiento	UNIQUE; aplicacion_pago	UNIQUE(movimiento_id)	RN-10

Nombre	Tipo y Tabla	Expresión	RN
ck_consecutivo_unico	CHECK; consecutivo_recibo	id='00000000-0000-0000-0000-000000000001'::uuid	RN-07
uq_recibo_pago	UNIQUE; recibo	UNIQUE(pago_id)	RN-07
uq_recibo_numero	UNIQUE; recibo	UNIQUE(numero)	RN-07
ck_documento_adjunto_ruta_interna	CHECK; documento_adjunto	ruta_interna ~ '^[0-9a-f-]{36}[.](pdf|png|jpg)$'	RN-25
ck_documento_adjunto_tamano_bytes	CHECK; documento_adjunto	tamano_bytes BETWEEN 1 AND 5242880	RN-25
ck_documento_duenio	CHECK; documento_adjunto	num_nonnulls(propiedad_id,unidad_id,inquilino_id,contrato_id,parametro_id)=1	RN-25
ck_control_fx_intentos_mes	CHECK; control_fx	intentos_mes BETWEEN 0 AND 100	RN-22
uq_migracion_nombre	UNIQUE; esquema_migracion	UNIQUE(nombre)	RN-29
uq_usuario_nombre	Índice único; usuario	lower(nombre_usuario)	RN-16
uq_contrato_activo	Índice único parcial; contrato	unidad_id WHERE estado IN ('Vigente','Por vencer','Vencido')	RN-01
uq_cargo_renta	Índice único parcial; cargo	contrato_id,periodo WHERE tipo='Renta'	RN-04
uq_cargo_mora	Índice único parcial; cargo	cargo_origen_id WHERE tipo='Mora'	RN-05

Nota. El Apéndice D incluye el catálogo íntegro de restricciones. Las PK y FK no listadas aquí se generan explícitamente en el Apéndice A.

Reserva de Unidad y Facturación Única

uq_contrato_activo impide que una unidad tenga dos contratos en Vigente, Por vencer o Vencido. El servicio bloquea primero la unidad y relee sus reservas antes de activar, renovar o dar de baja. Se adopta una reserva única más estricta que una exclusión por intervalos: un

sucesor futuro permanece Borrador hasta terminar al anterior dentro de una transacción. Esa precisión conserva RN-01 del PRD y evita que Vencido libere la unidad por una fecha sin cierre. Los cambios de estado posteriores a la fecha final no eliminan cargos ni modifican el canon de las obligaciones ya confirmadas.

uq_cargo_renta identifica una renta por contrato y primer día del período mensual, aun cuando el cargo se anule. H1 confirma por contrato y omite esa combinación en un reintento, conforme a RN-20. La corrección de una renta anulada no emite otra fila tipo Renta para el mismo período; crea un cargo Otro con referencia al cargo corregido y concepto explícito, o corrige movimientos de pago sin alterar el principal. uq_cargo_mora impide una segunda mora para la renta de origen. El servicio exige que ese origen sea Renta del mismo contrato y usa la política vigente al calcularla. Cada evento de Mora conserva {politica_id, tipo, valor, base} con la política vigente usada en el cálculo.

La Tabla 8 presenta responsabilidades de Integridad por Capa.

Tabla 8

Responsabilidades de Integridad por Capa

Invariante	Base de Datos	Servicio y Momento
RN-01, RN-11	Índice activo, FK RESTRICT.	Bloquea unidad y valida propiedad/inquilino activos antes de alta, activación o baja.
RN-05	Índice uq_cargo_mora y origen obligatorio.	Mora solo sobre Renta, principal pendiente tras gracia; guarda importe calculado sin capitalizar.
RN-06, RN-26	Importes positivos, FK y procedimiento de sumas.	Bloquea contrato, cargos y fuentes; no excede saldo, crédito ni garantía.
RN-07	Contador de una fila y dos unicidades en Recibo.	Reserva número al final, persiste pago y recibo, valida y confirma juntos.
RN-08, RN-31	Permisos de columnas; recibo sin UPDATE.	Propietario exige motivo y ausencia de aplicaciones posteriores activas antes de anular.

Invariante	Base de Datos	Servicio y Momento
RN-09, RN-22, RN-32	Autorización por fecha, tasa positiva y procedimiento de conversión.	Tasa aprobada por Propietario; HALF_EVEN y comprobación de fecha local.
RN-13 a RN-15	Roles de PostgreSQL, actor FK, bitácora append-only.	Rol funcional por sesión; hash Argon2id; bloqueo por intentos serializado por usuario.
RN-19, RN-27	CHECK de estado y fecha de cierre.	Matriz de transiciones, autorización de liquidación y renovación como nuevo contrato.
RN-21	UUID único y huella del comando.	Recupera el resultado; reutilización con otra huella produce conflicto, sin nuevo número.
RN-25, RN-29	Tamaño, ruta, propietario único; puerta global.	Firma de archivo, SHA-256, publicación controlada, respaldo de base más adjuntos.

Correcciones, Bajas y Datos Inmutables

Los recibos y los importes de pagos confirmados no reciben actualizaciones de negocio. El rol de ejecución solo actualiza columnas de anulación del pago y columnas de reversión de la aplicación; el contenido del recibo permanece intacto. v_recibo combina su instantánea con el estado del pago para reimprimir una copia marcada Anulado. La historia conserva número, importe y procedencia aunque ya no afecte el saldo. Los cargos admiten anulación mediante servicio autorizado, pero ese permiso no habilita editar su principal después de confirmar. La restricción de inmutabilidad del cargo depende también del servicio porque su tabla conserva UPDATE para Hibernate y cambios autorizados.

Las correcciones financieras producen una nueva operación identificable y enlazada con la referencia original en bitácora. El equipo adopta una autorización única de tasa por fecha para impedir valores de cobro incompatibles durante la misma jornada. La aplicación no ofrece edición destructiva de esa autorización; una equivocación requiere aislar la fecha afectada y resolver los pagos mediante anulación y migración revisada por el Propietario y el responsable de datos. Ese límite evita presentar una corrección de tasa como un cambio retroactivo

silencioso. Los estados de contrato y mantenimiento siguen siendo editables bajo control optimista y reglas de transición del servicio.

Índices y Rendimiento

Los índices combinan las unicidades de negocio con los recorridos de cobranza, historial y auditoría. PostgreSQL permite índices parciales que solo incluyen filas que cumplen un predicado y crea índices para claves primarias o restricciones únicas; Latch no duplica esos índices por convención (PGDG, s. f.-e). Los índices de FK añadidos responden a filtros o uniones frecuentes, sin imponer uno para cada columna de auditoría. Una consulta de cartera filtra contrato y fecha de vencimiento antes de proyectar resultados. La paginación usa fecha o nombre más UUID como desempate para que filas iguales no cambien de posición entre páginas estables.

La Tabla 9 presenta índices Adicionales y Consultas Atendidas.

Tabla 9

Índices Adicionales y Consultas Atendidas

Índice y Tabla	Columnas y Predicado	Tipo	Consulta o Regla
uq_usuario_nombre; usuario	lower(nombre_usuario)	B-tree único	Buscar cuenta sin distinguir mayúsculas. RN-16
uq_contrato_activo; contrato	unidad_id WHERE estado IN ('Vigente','Por vencer','Vencido')	B-tree único	Impedir segunda reserva activa, aun con fecha vencida. RN-01
uq_cargo_renta; cargo	contrato_id,periodo WHERE tipo='Renta'	B-tree único	Idempotencia de H1, incluye anulados. RN-04
uq_cargo_mora; cargo	cargo_origen_id WHERE tipo='Mora'	B-tree único	Una mora por renta, incluye anuladas. RN-05
ix_unidad_propiedad; unidad	propiedad_id	B-tree	Listado por propiedad. RN-11
ix_contrato_inquilino; contrato	inquilino_id	B-tree	Historial del inquilino. RN-18
ix_contrato_fin;	fin WHERE estado IN	B-tree	Alertas de vencimiento.

Índice y Tabla	Columnas y Predicado	Tipo	Consulta o Regla
contrato	('Vigente','Por vencer','Vencido')		RN-12
ix_contrato_predecesor; contrato	predecesor_id	B-tree	Renovaciones. RN-19
ix_cargo_cobranza; cargo	contrato_id,vence_el,id WHERE anulado_en IS NULL	B-tree	Asignación cronológica de cargos. RN-06
ix_pago_historial; pago	contrato_id,fecha_pago	B-tree	Historial y cortes. RN-28
ix_pago_autorizacion; pago	autorizacion_id	B-tree	Trazabilidad de conversión. RN-09
ix_aplicacion_cargo; aplicacion_pago	cargo_id,fecha_aplicacion	B-tree	Saldo por cargo y corte. RN-24
ix_aplicacion_pago; aplicacion_pago	pago_id WHERE revertida_en IS NULL	B-tree	Crédito y dependencias de anulación. RN-31
ix_movimiento_deposito; movimiento_deposito	deposito_id,fecha_movimiento	B-tree	Liquidación por garantía. RN-26
ix_mantenimiento_unidad; solicitud_mantenimiento	unidad_id,estado	B-tree	Solicitudes pendientes. RN-27
ix_gasto_propiedad; gasto_propiedad	propiedad_id,fecha_gasto	B-tree	Egresos por período. RN-23
ix_bitacora_fecha; bitacora	creado_en,id	B-tree	Paginación por fecha e id. RN-14
ix_bitacora_correlacion; bitacora	correlacion_id	B-tree	Revisión de operación atómica. RN-14
ix_tasa_actualizacion; tipo_cambio	actualizado_en DESC	B-tree	Última referencia HTTP. RN-22

Volumetría y Crecimiento

La carga de aceptación conserva V1 del PRD: cien propiedades, quinientas unidades, mil inquilinos, dos mil contratos históricos, diez mil cargos y cien mil eventos (Cuadra et al., 2026c). Es una carga sintética para medir consultas, sin afirmar cuántas unidades administrará un propietario real. La proyección de crecimiento supone quinientos contratos ocupados y doce

rentas anuales por contrato, equivalentes a seis mil rentas al año. Si cada renta recibe dos abonos y cada abono una aplicación, el ejercicio añade doce mil pagos, recibos y aplicaciones anuales. Las reparaciones, reversiones y la mora incrementan esos conteos según los escenarios configurados.

La Tabla 10 presenta volúmenes Propuestos y Metas de Aceptación.

Tabla 10

Volúmenes Propuestos y Metas de Aceptación

Objeto o Consulta	Supuesto o Meta	Método y Límite
Carga V1	100 propiedades; 500 unidades; 1000 inquilinos; 2000 contratos; 10000 cargos; 100000 eventos.	Fixture sintético para RNF-001 a RNF-004; T001 también queda pendiente de reejecución.
Rentas anuales	500 contratos × 12 = 6000 rentas.	Supuesto de ocupación completa; no pronóstico comercial.
Pagos y recibos	6000 × 2 = 12000 de cada uno por año.	Dos abonos por renta; anulaciones conservan filas y número.
Aplicaciones	12000 iniciales más 20 % de movimientos de crédito = 14400/año.	Supuesto de dimensionamiento; medir conteo real al operar.
Bitácora	5 eventos por operación y 30000 operaciones/año = 150000 eventos/año.	Reservar 500 MiB/año como presupuesto de diseño con índices; medir pg_total_relation_size.
Adjuntos	1000 archivos promedio 1 MiB = 1000 MiB; máximo 5 MiB por archivo.	Datos sintéticos; fuera de la base y dentro del respaldo coordinado.
Panel y búsqueda	RNF-001 p95 ≤2 s; RNF-002 p95 ≤1 s.	30 muestras, V1, consulta de 50 filas; no medido en este documento.
Generación y exportación	RNF-003 500 cargos ≤60 s; RNF-004 10000 filas XLSX ≤30 s.	Cinco lotes; JavaFX/Hibernate/Jasper requieren implementación posterior.
Crecimiento	Revisar a 1 millón de eventos o 5 GiB de base.	Medir planes, cardinalidad y espacio; no particionar la primera versión.

Mantenimiento y Planes de Consulta

El mantenimiento conserva autovacuum y estadísticas actualizadas; no se programa VACUUM FULL durante la operación normal. PostgreSQL necesita recuperar espacio de

versiones obsoletas y revisar estadísticas de sus tablas, tareas que puede atender el daemon de mantenimiento (PGDG, s. f.-n). El responsable de datos revisará consultas lentas, filas estimadas y espacio de índices antes de cambiar parámetros globales. No se fija un tamaño real por fila sin medición de los valores almacenados. Los presupuestos de espacio incluyen margen para respaldo y adjuntos, mientras las alertas del escritorio impedirán publicar un paquete parcial cuando el disco no permita completarlo.

El análisis previsto usa EXPLAIN (ANALYZE, BUFFERS) con consultas de lectura y datos V1 en una base de prueba. PostgreSQL ejecuta la consulta cuando se solicita ANALYZE; por eso las sentencias de escritura requieren transacción de ensayo con rollback o un entorno desechable (PGDG, s. f.-s). La evidencia guardará consulta, parámetros sintéticos, plan y duración, además de revisión de esquema y equipo. El diseño no exige que el plan elija un índice con pocas filas, pues un recorrido secuencial puede costar menos. Una meta incumplida obliga a revisar el filtro y la carga antes de añadir nuevos índices.

Objetos Programables

Las vistas ofrecen una fuente compartida para panel y reportes sin repetir ecuaciones en plantillas. fn_cargos_corte recibe una fecha y calcula obligaciones, abonos y estado al cierre del día local; las vistas sin parámetro usan la fecha vigente de America/Managua. El diseño mantiene versiones e instantes de reversión para reconstruir qué movimientos estaban confirmados en el corte. Un dato cargado después del corte no modifica retroactivamente un reporte que exige conocimiento registrado a esa fecha. El contrato del estado de cuenta identifica tanto la fecha de negocio como este criterio de registro para que una captura tardía no aparezca como información conocida antes de ingresarse.

La Tabla 11 presenta vistas del Esquema.

Tabla 11

Vistas del Esquema

Objeto	Contenido	Uso y Regla
v_estado_cuenta	Cargo, contrato, moneda, importe, abonado, saldo, estado.	M05, M10; RN-24, RN-28.
v_saldo_contrato	Deuda por contrato y moneda; incluye contratos sin cargos con cero.	M10, M11; RN-06.
v_cartera_vencida	Cargos con saldo y fecha vencida; días de atraso.	M05, M10; RN-05.
v_ocupacion	Una fila por unidad; reserva activa prevalece como Ocupada.	M02, M11; RN-01.
v_ingresos_mes	Cobros registrados por mes, propiedad y moneda original.	M10; RN-28; no incluye garantía recibida.
v_contratos_por_vencer	Contratos no vencidos dentro del umbral configurado.	M04, M11; RN-12.
v_credito_pago	Remanente disponible por pago registrado y moneda contractual.	M05, M06; RN-06, RN-31.
v_depositos	Inicial, remanente y estado derivado de garantía.	M07, M10; RN-26.
v_recibo	Instantánea inmutable y estado administrativo del pago.	M06, M10; RN-08.
v_gastos_mes	Gastos del propietario por mes, propiedad y moneda.	M08, M10; RN-23.

Funciones y Procedimiento de Verificación

PostgreSQL distingue funciones que retornan valores de procedimientos invocados con CALL; Latch usa ambos sin efectuar commit dentro de sus rutinas (PGDG, s. f.-d; PGDG, s. f.-f). El servicio conserva la frontera de transacción para integrar Hibernate y JDBC en la misma conexión. Las funciones de lectura se declaran STABLE y las que toman bloqueos o escriben son VOLATILE; fn_redondear_par y fn_fin_corte son IMMUTABLE. Cada rutina fija search_path cuando resuelve tablas y utiliza SECURITY INVOKER. El Apéndice B conserva

sus definiciones completas y el cierre de la transacción corresponde al caso de uso que las invoca.

La Tabla 12 presenta contratos de Rutinas SQL y PL/pgSQL.

Tabla 12

Contratos de Rutinas SQL y PL/pgSQL

Rutina e Invocación	Parámetros y Retorno	Justificación
SELECT fn_fin_corte(?)	DATE → TIMESTAMPTZ.	Límite exclusivo del día en America/Managua para reportes.
SELECT fn_redondear_par(?,?)	NUMERIC, INTEGER 0..8 → NUMERIC.	Reproduce HALF_EVEN; round(numeric) de PostgreSQL no sustituye la política RN-32.
SELECT * FROM fn_cargos_corte(?)	DATE → TABLE con diez columnas documentadas.	Lectura financiera a corte; sin escrituras ni obtención de tasa.
SELECT fn_puerta_escritura()	Sin parámetros → VOID; toma FOR SHARE en puerta.	Coordina toda escritura con el respaldo global, RN-29.
SELECT fn_reservar_numero()	Sin parámetros → BIGINT; SELECT FOR UPDATE e incremento.	Helper H4 sobre la conexión de Hibernate; no emite por sí solo un recibo.
SELECT fn_auditar(?,?,?,?,?,?,?)	UUID actor, UUID correlación, VARCHAR módulo/acción/entidad, UUID id, JSONB → UUID.	JDBC parametrizado inserta evento en la transacción del caso de uso. Actor NULL solo en M01 ACCESO_FALLIDO/ACCESO_BLOQUEADO, M05 MORA_AUTOMATICA/REVISION_VENCIMIENTO/INCIDENTE_H2 con detalle.origen='H2', o M09 REFERENCIA_ACTUALIZADA/REFERENCIA_EN_CUARENTENA/INCIDENTE_H3 con detalle.origen='H3'. autorizacion_tasa exige actor humano.
CALL sp_verificar_integridad(?)	UUID contrato → sin retorno; SQLSTATE 23514, 23503.	Comprueba sumas, contrato de aplicaciones, conversión, origen de mora y existencia de recibo tras flush.

El procedimiento valida la ecuación financiera sobre un contrato que el servicio ya mantiene bloqueado. Comprueba que un abono no supere el origen ni el destino, que las aplicaciones de garantía tengan movimiento equivalente y que el pago tenga recibo. También contrasta fecha y valor de autorización con la conversión almacenada y rechaza mora cuyo origen no sea Renta del mismo contrato. Las reglas de permiso, firma de archivos y transición permanecen en Java, porque dependen del contexto operativo. El procedimiento no es un disparador automático; una herramienta con credenciales de escritura que omita el contrato de servicios puede eludir esas validaciones entre filas.

Decisión sobre Disparadores y Cambios de Estado

La primera versión no define disparadores de negocio ni de auditoría. El servicio escribe la bitácora mediante PreparedStatement dentro de la conexión que ya participa en la transacción y registra la correlación del comando. Esta elección mantiene visible el orden de validación, evita dos emisores del mismo evento y conserva la distribución RA2 y RA4. El script limita UPDATE y DELETE en bitácora, pero no detecta toda manipulación de un administrador de PostgreSQL. La auditoría tiene como frontera las operaciones de la aplicación y no se presenta como evidencia inviolable frente a quien controla el servidor o sus respaldos.

H2 actualiza estados contractuales mediante entidades y transacciones cortas, con el mismo bloqueo de contrato que H1 utiliza para generar cargos. El estado de cargo se calcula al consultar, por lo que no necesita un hilo que cambie Pendiente a Vencido en cada fila. La mora sí crea un nuevo cargo una sola vez después del plazo de gracia, siempre sobre principal pendiente. Un porcentaje inicial cero impide emitir una mora de importe cero y deja intacta la renta. Cambiar la política solo afecta cálculos futuros; el importe de una mora confirmada se conserva como hecho financiero histórico del contrato.

Concurrencia y Transacciones en la Base de Datos

Las escrituras operan en READ COMMITTED con bloqueos explícitos de los agregados que protegen sumas financieras. PostgreSQL aplica MVCC, por lo que una lectura ordinaria observa versiones confirmadas sin sustituir los bloqueos necesarios para coordinar decisiones entre varias filas (PGDG, s. f.-r). Cada servicio abre una Session por caso de uso y cada hilo tiene su propia Session y Connection. Para reportes de varias consultas, el servicio toma REPEATABLE READ READ ONLY, materializa DTO y libera la conexión antes de renderizar. Una sola consulta parametrizada obtiene su instantánea sin mantener una transacción durante la exportación del archivo.

La Tabla 13 presenta fronteras de Transacción por Operación.

Tabla 13

Fronteras de Transacción por Operación

Caso	Bloqueos y Persistencia	Confirmación
M01 acceso y edición	Fila de usuario al cambiar intentos o rol; JDBC. Hash fuera del bloqueo; versión al revalidar.	Intentos y evento juntos; un login exitoso no ignora un bloqueo concurrente.
M04 activar o terminar con sucesor	Puerta compartida; unidad; contratos UUID; garantía; Hibernate y auditoría JDBC.	Estado, nueva garantía y reserva confirman juntos. Renovar crea el sucesor Borrador desde Vigente o Por vencer. Terminar permite el cierre atómico del anterior y la activación del sucesor desde Vigente, Por vencer o Vencido, sin ventana de doble reserva.
H1 generación mensual	Puerta; contrato; cargos; pagos origen de crédito; aplicaciones; Hibernate.	Una transacción por contrato; cancelación entre contratos.
H2 mora y vencimiento	Puerta; unidad si cambia reserva; contrato; renta de origen.	Sin mora duplicada; no se capitaliza Mora.
M06 pago	Puerta; unidad si requerida; contrato; cargos por vencimiento/id; pagos origen; contador al final.	Pago, aplicaciones, recibo y bitácora en un commit.

Caso	Bloqueos y Persistencia	Confirmación
M06 anulación	Puerta; contrato; cargos y aplicaciones; pago; contador no se altera.	Reversión y motivo confirman juntos; rechaza dependencias de crédito activas.
M07 liquidación	Puerta; unidad/contrato; cargos; depósito; aplicaciones.	Salidas, abonos, cierre y auditoría confirman juntos.
H3 HTTP	Transacción corta reserva concesión; sin conexión durante GET; otra transacción guarda referencia.	Caché y programación coordinadas; HTTP nunca retiene bloqueos financieros.
M10/M11 consulta	Snapshot solo lectura para obtener DTO.	Cierra antes de Jasper, interacción de usuario o escritura de archivos.
M12 respaldo	Puerta exclusiva en conexión coordinadora; pg_dump en otra conexión.	La puerta se libera después de copiar adjuntos y terminar manifiesto.

Consecutivo sin Saltos y Confirmación Incierta

La fila de consecutivo se inicializa con siguiente_numero igual a uno y un UUID fijo protegido por CHECK. fn_reservar_numero hace SELECT FOR UPDATE, incrementa el valor y devuelve el número reservado. El orden canónico es flush del pago y sus aplicaciones → fn_reservar_numero() → persistir Recibo @Immutable y hacer flush → auditoría JDBC en la misma conexión → sp_verificar_integridad → commit. Una reversión devuelve el contador al valor previo porque su UPDATE es transaccional. PostgreSQL documenta que nextval no revierte su consumo al deshacer la transacción; por ese motivo una secuencia, SERIAL o IDENTITY no satisface RN-07 para numeración comercial (PGDG, s. f.-p). La reserva, el pago, las aplicaciones, el recibo y la auditoría comparten una Session y una conexión.

El ReentrantLock justo de H4 coordina los hilos del mismo proceso y abarca la confirmación, según ADR-16. El bloqueo de fila PostgreSQL protege además la emisión desde distintos equipos, por lo que la corrección no depende del candado local. El servicio toma el contador al final para reducir tiempo en la sección crítica y nunca muestra el número como emitido antes del commit. La generación del PDF ocurre después de confirmar y su falla solo exige reimprimir. La anulación mantiene el recibo en la secuencia; el formato visible usa al

menos seis dígitos sin truncar números de siete o más posiciones. Los diez emisores en dos JVM son hilos de prueba; el máximo de tres sesiones LAN limita clientes de aplicación.

La Tabla 14 presenta intercalación de Dos Usuarios al Emitir Recibos.

Tabla 14

Intercalación de Dos Usuarios al Emitir Recibos

Momento	Usuario A	Usuario B	Efecto
1	Bloquea contrato A y prepara pago.	Bloquea contrato B y prepara pago.	No comparten contrato; aún pueden avanzar en paralelo.
2	Reserva n mediante FOR UPDATE.	Espera la fila del contador.	Solo A conoce una reserva todavía no emitida.
3a	Confirma pago, recibo n y evento.	Adquiere fila y reserva n+1.	Ambos recibos confirmados quedan contiguos.
3b	Revierte antes del commit.	Adquiere fila y reserva n.	La reserva fallida no deja hueco.
4	Pierde conexión al esperar confirmación.	Mantiene su propia operación.	A consulta solicitud_id antes de repetir; no decide por la excepción de red si hubo commit.

Orden de Bloqueo, Versiones y Reintentos

El orden global comienza en la puerta compartida y continúa con unidades, contratos por UUID, cargos por vencimiento e identificador, pagos de origen, depósitos y contador al final. Un caso que no necesita una categoría la omite sin invertir el orden de las demás. PostgreSQL detecta interbloqueos y cancela una transacción; las aplicaciones deben prever ese resultado y mantener un orden consistente de adquisición (PGDG, s. f.-i). Las escrituras ordinarias fijan SET LOCAL lock_timeout = '2000ms' y SET LOCAL statement_timeout = '5000ms'. El respaldo usa el parámetro propio respaldo_lock_timeout_ms = 3000, separado de esos límites. La interfaz muestra la operación en curso y permite cancelar antes de entrar a su sección de confirmación, sin interrumpir un commit ya iniciado.

Los SQLSTATE 40P01 y 40001 admiten como máximo tres intentos totales, cada uno con Session nueva y el mismo UUID de solicitud. La espera propuesta es de cien y doscientos cincuenta milisegundos, con variación aleatoria de cero a cincuenta, siempre fuera de transacción. Un conflicto de versión optimista o un SQLSTATE 23505 de negocio no se reintenta a ciegas: el servicio consulta el estado vigente y ofrece una corrección de entrada. Si el resultado del commit es incierto, primero busca solicitud_id y compara solicitud_hash. Un UUID confirmado devuelve su mismo recibo; una huella distinta produce ConflictoIdempotenciaException sin emitir otro número.

La columna version protege formularios editables con @Version y detecta que otra sesión cambió la fila desde su lectura. JDBC utiliza el mismo predicado de versión en usuarios. Los parámetros no se actualizan: cada cambio inserta otro registro con vigente_desde, motivo y actor. Los saldos requieren además bloqueo pesimista porque la lectura de una sola versión de contrato no limita por sí misma las aplicaciones de pagos nuevos. El servicio recarga saldos después de adquirir ese bloqueo y mantiene la transacción hasta confirmar. Un error antes de terminar produce rollback de Hibernate y de las instrucciones JDBC que usaron su Connection, sin dejar un evento financiero aislado de su operación.

Mapeo Objeto-Relacional

Hibernate administra las entidades de negocio y utiliza el mismo DataSource que JDBC para seguridad, parámetros y agregados. El caso financiero inicia una transacción Hibernate, realiza las escrituras de entidades y llama a Session.doWork o doReturningWork para ejecutar helpers y auditoría en su Connection. El adaptador JDBC no cambia autocommit ni confirma esa conexión por separado. El proveedor ORM documenta el alcance de Session, las asociaciones y el control de versiones como decisiones de persistencia (Hibernate, s. f.-c). La aplicación obtiene

un DTO antes de cerrar Session y evita entregar entidades con asociaciones diferidas a los controladores o al motor de reportes.

La Tabla 15 presenta mapeo de Entidades de Negocio.

Tabla 15

Mapeo de Entidades de Negocio

Entidad Java	Tabla y Clave	Relaciones	Propiedad, Carga y Cascada
Propiedad	propiedad; UUID	creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Unidad	unidad; UUID	propiedad_id → propiedad N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Inquilino	inquilino; UUID	creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Contrato	contrato; UUID	unidad_id → unidad N:1; inquilino_id → inquilino N:1; predecesor_id → contrato N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Cargo	cargo; UUID	contrato_id → contrato N:1; cargo_origen_id → cargo N:1; anulado_por → usuario N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
TipoCambio	tipo_cambio; UUID	creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
AutorizacionTasa	autorizacion_tasa; UUID	referencia_id → tipo_cambio N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Pago	pago; UUID	contrato_id → contrato N:1; autorizacion_id →	Propietario: esta entidad; LAZY; sin

Entidad Java	Tabla y Clave	Relaciones	Propiedad, Carga y Cascada
		autorizacion_tasa N:1; anulado_por → usuario N:1; creado_por → usuario N:1	cascada REMOVE ni ALL.
DepositoGarantia	deposito_garantia; UUID	contrato_id → contrato N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
MovimientoDeposito	movimiento_deposito; UUID	deposito_id → deposito_garantia N:1; cargo_id → cargo N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
AplicacionPago	aplicacion_pago; UUID	pago_id → pago N:1; movimiento_id → movimiento_deposito N:1; cargo_id → cargo N:1; revertida_por → usuario N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
Recibo	recibo; UUID	pago_id → pago 1:1; creado_por → usuario N:1	Recibo @Immutable; Hibernate realiza INSERT, nunca UPDATE; LAZY, sin cascadas REMOVE ni ALL. Auditoría JDBC y validación preceden al commit.
SolicitudMantenimiento	solicitud_mantenimiento; UUID	unidad_id → unidad N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
GastoPropiedad	gasto_propiedad; UUID	propiedad_id → propiedad N:1; solicitud_id → solicitud_mantenimiento N:1; creado_por → usuario N:1	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.
DocumentoAdjunto	documento_adjunto; UUID	propiedad_id → propiedad N:1; unidad_id → unidad N:1; inquilino_id → inquilino N:1; contrato_id → contrato N:1; creado_por → usuario N:1; parametro_id → parametro_sistema N:1 (UUID	Propietario: esta entidad; LAZY; sin cascada REMOVE ni ALL.

Entidad Java	Tabla y Clave	Relaciones	Propiedad, Carga y Cascada
		escalar, parámetro administrado por JDBC)	
Usuario	usuario; UUID	Destino de creado_por, anulado_por y revertida_por; rol_id como UUID escalar.	@Immutable y solo lectura en Hibernate; seguridad, claves, intentos y roles se escriben exclusivamente por JDBC.

Identidad, Enumeraciones e Inmutabilidad

Las entidades asignan UUID antes de persistir y no dependen de que la base entregue un identificador después del INSERT. equals compara clase persistente e id, y hashCode usa ese id estable sin recorrer colecciones o referencias. La identidad se mantiene al trabajar con una representación proxy mediante Hibernate.getClass y getId, con pruebas previstas de carga diferida en la implementación. Las relaciones se mapean desde la FK del dependiente y no incluyen cascadas de eliminación. Un agregado nuevo persiste cada componente de forma explícita; esta disciplina impide que borrar un formulario elimine pagos, documentos o historia contractual por un efecto de cascada.

EstadoContratoConverter transforma cada constante Java en la etiqueta exacta almacenada, como POR_VENCER hacia Por vencer. El convertidor rechaza un valor desconocido en lugar de usar ordinales que cambien al reordenar la enumeración. Todos los catálogos compartidos tienen enum Java y AttributeConverter con la etiqueta exacta, sin alternativa de cadenas sueltas. Los discriminadores internos OrigenAplicacion y TipoMovimientoDeposito usan las formas ASCII Inicial/Credito/Garantia y Aplicacion/Devolucion. Recibo se marca @Immutable y carece de métodos para editar su contenido después de construirlo. Pago, aplicaciones y movimientos tienen importes inmutables;

las anulaciones se escriben con sentencias de columnas permitidas y actualización controlada de la sesión para no conservar datos obsoletos.

Las entidades editables tienen @Version sobre Long y la columna BIGINT definida por el DDL. Las tablas históricas sin edición de contenido omiten version, y las reversiones permitidas se ejecutan bajo bloqueo del contrato en un adaptador específico. El rol de PostgreSQL impide que una actualización accidental alcance columnas financieras del pago o del recibo. Los ejemplos del Apéndice E son clases de mapeo y consultas, sin presentarse como una aplicación terminada. Las tablas operadas solo por JDBC no necesitan una entidad ORM durante la operación; sus clases de ejemplo ilustran el diccionario y no cambian la distribución de responsabilidades adoptada en ADR-04. Usuario es la excepción de lectura ORM necesaria para resolver los @ManyToOne de auditoría; sus escrituras de seguridad continúan en JDBC. El paquete raíz es ni.edu.uam.latch, con capas arranque, presentacion, aplicacion, dominio e infraestructura. Se documenta la excepción que permite a dominio importar jakarta.persistence.*, org.hibernate.annotations.* y Hibernate.getClass.

La Tabla 16 presenta consultas Nombradas en JPQL y HQL.

Tabla 16

Consultas Nombradas en JPQL y HQL

Nombre y Propósito	Consulta Completa	Parámetros
Propiedad.activas	select p from Propiedad p where p.activa=true order by p.nombre,p.id	Sin parámetros; inventario activo.
Unidad.porPropiedad	select u from Unidad u where u.propiedad.id=:propiedad order by u.codigo,u.id	propiedad: UUID; unidades de la propiedad.
Unidad.disponibles	select u from Unidad u where u.estado=:disponible and not exists (select c.id from Contrato c where c.unidad=u and c.estado in :activos) order by u.codigo,u.id	disponible: EstadoUnidad.DISPONIBLE; activos: Set<EstadoContrato>;

Nombre y Propósito	Consulta Completa	Parámetros
		selección de unidad.
Inquilino.buscar	select i from Inquilino i where i.activo=true and lower(i.nombre) like :patron order by i.nombre,i.id	patron: String escapado con %; paginación por 50.
Contrato.porInquilino	select c from Contrato c where c.inquilino.id=:inquilino order by c.inicio desc,c.id	inquilino: UUID; historial.
Contrato.activoUnidad	select c from Contrato c where c.unidad.id=:unidad and c.estado in :activos	unidad: UUID; activos: Set<EstadoContrato>; reserva.
Contrato.porVencer	select c from Contrato c where c.estado in :activos and c.fin between :desde and :hasta order by c.fin,c.id	activos: Set<EstadoContrato>; desde/hasta: LocalDate.
Cargo.porPeriodo	select c from Cargo c where c.contrato.id=:contrato and c.periodo=:periodo and c.tipo=:tipo	contrato: UUID; periodo: LocalDate día 1; tipo: TipoCargo.
Cargo.ordenCobro	select c from Cargo c where c.contrato.id=:contrato and c.anuladoEn is null and c.venceEl<:fecha order by c.venceEl,c.id	contrato: UUID; fecha: LocalDate; saldos se validan aparte.
Pago.porSolicitud	select p from Pago p where p.solicitudId=:solicitud	solicitud: UUID; recuperar resultado idempotente.
Pago.historial	select p from Pago p where p.contrato.id=:contrato and p.fechaPago between :desde and :hasta order by p.fechaPago,p.id	contrato: UUID; desde/hasta: LocalDate.
Recibo.porNumero	select r from Recibo r join fetch r.pago where r.numero=:numero	numero: Long; detalle 1:1.
AplicacionPago.porPago	select a from AplicacionPago a where a.pago.id=:pago and a.revertidaEn is null order by a.fechaAplicacion,a.id	pago: UUID; dependencias de crédito.
DepositoGarantia.porContrato	select d from DepositoGarantia d where d.contrato.id=:contrato	contrato: UUID; garantía única.
SolicitudMantenimiento.pendientes	select s from SolicitudMantenimiento s where s.unidad.id=:unidad and s.estado in :estados order by s.abiertaEl,s.id	unidad: UUID; estados: Set<EstadoMantenimiento> Abierta/En proceso.
GastoPropiedad.p	select g from GastoPropiedad g where	propiedad: UUID;

Nombre y Propósito	Consulta Completa	Parámetros
orFecha	g.propiedad.id=:propiedad and g.fechaGasto between :desde and :hasta order by g.fechaGasto,g.id	desde/hasta: LocalDate.
DocumentoAdjunto.porContrato	select d from DocumentoAdjunto d where d.contrato.id=:contrato and d.activo=true order by d.nombreOriginal,d.id	contrato: UUID; expediente.
AutorizacionTasa.porFecha	select t from AutorizacionTasa t where t.fechaEfectiva=:fecha	fecha: LocalDate; tasa autorizada.

Nota. El Apéndice E incluye orm.xml con las 18 consultas. El registro de mapping-file es META-INF/orm.xml. :activos recibe valores EstadoContrato, no etiquetas String. Los límites se aplican con setFirstResult y setMaxResults, sin fetch join de colecciones.

Proyecciones y Frontera con JDBC

Los reportes usan proyecciones inmutables con UUID, LocalDate, Instant y BigDecimal como tipos explícitos. El constructor del DTO recibe datos ya materializados y el reporte no abre asociaciones ORM al llenar una plantilla. Para consultas JPQL, Hibernate admite selección de valores y proyección mediante constructores; el diseño elige esa forma cuando no necesita un agregado editable (Hibernate, s. f.-b). Los agregados de cartera y panel permanecen en JDBC para evidenciar SQL y aprovechar las vistas compartidas. Ningún DAO retorna ResultSet vivo: el adaptador consume filas dentro de try-with-resources y cierra recursos antes de devolver el resultado al servicio.

La Tabla 17 presenta dTO Compartidos entre Servicios y Reportes.

Tabla 17

DTO Compartidos entre Servicios y Reportes

DTO	Campos y Tipos	Uso
ContextoOperacion	UUID usuarioId, UUID correlacionId, UUID solicitudId, Rol rol. Fábrica ContextoOperacion.sistema(tarea) para H2/H3, sin	Sesión comprobada; instante financiero = now() de la BD,

DTO	Campos y Tipos	Uso
	usuario ni rol humano; solo permite la lista cerrada de acciones de sistema.	guardado en creado_en, y fecha operativa = creado_en en America/Managua. Antes de toda operación financiera, |reloj local − BD| ≤ 5 min, con control por fecha operativa; en otro caso RelojDesfasadoExceptio n, ERR-057.
CargoCorteDto	UUID cargoId, UUID contratoId, Moneda moneda, BigDecimal importe/abonado/saldo, LocalDate venceEl, EstadoCargo estado.	Estado de cuenta, cartera y panel.
PagoConfirmado Dto	UUID pagoId, UUID solicitudId, long numeroRecibo, EstadoPago estado, BigDecimal convertido/credito.	Confirmación y recuperación idempotente.
ReporteDto	TipoReporte tipo, LocalDate corte, Instant generadoEn, List<FilaReporteDto> filas, Map<String,BigDecimal> totalesPorMoneda.	H5 sin Session abierta; colección inmutable.
TasaDto	BigDecimal valor, String fuente, LocalDate fechaEfectiva, Instant actualizadoEn/consultadoEn, boolean autorizada.	M09 distingue referencia y autorización.
Pagina<T>	List<T> elementos, int limite, String cursorSiguiente opcional.	Consulta estable con máximo 50 filas por página de interfaz.

Contratos de la Capa de Servicios

Cada servicio público recibe un contexto de sesión emitido por M01 y valida la capacidad funcional antes de realizar una escritura. El servicio vuelve a comprobar usuario activo, rol y caducidad cuando una tarea diferida empieza a operar, de modo que ocultar un botón no sustituye la autorización. Los métodos no reciben Session o Connection desde JavaFX y retornan DTO, identificadores o resultados definidos. La Tabla de cada módulo declara firmas, parámetros, retorno, excepciones y frontera transaccional. Los tipos de solicitud agrupan

entradas validadas, conservan UUID de idempotencia cuando corresponde y no permiten que el formulario imponga importes calculados o un número de recibo.

La jerarquía pública tiene como base única LatchException. La unión de TRD Tabla 8 y estos contratos comprende: AccesoDenegadoException, ArchivoException, CargoAplicadoException, CargoIncompatibleException, ConflictoConcurrenciaException, ConflictoIdempotenciaException, ConsultaReporteException, CuentaBloqueadaException, CuotaFxException, DependenciaActivaException, DependenciaCreditoException, DuplicadoException, ExportacionException, FalloAccesoDatosException, GarantiaInsuficienteException, GarantiaPendienteException, IntegridadException, OperacionCanceladaException, ParametroInvalidoException, PeriodoDuplicadoException, PlantillaException, PropiedadIncompatibleException, RelojDesfasadoException, ReporteException, RespaldoException, RespuestaFxInvalidaException, RestauracionException, ResultadoInciertoException, SaldoInsuficienteException, ServicioExternoException, TareasActivasException, TasaNoAutorizadaException, TransicionInvalidaException, UnidadReservadaException, ValidacionException. AccesoDenegadoException corresponde a ERR-009 / MSG-062; RelojDesfasadoException a ERR-057 / MSG-080. El catálogo aprobado comprende 66 ERR y 89 MSG; el Brief conserva el texto literal, FLU define situación y recuperación y TRD relaciona excepciones por escenario. Un ERR puede corresponder a varias excepciones si mantiene su significado. CargoIncompatibleException usa ERR-061 / MSG-084; GarantiaPendienteException, ERR-062 / MSG-085; IntegridadException, ERR-063 / MSG-086 solo con rollback confirmado; ParametroInvalidoException, ERR-064 / MSG-051; PropiedadIncompatibleException, ERR-065 / MSG-087; TransicionInvalidaException, ERR-066 / MSG-088. OperacionCanceladaException es cancelación normal sin ERR, con MSG-059 en

H1 y MSG-089 en H5. Un commit incierto conserva ERR-031 o ERR-054 y MSG-064, sin afirmar ausencia de cambios. Las operaciones financieras de los contratos M04 a M07 comprueban también RelojDesfasadoException antes de escribir.

M01 SeguridadService

SeguridadService emite sesiones que contienen el UUID del usuario y su rol comprobado, con caducidad por quince minutos de inactividad. La verificación de contraseña calcula Argon2id antes de mantener un bloqueo prolongado y después relee versión, estado e intentos al confirmar el resultado. Cinco fallos consecutivos bloquean quince minutos por defecto; un acceso correcto reinicia el contador si la cuenta sigue habilitada. La modificación del rol invalida las sesiones anteriores al siguiente control de operación. El aprovisionamiento inicial usa una herramienta local separada y obliga a cambiar la clave antes de habilitar módulos financieros.

La Tabla 18 presenta contrato de SeguridadService.

Tabla 18

Contrato de SeguridadService

Elemento	Especificación
Nombre y módulo	SeguridadService; M01
Métodos y firmas	SesionDto autenticar(String cuenta, char[] clave) UUID guardarUsuario(ContextoOperacion ctx, UsuarioCommand cmd) void cambiarClave(ContextoOperacion ctx, char[] actual, char[] nueva) void cerrarSesion(ContextoOperacion ctx)
Parámetros	UsuarioCommand: UUID id opcional, long version, String nombre, Rol rol, boolean activo. Contraseñas: 12 a 128 caracteres, sin eco ni logs.
Retorno	Sesión opaca con id/rol/vencimiento; UUID de cuenta o fin de operación.
Excepciones	AccesoDenegadoException; CuentaBloqueadaException;

Elemento	Especificación
	ValidacionException; ConflictoConcurrenciaException.
Transacción	JDBC por cambio de usuario, intento o clave; hash fuera de bloqueo; update con versión y auditoría juntos. fn_auditar: Actor NULL solo en M01 ACCESO_FALLIDO/ACCESO_BLOQUEADO, M05 MORA_AUTOMATICA/REVISION_VENCIMIENTO/INCIDENTE_H2 con detalle.origen='H2', o M09 REFERENCIA_ACTUALIZADA/REFERENCIA_EN_CUARENTENA/INCIDENTE_H3 con detalle.origen='H3'. autorizacion_tasa exige actor humano. En M01 el detalle contiene nombre_hash, SHA-256 truncado del nombre normalizado, nunca texto ingresado.
Rol requerido	Acceso sin sesión; alta, rol y activación solo Propietario; cambio propio y cierre ambos.
Reglas y requisitos	RN-13, RN-14, RN-15, RN-16; RF-M01-01 a RF-M01-11; ADR-07.

M02 PropiedadUnidadService

La edición de una unidad conserva el código único dentro de su propiedad y su canon de referencia, sin modificar contratos históricos. El método de baja verifica la versión del formulario y bloquea la unidad antes de consultar reservas activas. Si existe un contrato Vigente, Por vencer o Vencido, el servicio rechaza la baja con la referencia del contrato que la impide. Una propiedad tampoco se desactiva mientras tenga unidades activas. Las fotografías son opcionales y pasan por la validación de adjuntos, cuyo límite de cinco MiB y huella se aplica antes de asociarlas al registro.

La Tabla 19 presenta contrato de PropiedadUnidadService.

Tabla 19

Contrato de PropiedadUnidadService

Elemento	Especificación
Nombre y módulo	PropiedadUnidadService; M02
Métodos y firmas	UUID guardarPropiedad(ContextoOperacion ctx, PropiedadCommand cmd)

Elemento	Especificación
	UUID guardarUnidad(ContextoOperacion ctx, UnidadCommand cmd) void darBaja(ContextoOperacion ctx, UUID id, long version) Pagina<UnidadDto> listar(ContextoOperacion ctx, FiltroUnidadCommand f)
Parámetros	PropiedadCommand: nombre/dirección/versión. UnidadCommand: propiedad, código, tipo, canon de referencia y moneda. FiltroUnidadCommand: propiedad, estado, cursor. EstadoUnidad persistido sin Ocupada; UnidadDto expone EstadoUnidadVisible calculado.
Retorno	UUID guardado; lista de DTO con código, estado, tipo y canon.
Excepciones	ValidacionException; DuplicadoException; DependenciaActivaException; ConflictoConcurrenciaException.
Transacción	Hibernate por alta/edición/baja; bloqueo de unidad antes de comprobar contratos y de propiedad al darla de baja.
Rol requerido	Propietario: todas las operaciones. Asistente: consulta de unidades y edición de condiciones/foto (PAN-05 y PAN-21 en edición); sin alta de propiedad/unidad, sin bajas y sin restricción por estado En mantenimiento.
Reglas y requisitos	RN-01, RN-03, RN-11, RN-16; RF-M02-01 a RF-M02-11; ADR-04.

M03 InquilinoService

El expediente del inquilino conserva el nombre necesario para el contrato y permite omitir identificación, emergencia o documentos que no hagan falta al escenario. La búsqueda muestra resultados paginados y no convierte un campo de identificación opcional en requisito de unicidad universal. InquilinoService conserva referencias históricas al dar de baja, de modo que los recibos y los estados de cuenta sigan explicando la relación contractual. Los adjuntos usan rutas internas aleatorias y nombres visibles separados. El servicio limita la lectura del expediente a sesiones autorizadas, sin habilitar un portal o una cuenta operativa para el inquilino.

La Tabla 20 presenta contrato de InquilinoService.

Tabla 20

Contrato de InquilinoService

Elemento	Especificación
Nombre y módulo	InquilinoService; M03
Métodos y firmas	UUID guardar(ContextoOperacion ctx, InquilinoCommand cmd) ExpedienteDto expediente(ContextoOperacion ctx, UUID id) void darBaja(ContextoOperacion ctx, UUID id, long version)
Parámetros	InquilinoCommand: id/versión, nombre, identificación/contactos opcionales. UUID de expediente sin exponer cédula como clave.
Retorno	UUID; ExpedienteDto con ficha, contratos y referencias de adjuntos.
Excepciones	ValidacionException; DependenciaActivaException; AccesoDenegadoException; ConflictoConcurrenciaException.
Transacción	Hibernate en escritura; consulta materializada antes de cerrar Session; adjunto publicado bajo puerta de escritura.
Rol requerido	Altas, consulta y edición: ambos. Baja lógica: Propietario.
Reglas y requisitos	RN-11, RN-18, RN-25; RF-M03-01 a RF-M03-09; ADR-04.

M04 ContratoService

ContratoService valida un mes calendario mínimo con el mismo ajuste de día que LocalDate.plusMonths(1). La activación relee unidad e inquilino habilitados, verifica la reserva activa y registra la garantía pactada, incluso si su importe es cero. Renovar solo se inicia desde Vigente o Por vencer y crea el sucesor Borrador con UUID nuevo y referencia al contrato anterior; no modifica rentas confirmadas. El cierre atómico que termina el anterior y activa el sucesor cuenta como Terminar y se permite desde Vigente, Por vencer o Vencido. Se conserva fin >= inicio + 1 mes; para cubrir un mes natural completo del día 1 al 31, fin debe ser como mínimo el día 1 del mes siguiente. La fecha final inclusiva no acredita desocupación por sí sola,

por lo que Vencido sigue reservando la unidad. Terminar coordina las decisiones de garantía y deuda pendientes con el Propietario, sin emitir rentas futuras después del cierre autorizado. GarantiaPendienteException (ERR-062 / MSG-085) bloquea la terminación si quedan decisiones, movimientos o remanente de garantía sin resolver; se conserva la excepción de garantía inicial cero sin movimientos y puede quedar deuda residual después de terminar.

La Tabla 21 presenta contrato de ContratoService.

Tabla 21

Contrato de ContratoService

Elemento	Especificación
Nombre y módulo	ContratoService; M04
Métodos y firmas	UUID guardarBorrador(ContextoOperacion ctx, ContratoCommand cmd) ContratoDto activar(ContextoOperacion ctx, UUID id, long version) UUID renovar(ContextoOperacion ctx, UUID anterior, ContratoCommand sucesor) void terminar(ContextoOperacion ctx, UUID id, CierreCommand cmd)
Parámetros	ContratoCommand: unidad/inquilino, inicio/fin, canon, moneda, día 1..31, depósito. CierreCommand: fecha y liquidación autorizada.
Retorno	UUID o ContratoDto con reserva y garantía; Renovar devuelve UUID del sucesor Borrador desde Vigente o Por vencer. Terminar puede cerrar el anterior y activar el sucesor atómicamente desde Vigente, Por vencer o Vencido.
Excepciones	UnidadReservadaException; ValidacionException; GarantiaPendienteException; ConflictoConcurrenciaException. RelojDesfasadoException (ERR-057).
Transacción	Hibernate y auditoría JDBC compartida; puerta, unidad, contratos, cargos y depósito. Activación y depósito son atómicos.
Rol requerido	Borrador/activación: ambos. Ambos pueden preparar renovación y terminación. El cierre con garantía/liquidación lo confirma el Propietario, excepto garantía inicial cero sin movimientos, que pueden confirmar ambos.
Reglas y	RN-01 a RN-03, RN-10 a RN-12, RN-17, RN-19, RN-33, RN-34; RF-M04-01

Elemento	Especificación
requisitos	a RF-M04-13.

M05 CobranzaService

H1 obtiene los contratos elegibles y aplica una transacción por contrato para que una cancelación conserve los ya confirmados. La generación respeta el período facturado y asigna vencimiento con ajuste al último día disponible del mes. H2 utiliza el mismo bloqueo para comprobar principal pendiente después de la gracia y emitir una sola mora por renta. La política inicial de tres días y porcentaje cero desactiva el recargo mientras el Propietario no lo configure. Los cargos manuales exigen concepto y no convierten de oficio un gasto de mantenimiento en deuda del inquilino; la repercusión requiere un comando independiente. La cancelación voluntaria usa MSG-059 sin ERR: termina la transacción del contrato en curso, detiene el lote antes del siguiente y conserva confirmados e identifica pendientes en el resumen.

La Tabla 22 presenta contrato de CobranzaService.

Tabla 22

Contrato de CobranzaService

Elemento	Especificación
Nombre y módulo	CobranzaService; M05
Métodos y firmas	ResultadoLoteDto generarMes(ContextoOperacion ctx, YearMonth periodo, Cancelacion cancelacion) UUID crearCargo(ContextoOperacion ctx, CargoCommand cmd) ResultadoRevisionDto revisarMora(ContextoOperacion ctx, LocalDate fecha) void anularCargo(ContextoOperacion ctx, UUID id, String motivo)
Parámetros	CargoCommand: contrato, tipo, concepto, importe, emisión/vencimiento, período. El importe Mora lo calcula el servicio.
Retorno	Conteos confirmados/omitidos/cancelados; UUID de cargo o revisión sin duplicados.

Elemento	Especificación
Excepciones	PeriodoDuplicadoException; ValidacionException; CargoAplicadoException; AccesoDenegadoException. RelojDesfasadoException (ERR-057). OperacionCanceladaException: sin ERR, MSG-059 (H1).
Transacción	H1/H2 una transacción Hibernate por contrato; crédito aplicado con el cargo; cancelación entre contratos. H2 usa ContextoOperacion.sistema(H2), actor NULL y detalle.origen=H2 solo para MORA_AUTOMATICA, REVISION_VENCIMIENTO e INCIDENTE_H2; arranca tras validar el esquema (FLU-49).
Rol requerido	Generación y cargo manual: ambos. Anulación y parámetros de mora: Propietario. H2: tarea de sistema tras validar el esquema al arrancar, sin exigir acceso de usuario.
Reglas y requisitos	RN-04 a RN-06, RN-17, RN-20, RN-24, RN-31; RF-M05-01 a RF-M05-12.

M06 PagoService

PagoService convierte el monto completo con la autorización de la fecha y calcula el equivalente contractual antes de distribuirlo. La selección automática cubre cargos vencidos en orden de fecha e id; si no existen, todo el monto queda como crédito. La selección manual admite cargos abiertos del mismo contrato, sin exceder sus saldos al releerlos bajo bloqueo. El servicio emite un recibo por pago y conserva el UUID de solicitud para recuperar una confirmación incierta. La anulación exige ausencia de aplicaciones posteriores activas del crédito y revierte sus aplicaciones iniciales con motivo, actor e instante, sin reutilizar el número.

La Tabla 23 presenta contrato de PagoService.

Tabla 23

Contrato de PagoService

Elemento	Especificación
Nombre y módulo	PagoService; M06
Métodos y firmas	PagoConfirmadoDto registrar(ContextoOperacion ctx, PagoCommand cmd)

Elemento	Especificación
	PagoConfirmadoDto recuperar(ContextoOperacion ctx, UUID solicitud) void anular(ContextoOperacion ctx, UUID pago, String motivo) ReciboDto obtenerRecibo(ContextoOperacion ctx, long numero)
Parámetros	PagoCommand: solicitud UUID/hash, contrato, LocalDate fechaEfectiva ingresada por el usuario, no futura respecto a la fecha operativa. Instante financiero = now() de la BD, guardado en pago.creado_en; fecha operativa = creado_en en America/Managua, sin fecha impuesta por el cliente. La tasa autorizada se busca por fecha efectiva. Moneda/monto original, método, asignación automática o Map<UUID,BigDecimal>. Sin número recibido del cliente.
Retorno	PagoConfirmadoDto; ReciboDto de instantánea y estado, sin nuevo número al reimprimir.
Excepciones	TasaNoAutorizadaException; SaldoInsuficienteException; DependenciaCreditoException; ConflictoIdempotenciaException. RelojDesfasadoException (ERR-057). IntegridadException: ERR-063 / MSG-086 únicamente con rollback confirmado; commit incierto: ERR-031 o ERR-054 / MSG-064.
Transacción	Hibernate: flush del pago/aplicaciones → fn_reservar_numero() → persistir Recibo @Immutable y flush → auditoría JDBC → sp_verificar_integridad → commit. Una sola Session, conexión y transacción; anulación por columnas bajo los mismos bloqueos.
Rol requerido	Registro, consulta y reimpresión: ambos. Anulación: Propietario.
Reglas y requisitos	RN-06 a RN-09, RN-14, RN-21, RN-22, RN-28, RN-31, RN-32; RF-M06-01 a RF-M06-12; ADR-16.

M07 DepositoService

La recepción inicial de garantía forma parte de activar el contrato y no se repite desde una pantalla independiente con otro monto. DepositoService consulta sus movimientos y exige confirmación del Propietario para el cierre con garantía/liquidación y para cada aplicación o devolución. El Asistente puede preparar renovación y terminación; ambos roles pueden confirmar el cierre si la garantía inicial es cero y no tiene movimientos. La suma de las salidas debe ser menor o igual al remanente bloqueado, y una aplicación identifica el cargo del mismo contrato y su concepto de deducción. El comprobante de garantía se diferencia del recibo de renta mediante referencia UUID. El método de reversión de crédito permite liberar dependencias

de un pago antes de anularlo, sin modificar la historia de la garantía.

CargoIncompatibleException (ERR-061 / MSG-084) rechaza un cargo inexistente, anulado o de otro contrato y conserva la propuesta para corregirla; el exceso sobre saldo conserva su error específico.

La Tabla 24 presenta contrato de DepositoService.

Tabla 24

Contrato de DepositoService

Elemento	Especificación
Nombre y módulo	DepositoService; M07
Métodos y firmas	DepositoDto consultar(ContextoOperacion ctx, UUID contrato) LiquidacionDto liquidar(ContextoOperacion ctx, LiquidacionCommand cmd) void revertirAplicacionCredito(ContextoOperacion ctx, UUID aplicacion, String motivo)
Parámetros	LiquidacionCommand: UUID liquidacionId, String liquidacionHash (SHA-256 hexadecimal de 64 caracteres), contrato, lista ordenada de aplicación cargo/importe/concepto y devolución/importe/concepto. Todos los movimientos del paquete comparten liquidacion_id y hash, con orden SMALLINT y UNIQUE(liquidacion_id,orden). La idempotencia consulta por liquidacion_id y rechaza contenido distinto. Reversión solo de crédito de pago; la garantía no se edita.
Retorno	Inicial, aplicado, devuelto, remanente y estado; referencias UUID de movimientos.
Excepciones	GarantiaInsuficienteException; CargoIncompatibleException (ERR-061 / MSG-084); DependenciaCreditoException; AccesoDenegadoException. RelojDesfasadoException (ERR-057).
Transacción	Una transacción Hibernate por paquete de liquidación y cierre; movimiento y aplicación equivalentes. Consultar por liquidacion_id para recuperar el paquete idempotente y comprobar liquidacion_hash. Reversión de crédito se audita en la transacción del contrato.
Rol requerido	Consulta: ambos. Liquidación y reversión de crédito: Propietario; recepción inicial por ContratoService.
Reglas y	RN-10, RN-13, RN-21, RN-26, RN-31; RF-M07-01 a RF-M07-07.

Elemento	Especificación
requisitos	

M08 MantenimientoService

Las solicitudes avanzan de Abierta a En proceso o Cancelada y de En proceso a Resuelta o Cancelada. Resuelta y Cancelada son terminales, con fecha de cierre obligatoria en el registro. El gasto confirmado guarda propiedad, moneda y concepto, y si referencia una solicitud comprueba que la unidad pertenece a esa propiedad. MantenimientoService no descuenta garantía ni añade reparaciones a un estado de cuenta por registrar un costo. El traslado al inquilino exige crear un cargo Reparación en M05 y conserva en auditoría la relación que justifica esa decisión, sin duplicar el egreso del propietario. Repetir una solicitud hacia el estado actual devuelve ese estado sin duplicar cambios ni eventos; cualquier otra transición se rechaza con ERR-066 / MSG-088 conservando el estado anterior. La propiedad incompatible al registrar un gasto se rechaza con ERR-065 / MSG-087 y permite corregir la selección.

La Tabla 25 presenta contrato de MantenimientoService.

Tabla 25

Contrato de MantenimientoService

Elemento	Especificación
Nombre y módulo	MantenimientoService; M08
Métodos y firmas	UUID abrir(ContextoOperacion ctx, SolicitudCommand cmd) void cambiarEstado(ContextoOperacion ctx, UUID id, long version, EstadoMantenimiento destino) UUID registrarGasto(ContextoOperacion ctx, GastoCommand cmd)
Parámetros	SolicitudCommand: unidad, descripción, prioridad, responsable. GastoCommand: propiedad, solicitud opcional, fecha, moneda, importe y concepto.
Retorno	UUID de solicitud o gasto; estado confirmado con versión actualizada.

Elemento	Especificación
Excepciones	TransicionInvalidaException (ERR-066 / MSG-088); ValidacionException; PropiedadIncompatibleException (ERR-065 / MSG-087); ConflictoConcurrenciaException.
Transacción	Hibernate por comando; gasto y auditoría juntos; no genera cargo al inquilino.
Rol requerido	Propietario y Asistente, conforme acciones operativas M08.
Reglas y requisitos	RN-23, RN-27; RF-M08-01 a RF-M08-07.

M09 TipoCambioService

TipoCambioService conserva las versiones de referencia recibidas por HTTP y muestra proveedor, actualización y antigüedad antes de permitir una interpretación de la cifra. La concesión y el contador compartidos impiden que tres clientes LAN consuman el presupuesto como instalaciones independientes. La referencia comercial se valida en la banda de 1–100 NIO/USD. Una referencia comercial nunca se convierte en autorización de cobro por refrescar la caché. Para una tasa manual fuera de esa banda se muestra una advertencia de revisión y se exige confirmación del Propietario, sin imponer esa banda como restricción en la BD. El Propietario registra fecha efectiva, fuente y fundamento de la tasa que usará el pago, con su propio actor e instante. Si no existe autorización para una fecha, la aplicación rechaza la conversión entre monedas y conserva disponibles los pagos que no requieren cambio de moneda.

La Tabla 26 presenta contrato de TipoCambioService.

Tabla 26

Contrato de TipoCambioService

Elemento	Especificación
Nombre y módulo	TipoCambioService; M09
Métodos y firmas	ReferenciaTasaDto consultarCache(ContextoOperacion ctx)
	ResultadoActualizacionDto actualizar(ContextoOperacion ctx, UUID

Elemento	Especificación
	instancia) TasaDto autorizar(ContextoOperacion ctx, AutorizacionCommand cmd)
Parámetros	AutorizacionCommand: fecha local, valor NIO/USD, fuente y fundamento. Instancia UUID participa en concesión; llave API externa al comando. Si la tasa manual queda fuera de 1–100 NIO/USD, se muestra una advertencia de revisión y se exige confirmación del Propietario; esa banda no es una restricción de la BD.
Retorno	Referencia con antigüedad; resultado actualizado/omitido/error; tasa autorizada por fecha.
Excepciones	RespuestaFxInvalidaException; CuotaFxException; TasaNoAutorizadaException; DuplicadoException. Autorización existente: DuplicadoException (ERR-060, MSG-083: «Ya existe una tasa autorizada para esa fecha»); registro inmutable, una fecha efectiva.
Transacción	Concesión JDBC corta; HTTP sin transacción; referencia y autorización con Hibernate y bitácora compartida. H3 usa ContextoOperacion.sistema(H3), actor NULL y detalle.origen=H3 solo para REFERENCIA_ACTUALIZADA, REFERENCIA_EN_CUARENTENA e INCIDENTE_H3; arranca tras validar el esquema (FLU-49).
Rol requerido	Consulta: ambos. Actualización programada H3: sistema tras validar el esquema al arrancar. Autorizar: Propietario, con actor humano obligatorio.
Reglas y requisitos	RN-09, RN-22, RN-32; RF-M09-01 a RF-M09-09; ADR-08, ADR-20.

M10 ReporteService

ReporteService toma filtros explícitos y produce una colección inmutable antes de iniciar H5 con JasperReports. Cada reporte conserva corte y moneda, y no suma USD con NIO sin una política de conversión que este alcance no adopta para totales generales. El recibo usa la instantánea emitida y consulta el estado administrativo para marcar una anulación en la copia. El recibo se exporta solo a PDF; los otros nueve reportes admiten PDF y XLSX. Los archivos se generan en un archivo temporal del mismo destino y se publican mediante cambio de nombre al terminar. Una cancelación o error de disco conserva el archivo anterior y no anuncia una exportación parcial como exitosa. H5 solicita cancelación cooperativa durante consulta, llenado o exportación y la confirma al terminar el trabajo y limpiar temporales, con MSG-089 y sin ERR

si aún no publicó. La publicación final resuelve la carrera: si ya se publicó, informa el resultado real. Se conserva el destino previo hasta publicación exitosa y todo pago o anulación ya confirmado.

La Tabla 27 presenta contrato de ReporteService.

Tabla 27

Contrato de ReporteService

Elemento	Especificación
Nombre y módulo	ReporteService; M10
Métodos y firmas	ReporteDto consultar(ContextoOperacion ctx, ReporteFiltroCommand filtro) Path exportar(ContextoOperacion ctx, ReporteDto reporte, Formato formato, Path destino, Cancelacion cancelacion)
Parámetros	ReporteFiltroCommand: tipo de diez reportes, fecha corte/rango, contrato o propiedad, moneda. Ocupación: instante de generación sin corte histórico. Recibo: solo PDF. Los otros nueve reportes: PDF o XLSX; ruta elegida con FileChooser.
Retorno	DTO materializado y ruta publicada; encabezado con filtros y totales separados por moneda.
Excepciones	ConsultaReporteException; PlantillaException; ExportacionException; OperacionCanceladaException (sin ERR; MSG-089 en H5).
Transacción	JDBC READ ONLY; REPEATABLE READ si usa varias consultas; termina antes de Jasper y archivos.
Rol requerido	Ambos para reportes operativos. La bitácora se consulta solo en PAN-19 mediante ConfiguracionAuditoriaService y requiere Propietario.
Reglas y requisitos	RN-08, RN-24, RN-28; RF-M10-01 a RF-M10-13; ADR-09.

M11 PanelService

El panel consulta indicadores sin escribir cargos ni cambiar estados como efecto de abrir la ventana. La ocupación cuenta reservas activas y separa unidades en mantenimiento o inactivas de las disponibles. Las tarjetas financieras presentan moneda, fecha de corte y diferencia entre lo cobrado y la deuda consultada; la tasa comercial muestra su carácter orientativo. Las alertas

enlazan al detalle del contrato o cargo sin modificar filtros del usuario por accidente. El servicio de tema pertenece a presentación y usa Preferences local, de modo que elegir Claro, Oscuro o sin efectos no modifica los privilegios administrativos del panel. El Brief es la fuente canónica de tokens, componentes y variantes; las excepciones visuales se justifican y registran. La presentación define los estados aplicables de cada PAN y ofrece Guardar, Descartar o Continuar editando al abandonar un formulario modificado; una validación fallida conserva los datos válidos no secretos.

La Tabla 28 presenta contrato de PanelService.

Tabla 28

Contrato de PanelService

Elemento	Especificación
Nombre y módulo	PanelService; M11
Métodos y firmas	PanelDto obtener(ContextoOperacion ctx, LocalDate corte) Pagina<AlertaDto> alertas(ContextoOperacion ctx, FiltroAlertaCommand filtro)
Parámetros	Corte local; filtro por cartera, vencimiento o mantenimiento. Indicadores llevan moneda y no aceptan importes desde JavaFX. La ocupación se calcula al instante de generación, sin selector de fecha histórica.
Retorno	Ocupación, cobrado, por cobrar, mora, contratos por vencer y referencia de tasa con fuente.
Excepciones	FalloAccesoDatosException; ValidacionException; AccesoDenegadoException.
Transacción	JDBC solo lectura, snapshot breve; sin crear cargos al consultar el panel.
Rol requerido	Propietario y Asistente en indicadores permitidos; preferencias visuales son personales.
Reglas y requisitos	RN-12, RN-24, RN-28, RN-35 a RN-38; RF-M11-01 a RF-M11-10; ADR-18.

M12 ConfiguracionAuditoriaService

M12 valida el tipo y rango de cada parámetro antes de confirmar un nuevo registro con vigente_desde y motivo, sin UPDATE ni columna version junto al evento de auditoría. La consulta de bitácora es paginada y no expone acciones de edición o borrado al operador. Un respaldo toma la puerta exclusiva, espera escrituras ya iniciadas y conserva la conexión coordinadora hasta terminar base, adjuntos y manifiesto. H6 solicita cancelación de H1, H2, H3 y H5, y espera que H4 concluya su transacción. Si el cierre no termina en diez segundos, informa las tareas activas y mantiene el proceso abierto para evitar una terminación forzada del pago. La consulta vigente selecciona la última fila por clave con vigente_desde ≤ now(); el logotipo se vincula al UUID del registro emisor mediante documento_adjunto.parametro_id. Los días o montos configurables negativos se rechazan con ParametroInvalidoException (ERR-064 / MSG-051), conservando el formulario; las demás validaciones mantienen su mensaje específico.

La Tabla 29 presenta contrato de ConfiguracionAuditoriaService.

Tabla 29

Contrato de ConfiguracionAuditoriaService

Elemento	Especificación
Nombre y módulo	ConfiguracionAuditoriaService; M12
Métodos y firmas	void cambiarParametro(ContextoOperacion ctx, ParametroCommand cmd) Pagina<EventoDto> consultarBitacora(ContextoOperacion ctx, FiltroAuditoriaCommand filtro) RespaldoDto respaldar(ContextoOperacion ctx, Path destino) void restaurar(ContextoOperacion ctx, Path paquete) CierreDto cerrarAplicacion(ContextoOperacion ctx)
Parámetros	ParametroCommand: clave, valor JSON tipado, vigente_desde TIMESTAMPTZ y motivo de hasta 500 caracteres; sin version. Cada cambio

Elemento	Especificación
	inserta una fila. Vigente: última fila por clave con vigente_desde ≤ now(). Paquete íntegro, clave de respaldo sin logs y filtros por actor/acción/fecha.
Retorno	UUID del registro de parámetro confirmado; eventos paginados, manifiesto de respaldo o estado de cierre.
Excepciones	ParametroInvalidoException (ERR-064 / MSG-051); RespaldoException; RestauracionException; TareasActivasException; AccesoDenegadoException.
Transacción	JDBC: INSERT de parámetro y evento en la misma transacción; sin UPDATE ni DELETE. Puerta exclusiva durante respaldo; restauración con clientes detenidos y cuenta administrativa externa con CREATEDB.
Rol requerido	Parámetros, auditoría, respaldo/restauración: Propietario. Cierre de sesión/aplicación: ambos sin autorizar restauración.
Reglas y requisitos	RN-13 a RN-15, RN-29, RN-30; RF-M12-01 a RF-M12-08; ADR-17.

Esquema del Servicio Web Externo

Latch conserva ExchangeRate-API de AYR Tech como proveedor de referencia informativa, conforme al PRD y ADR-20 del TRD. Su catálogo admite NIO y el endpoint estándar permite base USD, con respuesta JSON documentada (AYR Tech (Pty) Ltd., s. f.-c; AYR Tech (Pty) Ltd., s. f.-d). La autenticación Bearer utiliza la clave en cabecera y elimina ese segmento de la URL (AYR Tech (Pty) Ltd., s. f.-a). La consulta no transmite datos del inquilino. La documentación consultada confirma un contrato técnico y una organización responsable, sin convertir la publicidad de disponibilidad del proveedor en una medición realizada por el equipo.

El plan Free publica precio USD 0, actualización diaria y mil quinientas solicitudes mensuales a la fecha de consulta (AYR Tech (Pty) Ltd., s. f.-b). Los términos permiten caché para uso interno, restringen redistribución y describen las tasas como informativas, con uso transaccional desaconsejado (AYR Tech (Pty) Ltd., s. f.-e). Por esa razón, M09 separa la referencia de la autorización manual por fecha que exige un pago entre monedas. La selección no supone cuenta contratada ni una cuota garantizada de por vida. El responsable del despliegue conserva una copia fechada de las condiciones al habilitar credenciales operativas.

La Tabla 30 presenta proveedor Adoptado y Alternativa Contrastada.

Tabla 30

Proveedor Adoptado y Alternativa Contrastada

Opción	Contrato y Condiciones Consultadas	Decisión
ExchangeRate-API Free	Base USD, NIO, JSON conversion_rates; Bearer; 1500 solicitudes/mes, actualización diaria, USD 0; caché interna sin redistribución.	Adoptada para referencia informativa y RA5; cobro requiere autorización independiente.
Open Exchange Rates Free	API JSON con base, rates y timestamp; registro gratuito publica 1000 solicitudes/mes y actualización horaria (Open Exchange Rates, s. f.-a, s. f.-b).	Alternativa documentada; no adoptada por continuidad del PRD y ausencia de necesidad horaria. No se afirma que carezca de NIO.
BCN	Comunicado institucional fija referencia oficial de 2026; no se verificó contrato JSON público equivalente.	Fuente documental para tasa manual autorizada; sin inventar endpoint ni extraer automáticamente datos de la web.
FXRatesAPI y CurrencyFreaks	Candidatos de la base común; no se presume cuota, licencia ni reputación sin documentación adoptada.	No seleccionados: ya se conserva proveedor del PRD y se contrasta una alternativa atribuible.

Solicitud, Respuesta y Mapeo Persistente

El adaptador envía GET a https://v6.exchangerate-api.com/v6/latest/USD con Accept application/json y Authorization Bearer, cuyo valor procede de LATCH_FX_API_KEY. El registro de diagnóstico elimina esa cabecera y nunca imprime la llave. El JSON ilustrativo incluye únicamente USD y NIO para mostrar los campos que Latch consume; sus números y marcas temporales son sintéticos y no proceden de una llamada autenticada. La respuesta debe tener result igual a success y base_code igual a USD. Una respuesta con HTTP 200 y result error sigue el mismo tratamiento de error semántico que un rechazo del proveedor, sin reemplazar la caché válida anterior.

{

```
  "result": "success",
  "documentation": "https://www.exchangerate-api.com/docs",
  "terms_of_use": "https://www.exchangerate-api.com/terms",
  "time_last_update_unix": 1792713600,
  "time_last_update_utc": "Fri, 23 Oct 2026 00:00:00 +0000",
  "time_next_update_unix": 1792800000,
  "time_next_update_utc": "Sat, 24 Oct 2026 00:00:00 +0000",
  "base_code": "USD",
  "conversion_rates": {"USD": 1, "NIO": 36.50000000}
}
```


La Tabla 31 presenta campos JSON y Mapeo a TipoCambio.

Tabla 31

Campos JSON y Mapeo a TipoCambio

Campo o Dato	Validación	Columna o Uso
result	Obligatorio String success. error-type se lee si es error.	Control del adaptador; no promueve una referencia inválida.
base_code	Obligatorio USD exacto.	base; destino=NIO fijado por el caso de uso.
conversion_rates.NIO	Número decimal finito entre 1 y 100 NIO/USD, inclusive; BigDecimal; escala máxima almacenada de ocho decimales.	valor; HALF_EVEN a ocho decimales si procede. Fuera de banda: no guardar como referencia vigente; bitacora M09, REFERENCIA_EN_CUARENTENA, detalle con valor y huella.
conversion_rates.USD	Si existe, debe ser 1; contradicción rechaza la respuesta.	Verificación de coherencia de base.
time_last_update_unix	Entero epoch válido; no más de cinco minutos en el futuro.	actualizado_en Instant.
time_next_update_unix	Entero epoch posterior a last_update.	proximo_en; condiciona la programación compartida.
time_*_utc	Campos descriptivos opcionales; si existen se contrastan con epoch.	No sustituyen los epoch; ausencia no invalida un epoch correcto.
documentation / terms_of_use	URLs informativas; no se abren ni ejecutan desde la respuesta.	Referencias del contrato conocido, no fuentes arbitrarias de configuración.
Reloj del cliente y proveedor	Instante después de validar; nombre fijo ExchangeRate-API / AYR Tech.	consultado_en, proveedor; creado_por corresponde al

Campo o Dato	Validación	Columna o Uso
		actor de la tarea.
Cuerpo original validado	Límite 256 KiB; SHA-256 calculado, sin copiar secretos.	huella_respuesta; el cuerpo completo no se guarda como datos del pago.

La validación rechaza campos obligatorios ausentes, tipos inesperados, JSON incompleto y valores NIO fuera de la banda técnica de uno a cien NIO por USD adoptada en el flujo. Esa banda deriva anomalías a revisión y no define una cotización válida ni una obligación legal. El adaptador limita conexión a cinco segundos y solicitud total a diez, con cuerpo máximo de doscientos cincuenta y seis KiB, según RNF-039 y TRD. Los campos adicionales no utilizados se ignoran después de comprobar tamaño y esquema. Un fallo conserva la última referencia, muestra su antigüedad y no inventa una nueva tasa para completar el pago. La cuarentena no crea tabla: conserva la referencia vigente anterior y registra M09 / REFERENCIA_EN_CUARENTENA con valor y huella en bitacora.

Caché, Concesión y Respaldo Manual

H3 solicita una concesión de noventa segundos en control_fx y termina esa transacción antes de esperar HTTP. La instancia designada publica la nueva referencia y actualiza proximo_intento en otra transacción, con presupuesto compartido de cien intentos al mes local. El próximo intento toma el mayor instante entre la ventana diaria y time_next_update_unix. Un mes de treinta y un días consume como máximo treinta y una consultas ordinarias y treinta y un reintentos de red, HTTP 408 o HTTP 5xx, por debajo del presupuesto interno. Las aperturas de sesión y las consultas manuales de pantalla leen caché sin reiniciar cuota ni programación.

La Tabla 32 presenta tratamiento de Fallos de Integración.

Tabla 32

Tratamiento de Fallos de Integración

Fallo	Acción	Estado Persistente
Red, HTTP 408 o HTTP 5xx	Un reintento después de 60 s, fuera de transacción.	Contar ambos intentos; conservar caché anterior.
HTTP 429	Respetar Retry-After; si falta, esperar 24 h con aviso.	Guardar proximo_intento; no abrir un bucle de reintentos.
401/403, invalid-key o inactive-account	Suspender actualización hasta corregir credencial o activación.	Incidente sin secreto; las operaciones locales continúan.
quota-reached	Suspender hasta restablecer cuota o intervención del Propietario.	No superar el presupuesto interno ni ocultar la causa.
JSON inválido	Sin reintento automático; rechazar respuesta y conservar referencia anterior con su antigüedad.	No reemplazar caché válida ni autorizar cobro.
Sin tasa aprobada para fecha	Rechazar conversión entre monedas antes de transacción financiera.	Permitir pago en moneda contractual con tasa_usada NULL.

El BCN informó el 20 de octubre de 2025 que el deslizamiento sería cero por ciento anual y la referencia oficial de 2026 sería 36.6243 córdobas por dólar (Banco Central de Nicaragua, 2025). La cifra describe una publicación institucional y no transforma una tasa del proveedor en tasa oficial. El Propietario puede documentar esa fuente al autorizar una fecha, con valor y fundamento registrados. Los datos semilla no precargan ese número como autorización real, pues usan una tasa ficticia rotulada. Si la autorización no existe para la fecha local del pago, la interfaz solicita el acto administrativo y no reutiliza una tasa histórica como vigente.

Fuentes de Datos de los Reportes

Los diez reportes conservan moneda, fecha de corte o intervalo y filtros aplicados en encabezado. El estado de cuenta reúne el detalle de cargos con aplicaciones y crédito, mientras la cartera solo suma deuda vencida a esa fecha. Las vistas v_ingresos_mes y v_gastos_mes sirven

al período vigente y las consultas parametrizadas reconstruyen anulaciones al corte cuando el reporte es histórico. La ocupación es una fotografía actual de reservas, pues el esquema no mantiene una línea temporal completa de cada estado de unidad. Su encabezado identifica instante de consulta y no ofrece un selector de fecha histórica que el modelo no pueda reconstruir.

Recibo De Pago

La Tabla 33 presenta fuente del Reporte Recibo de pago.

Tabla 33

Fuente del Reporte Recibo de pago

Elemento	Especificación
Consulta o vista	SELECT * FROM latch.v_recibo WHERE numero=?
Parámetros	numero BIGINT; actor autorizado.
Campos	numero, pago_id, instantanea, creado_en, estado, motivo_anulacion.
Agrupaciones	Un pago y su distribución congelada en instantanea; sin agrupación cruzada.
Totales	Monto original y equivalente contractual identificados por moneda; no sumar ambos como dos ingresos.

Estado De Cuenta

La Tabla 34 presenta fuente del Reporte Estado de cuenta.

Tabla 34

Fuente del Reporte Estado de cuenta

Elemento	Especificación
Consulta o vista	SELECT * FROM latch.fn_cargos_corte(?) WHERE contrato_id=? ORDER BY vence_el,cargo_id
Parámetros	corte DATE; contrato UUID; consultas auxiliares de pagos/aplicaciones y crédito al mismo corte.
Campos	tipo, periodo, vence_el, importe, abonado, saldo, estado; pagos y recibos, fecha, crédito.

Elemento	Especificación
Agrupaciones	Por contrato y moneda; movimientos por fecha e id; anulaciones visibles con efecto cero.
Totales	Principal no anulado, abonos, deuda y crédito mostrados por separado; deuda no se compensa automáticamente con crédito aún no aplicado.

Cartera Vencida

La Tabla 35 presenta fuente del Reporte Cartera vencida.

Tabla 35

Fuente del Reporte Cartera vencida

Elemento	Especificación
Consulta o vista	SELECT e.*,k.inquilino_id,k.unidad_id FROM latch.fn_cargos_corte(?) e JOIN latch.contrato k ON k.id=e.contrato_id WHERE e.saldo>0 AND e.vence_el<? AND e.estado<>'Anulado' ORDER BY e.vence_el,e.cargo_id
Parámetros	corte DATE repetido en ambos parámetros; propiedad y moneda como filtros adicionales parametrizados.
Campos	Contrato, inquilino, unidad, vencimiento, saldo, moneda y días de atraso calculados con corte.
Agrupaciones	Propiedad, moneda y rangos 1..30, 31..60, 61..90, más de 90 días.
Totales	Sumatoria saldo por rango y moneda; ninguna mezcla NIO/USD.

Ingresos Y Gastos

La Tabla 36 presenta fuente del Reporte Ingresos y gastos.

Tabla 36

Fuente del Reporte Ingresos y gastos

Elemento	Especificación
Consulta o vista	WITH i AS (SELECT u.propiedad_id,p.moneda_original moneda,sum(p.monto_original) ingreso FROM latch.pago p JOIN latch.contrato k ON k.id=p.contrato_id JOIN latch.unidad u ON u.id=k.unidad_id WHERE p.fecha_pago BETWEEN ? AND ? AND (p.anulado_en IS NULL OR p.anulado_en>=latch.fn_fin_corte(?)) AND p.creado_en<latch.fn_fin_corte(?) GROUP BY 1,2), g AS (SELECT propiedad_id,moneda,sum(importe) gasto FROM latch.gasto_propiedad WHERE fecha_gasto BETWEEN ? AND ? GROUP BY 1,2) SELECT coalesce(i.propiedad_id,g.propiedad_id)

Elemento	Especificación
	propiedad_id,coalesce(i.moneda,g.moneda) moneda,coalesce(ingreso,0) ingreso,coalesce(gasto,0) gasto,coalesce(ingreso,0)-coalesce(gasto,0) neto FROM i FULL JOIN g USING(propiedad_id,moneda)
Parámetros	desde/hasta DATE; corte de anulación y registro=hasta; fechas de gasto=desde/hasta.
Campos	Propiedad, moneda, ingreso, gasto y neto; detalle de pagos y gastos en subreportes.
Agrupaciones	Propiedad y moneda; el encabezado distingue flujo de caja de ingreso devengado.
Totales	Neto por moneda = cobros registrados no anulados al corte menos gastos. Garantía recibida y aplicada se informa aparte, sin duplicar caja.

Ocupación Actual

La Tabla 37 presenta fuente del Reporte Ocupación actual.

Tabla 37

Fuente del Reporte Ocupación actual

Elemento	Especificación
Consulta o vista	SELECT propiedad_id,estado,count(*) unidades FROM latch.v_ocupacion GROUP BY propiedad_id,estado ORDER BY propiedad_id,estado
Parámetros	Propiedad UUID opcional; instante generadoEn del snapshot, sin corte histórico.
Campos	Propiedad, Disponible/Ocupada/En mantenimiento/Inactiva y cantidad.
Agrupaciones	Por propiedad y estado de unidad.
Totales	Ocupadas/activas ×100; activas=total menos Inactiva; denominador cero muestra sin unidades activas, no división.

Contratos Por Vencer

La Tabla 38 presenta fuente del Reporte Contratos por vencer.

Tabla 38

Fuente del Reporte Contratos por vencer

Elemento	Especificación
Consulta o vista	SELECT * FROM latch.contrato WHERE estado IN ('Vigente','Por vencer') AND fin BETWEEN ? AND ? ORDER BY fin,id

Elemento	Especificación
Parámetros	desde=fecha local actual; hasta=desde más parámetro dias_por_vencer; propiedad opcional.
Campos	Contrato, inquilino, unidad, fin, días restantes, canon y moneda.
Agrupaciones	Propiedad y moneda; lista cronológica de vencimientos.
Totales	Cantidad de contratos y suma de canon separada por moneda; no se presenta como deuda.

Historial De Pagos

La Tabla 39 presenta fuente del Reporte Historial de pagos.

Tabla 39

Fuente del Reporte Historial de pagos

Elemento	Especificación
Consulta o vista	SELECT p.*,r.numero FROM latch.pago p JOIN latch.recibo r ON r.pago_id=p.id WHERE p.fecha_pago BETWEEN ? AND ? ORDER BY p.fecha_pago,r.numero
Parámetros	desde/hasta DATE; contrato y método opcionales como parámetros.
Campos	Número, fecha, contrato, original, moneda, convertido, tasa, método, estado y motivo de anulación.
Agrupaciones	Contrato, moneda original y método; anulados visibles separados.
Totales	Cobro neto confirmado al corte; original y equivalente en columnas independientes sin sumarlos juntos.

Garantías Y Liquidaciones

La Tabla 40 presenta fuente del Reporte Garantías y liquidaciones.

Tabla 40

Fuente del Reporte Garantías y liquidaciones

Elemento	Especificación
Consulta o vista	SELECT d.*,m.id movimiento_id,m.tipo,m.fecha_movimiento,m.importe,m.concepto,m.metodo FROM latch.v_depositos d LEFT JOIN latch.movimiento_deposito m ON m.deposito_id=d.deposito_id ORDER BY

Elemento	Especificación
	d.contrato_id,m.fecha_movimiento,m.id
Parámetros	Contrato UUID o propiedad; snapshot actual. Para corte histórico se filtran salidas por fecha y creado_en.
Campos	Inicial, aplicado, devuelto, remanente, estado, concepto y método de devolución.
Agrupaciones	Contrato y moneda; UUID del comprobante de movimiento.
Totales	Inicial = aplicaciones + devoluciones + remanente; no sumar estas salidas como ingreso de renta.

Gastos De Mantenimiento

La Tabla 41 presenta fuente del Reporte Gastos de mantenimiento.

Tabla 41

Fuente del Reporte Gastos de mantenimiento

Elemento	Especificación
Consulta o vista	SELECT g.*,s.unidad_id,s.estado FROM latch.gasto_propiedad g LEFT JOIN latch.solicitud_mantenimiento s ON s.id=g.solicitud_id WHERE g.fecha_gasto BETWEEN ? AND ? ORDER BY g.propiedad_id,g.fecha_gasto,g.id
Parámetros	Desde/hasta DATE; propiedad y estado opcionales; la solicitud relacionada debe pertenecer a la propiedad.
Campos	Fecha, propiedad, unidad opcional, solicitud, concepto, importe y moneda.
Agrupaciones	Propiedad, moneda y solicitud; gastos sin solicitud siguen incluidos.
Totales	Suma importe por moneda; detalle separado de cargos Reparación a inquilinos.

Tasas Utilizadas

La Tabla 42 presenta fuente del Reporte Tasas utilizadas.

Tabla 42

Fuente del Reporte Tasas utilizadas

Elemento	Especificación
Consulta o vista	SELECT p.id AS pago_id,p.contrato_id,p.fecha_pago,p.moneda_original, p.monto_original,p.monto_contrato,p.tasa_usada,p.estado, a.id AS autorizacion_id,a.fecha_efectiva,a.valor AS tasa_autorizada, a.fuente AS fuente_autorizacion,a.fundamento,

Elemento	Especificación
	t.id AS referencia_id,t.valor AS tasa_referencia,t.proveedor,t.actualizado_en FROM latch.pago p JOIN latch.autorizacion_tasa a ON a.id=p.autorizacion_id LEFT JOIN latch.tipo_cambio t ON t.id=a.referencia_id WHERE p.fecha_pago BETWEEN ? AND ? ORDER BY p.fecha_pago,p.id;
Parámetros	Desde/hasta DATE de fecha_pago; autorizaciones utilizadas en pagos con conversión. No sustituye la tasa histórica por la referencia actual.
Campos	Pago, contrato, fecha, monedas/importes, tasa usada, estado, autorización por fecha, fuente/fundamento y referencia informativa opcional.
Agrupaciones	Por fecha efectiva, autorización y moneda; pagos anulados visibles con su estado.
Totales	Cantidad de pagos e importes por moneda y estado. Las tasas no se suman; conciliación de tasa_usada con el registro autorizado.

Los filtros opcionales se agregan con fragmentos SQL predefinidos y valores parametrizados, sin concatenar nombres recibidos del usuario. Los campos de orden permitidos pertenecen a una lista cerrada del adaptador y los valores se enlazan con PreparedStatement. La consulta se materializa bajo la misma instantánea cuando necesita subreportes y el servicio conserva la fecha generada junto al DTO. Un archivo con cero filas mantiene encabezados, filtros y totales cero por moneda aplicable. La exportación no recalcula estados dentro de Jasper ni utiliza una tasa actual para alterar un recibo histórico; presenta los hechos que la fuente del reporte entregó en su corte.

Seguridad en la Base de Datos

Los roles funcionales Propietario y Asistente pertenecen a Latch y se comprueban en sus servicios. PostgreSQL usa cuentas técnicas distintas para propiedad del esquema, ejecución, migración y respaldo; LOGIN permite abrir sesión y NOLOGIN identifica al propietario de objetos (PGDG, s. f.-g). La aplicación no distribuye la credencial del migrador al operador cotidiano. El script de privilegios del Apéndice A crea los roles sin contraseña fija y traslada la propiedad de tablas, vistas y rutinas a latch_owner. El DBA asigna secretos mediante el comando

interactivo de psql, sin almacenarlos en Git, argumentos de procesos o archivos SQL de la entrega.

La Tabla 43 presenta roles Técnicos y Privilegios Mínimos.

Tabla 43

Roles Técnicos y Privilegios Mínimos

Rol	LOG IN	Privilegios Concedidos	Límite
latch_owner	No	Propiedad de esquema, tablas, vistas y rutinas.	Solo asumido para migraciones; nunca credencial de JavaFX.
latch_runtime	Sí	USAGE del esquema, SELECT; INSERT/UPDATE definidos por tabla y por columna; EXECUTE de helpers y procedimiento.	Sin CREATE, DELETE, TRUNCATE, SUPERUSER, CREATEDB, CREATEROLE ni REPLICATION.
latch_migrator	Sí	Miembro de latch_owner; SET ROLE para migrar y registrar huellas.	Credencial separada; clientes detenidos; sin permisos administrativos de clúster.
latch_backup	Sí	SELECT en tablas y UPDATE de la columna de puerta necesaria para tomar bloqueo.	Lee credenciales cifradas por hash dentro del dump; paquete protegido; sin escritura financiera.

La Tabla 44 presenta matriz de Acceso SQL de Ejecución.

Tabla 44

Matriz de Acceso SQL de Ejecución

Objetos	SELECT	INSERT	UPDATE	DELETE
Usuario, propiedad, unidad, inquilino, contrato, cargo, mantenimiento, documentos y control_fx	Sí	Sí	Sí; servicios limitan campos y rol funcional.	No
Pago	Sí	Sí	Solo estado, anulado_en, anulado_por y motivo_anulacion.	No

Objetos	SELECT	INSERT	UPDATE	DELETE
Recibo, garantía, movimientos, gasto, referencia de tasa, autorización y bitácora	Sí	Sí	No	No
AplicacionPago	Sí	Sí	Solo revertida_en, revertida_por y motivo_reversion.	No
ConsecutivoRecibo	Sí	No	Solo siguiente_numero, dentro de transacción H4.	No
Rol, esquema_migracion	Sí	No	No	No
PuertaMantenimiento	Sí	No	Solo descripcion, para privilegio requerido por bloqueo; sin cambio operativo del texto.	No
Vistas	Sí	No	No	No
parametro_sistema	Sí	Sí; un registro por cambio	No; historial de solo inserción	No

PostgreSQL permite privilegios por columna y distingue permisos sobre tablas, esquemas y rutinas; el diseño usa esa separación para impedir modificaciones de importes de pago o contenido de recibo desde la cuenta de ejecución (PGDG, s. f.-j). No existe correspondencia de una cuenta PostgreSQL por inquilino o por cada rol funcional. Una persona con acceso a la credencial técnica puede intentar operaciones SQL que eviten la autorización Java, por lo que el despliegue supone cuentas de Windows confiables y archivos de configuración con ACL restringida. Las pruebas de permisos funcionales deberán invocar servicios con ambos roles y no limitarse a revisar botones ocultos.

Credenciales, Conexión y Cifrado en Tránsito

Las credenciales de Latch usan Argon2id con diecinueve MiB, dos iteraciones, paralelismo uno, sal aleatoria de dieciséis bytes y derivado de treinta y dos, de acuerdo con ADR-07 y el mínimo de OWASP adoptado (OWASP Foundation, s. f.). El valor PHC conserva

parámetros y sal junto al derivado sin guardar la contraseña en texto. Bouncy Castle proporciona la implementación Java utilizada por la herramienta de aprovisionamiento (Legion of the Bouncy Castle, s. f.). La comparación debe ser de tiempo constante y la interfaz no informa si la cuenta inexistente o el secreto incorrecto provocó el rechazo. Los logs tampoco copian hashes o contraseñas.

El despliegue LAN exige TLS con verificación de certificado y nombre del servidor, usando sslmode=verify-full y sslrootcert en pgJDBC. La documentación del controlador distingue ese modo de otros que cifran sin verificar la identidad con el mismo alcance (PostgreSQL JDBC Driver, s. f.-c). PostgreSQL debe habilitar ssl, certificado y clave del servidor, y limitar conexiones mediante reglas hostssl con scram-sha-256 (PGDG, s. f.-o; PGDG, s. f.-q). La instalación local usa el mismo contrato con un certificado que incluya localhost. Las comprobaciones SQL y de TLS quedan pendientes de reejecución; el diseño no acredita un despliegue cifrado ni validación de certificados.

La Tabla 45 presenta configuración de Conexión Adoptada.

Tabla 45

Configuración de Conexión Adoptada

Elemento	Valor o Regla
URL JDBC local	jdbc:postgresql://localhost:5432/latch?sslmode=verify-full&ApplicationName=Latch&tcpKeepAlive=true
Certificado raíz	Properties.sslrootcert = C:/ProgramData/Latch/certs/ca.crt; CA de la instalación, fuera del repositorio.
Usuario y secreto	LATCH_DB_USER=latch_runtime; contraseña leída de configuración externa protegida. Nunca se añade a la URL.
Servidor	ssl=on; ssl_cert_file y ssl_key_file apuntan al certificado y clave provisionados; ssl_min_protocol_version=TLSv1.2.
pg_hba.conf	hostssl latch latch_runtime 127.0.0.1/32 scram-sha-256 para local; en LAN la instalación registra solo direcciones de clientes autorizados, no una red pública universal.

Elemento	Valor o Regla
Pool del TRD	maximumPoolSize=6; minimumIdle=1; connectionTimeout=3000 ms; validationTimeout=1000 ms; maxLifetime=1800000 ms. Tres clientes: hasta 18 conexiones operativas.
Comprobación de conexión	Inspeccionar pg_stat_ssl para la sesión; certificado no confiable o nombre incorrecto debe impedir conexión.
Propiedades pgJDBC	Usuario, contraseña, sslrootcert y currentSchema=latch se pasan con Properties; el driver soporta esa configuración (PostgreSQL JDBC Driver, s. f.-b).

Los certificados y sus claves son material de instalación y requieren una CA confiable para los equipos; el documento no distribuye certificados de confianza ficticios. La variante LAN usa un nombre de servidor contenido en el certificado y sustituye el host mediante configuración externa validada. El administrador de la instalación publica CA y permisos de archivos antes de abrir accesos de red, y restringe el puerto a los clientes previstos. Una falla de verificación produce error de conexión, sin degradar a sslmode=disable o trust. La autenticación SCRAM de PostgreSQL protege su secreto técnico y es independiente del hash Argon2id de las cuentas funcionales de Latch.

Privacidad, Retención y Datos de Demostración

El diseño conserva los campos mínimos para explicar el alquiler y mantiene identificación, emergencia y adjuntos como opcionales. Los artículos 11 y 12 de la Ley N.º 787, en el ejemplar original de La Gaceta, Diario Oficial, N.º 61 del 29 de marzo de 2012 (p. 2404), orientan las medidas de seguridad y la confidencialidad del tratamiento de datos personales (Asamblea Nacional de la República de Nicaragua, 2012). Esa lectura del texto de 2012 orienta controles de diseño y no certifica una revisión consolidada de toda reforma o trámite vigente. El prototipo no contiene expedientes reales ni participantes externos, por lo que sus pruebas de privacidad usan identidades y archivos sintéticos rotulados.

La Tabla 46 presenta retención Adoptada y Separación de Datos.

Tabla 46

Retención Adoptada y Separación de Datos

Categoría	Valor Adoptado	Aplicación y Límite
Demostración académica	Hasta 31 de diciembre de 2026.	Propuesta del PRD para demo y respaldos; fixtures sin datos reales pueden conservarse en Git. La entrega no ejecuta una purga automática.
Expediente identificable futuro	Ventana administrativa propuesta de cinco años desde el cierre.	Acceso restringido al Propietario después del cierre; no se presenta como plazo legal universal. Suspender purga ante conservación justificada.
Contactos y adjuntos innecesarios	Revisión al cerrar el contrato.	Suprimir cuando cese finalidad y no exista deber de conservación, con decisión y evento; no usar eliminación ciega por antigüedad.
Bitácora técnica sintética	Noventa días o purga de demo, lo primero.	Rotar logs técnicos; la bitácora financiera se conserva con el expediente y no comparte ese plazo corto.
Respaldos de operación	Siete diarios y cuatro semanales.	Paquete cifrado y ACL; rotación registrada, separada de la conservación del expediente.
Preferencias visuales	Local por id opaco hasta desinstalación o limpieza del usuario.	Sin nombre, cédula o dato contractual en la clave; sin tabla del dominio, RNF-059.

Respaldo, Recuperación y Migración

La política conserva respaldo manual diario y alerta cuando el último conjunto válido supera veinticuatro horas, según RNF-038. pg_dump produce una copia lógica consistente en formato personalizado y pg_restore reconstruye ese archivo; las herramientas no respaldan por sí mismas el directorio externo de adjuntos (PGDG, s. f.-l; PGDG, s. f.-m). Por esa razón, cada escritura adquiere la puerta compartida y el respaldo toma FOR UPDATE sobre la misma fila antes de copiar base y archivos. Una conexión coordinadora mantiene el bloqueo mientras otro proceso realiza pg_dump. Los adjuntos residen en una carpeta local en la instalación individual y en un recurso SMB en LAN, conservando rutas internas relativas y el mismo paquete de

respaldo. Una copia incompleta o sin manifiesto verificado no se presenta como un conjunto recuperable.

Procedimiento de Copia y Recuperación

El paquete contiene base.dump, adjuntos, manifest.json, revisión del esquema y SHA-256 de cada archivo, con fecha de corte y contador del recibo. Las credenciales, la llave API y la clave privada TLS quedan fuera del conjunto académico de respaldo. El TRD adopta cifrado AES-256-GCM con contraseña de respaldo derivada por PBKDF2-HMAC-SHA256, seiscientas mil iteraciones, sal de dieciséis bytes e IV único de doce bytes por cifrado (Cuadra et al., 2026d). La autenticidad se verifica antes de publicar una restauración y el secreto de recuperación se custodia separado del archivo. El dump lógico, el empaquetado cifrado y los adjuntos reales quedan pendientes de reejecución y validación. Un hash incorrecto conserva ERR-055 / MSG-079; autenticidad o contraseña inválida, ERR-059 / MSG-082, sin usar el error de integridad financiera.

La Tabla 47 presenta procedimiento de Respaldo con Herramientas PostgreSQL.

Tabla 47

Procedimiento de Respaldo con Herramientas PostgreSQL

Paso	Acción y Comando	Comprobación
1	Propietario inicia respaldo; M12 abre conexión coordinadora y BEGIN; respaldo_lock_timeout_ms=3000; SET LOCAL lock_timeout='3000ms'; SELECT id FROM latch.puerta_mantenimiento FOR UPDATE.	Esperar escrituras previas; si hay timeout, cancelar sin paquete final.
2	Crear directorio temporal de destino mediante DirectoryChooser y verificar espacio; ejecutar pg_dump --dbname="service=latch_backup" --format=custom --file=base.dump.	Exigir código de salida 0; servicio de libpq configurado por instalación con host, base, TLS y cuenta sin secreto en el comando.
3	Copiar el directorio controlado de adjuntos a adjuntos dentro del paquete temporal, manteniendo la puerta.	Validar tamaño y SHA-256 contra metadatos; archivo

Paso	Acción y Comando	Comprobación
		ausente impide publicar.
4	Construir manifest.json con ruta relativa, bytes, huella, fecha y esquema; ejecutar pg_restore --list base.dump.	El índice debe abrirse; comparar archivos y conteo de referencias.
5	Cifrar y verificar el paquete temporal; publicar por cambio de nombre en el mismo volumen.	Solo después de cerrar y validar se informa Respaldo válido.
6	COMMIT en conexión coordinadora; registrar resultado administrativo sin secretos.	Liberar puerta incluso ante falla; conservar diagnóstico de copia fallida sin confundirlo con respaldo válido.

Los objetivos adoptados son quince minutos para copiar un GiB de base y un GiB de adjuntos, y treinta minutos para recuperar ese conjunto con cero huellas discordantes. Esas cifras conservan RNF-036 y RNF-037 como metas propuestas, sin medición de esa carga en el entorno utilizado. El RPO de veinticuatro horas depende de ejecutar la copia diaria y el RTO de treinta minutos empieza cuando operador y paquete están disponibles. La copia lógica no proporciona recuperación a un instante entre respaldos. Si el alcance futuro exigiera un RPO menor, requeriría copia base y archivado WAL, conforme al mecanismo PITR documentado por PostgreSQL (PGDG, s. f.-c). Las mediciones de aceptación identificarán entorno y revisión; no se presentan como resultados ejecutados.

La Tabla 48 presenta procedimiento de Restauración sin Sobrescribir la Base Activa.

Tabla 48

Procedimiento de Restauración sin Sobrescribir la Base Activa

Paso	Acción y Comando	Criterio de Avance
1	Detener clientes y tareas; tomar copia previa del conjunto activo; el Propietario autoriza mantenimiento.	Ninguna escritura activa y paquete previo identificado.
2	Verificar autenticidad del paquete cifrado, manifest.json y SHA-256 antes de publicar archivos extraídos.	Cero huellas discordantes; conservar base activa ante cualquier falla. Verificar

Paso	Acción y Comando	Criterio de Avance
		autenticidad GCM antes de sustituir la instalación.
3	Una cuenta administrativa del clúster con CREATEDB, externa a latch_owner/runtime/migrator/backup, crea el destino: createdb --host=localhost --port=5432 --username=postgres --maintenance-db=postgres --template=template0 --encoding=UTF8 --locale=C latch_recovery_20261126. La instalación sustituye postgres por su cuenta administrativa real.	Nombre de ejemplo planificado; el instalador agrega sufijo único si ya existe y nunca borra una base para reutilizarlo.
4	Con la misma cuenta administrativa externa: pg_restore --host=localhost --port=5432 --username=postgres --dbname=latch_recovery_20261126 --exit-on-error --no-owner base.dump. Autenticación por mecanismo externo seguro; sin contraseña en argumentos. Reasignar propiedad y permisos con V004 en el destino.	Código 0; roles ya provisionados y propiedad reasignada mediante V004 en el destino.
5	Ejecutar CALL latch.sp_verificar_integridad(?) para cada contrato; comparar conteos, relaciones y números de recibo.	Contador igual al máximo +1; números 1..máximo conservados, incluidos anulados.
6	Restaurar adjuntos a directorio nuevo y comprobar referencias; iniciar Latch en consulta con la nueva configuración.	Apertura, recibo y estado de cuenta coinciden; no se borran archivos activos.
7	Conciliar recibos emitidos después del corte disponible con evidencia externa; cambiar configuración de clientes a la base recuperada solo al cerrar diferencias.	Si faltan pagos emitidos, mantener solo lectura y recuperar historia; no emitir otra vez los mismos números. No reutilizar ni renumerar consecutivos; mantener mantenimiento hasta conciliar.
8	Registrar revisión, operador, paquete, corte y resultado; conservar base anterior como copia de recuperación según política.	Validación funcional documentada antes de liberar escrituras.

Versionado y Política de Migración

La instalación aplica V001 modelo, V002 objetos, V003 catálogos y V004 privilegios en ese orden, con una identidad administrativa de aprovisionamiento para crear roles. Las migraciones posteriores usan latch_migrator y SET ROLE latch_owner, sin entregar esa credencial al escritorio. Cada script nuevo registra nombre, SHA-256 e instante en

esquema_migracion después de ejecutarse con éxito; una huella distinta de una versión aplicada detiene la apertura. La herramienta de migración conserva una sola sesión exclusiva mediante la puerta y un bloqueo asesor transaccional para impedir dos migradores. Hibernate inicia con hbm2ddl.auto=validate y nunca modifica el esquema operativo con update o create-drop. Cada incremento identifica versión, fuentes, esquema compatible, documentos, respaldo y huellas de entrega; los cambios de alcance o configuración registran motivo, impacto y responsable.

La Tabla 49 presenta scripts y Orden de Aplicación.

Tabla 49

Scripts y Orden de Aplicación

Archivo	Contenido	Política
V001__modelo.sql	23 tablas, claves, restricciones e índices.	Primera instalación, transacción completa; no es un script idempotente para repetir sobre datos existentes.
V002__objetos.sql	10 vistas, 6 funciones y 1 procedimiento.	Después del DDL; define search_path y revoca ejecución PUBLIC.
V003__catalogos.sql	Dos roles funcionales, contador inicial, puerta, control_fx y parámetros.	Sin cuenta activa ni contraseña fija; no ejecutar después de emitir recibos.
V004__privilegios.sql	Roles técnicos y permisos por tabla/columna; cambio de propietario.	Una vez por clúster en el aprovisionamiento inicial. Destinos con roles existentes usan el modo de reasignación previsto, sin recrearlos.
T001__demostracion.sql	Fixture sintético completo, cuentas inactivas con hashes calculados.	Solo base de prueba vacía después de V003. No se mezcla con operación real ni se ejecuta como migración.
V005 y posteriores	Un cambio por versión con revisión, checksum y respaldo previo.	No editar V001 ya aplicada; ampliar de forma compatible y probar conversión de datos antes de activar clientes.

Una migración que agregue campos opcionales precede al cliente que los usa; si después se exige NOT NULL, primero convierte y valida los registros históricos. Las modificaciones de catálogos incluyen el convertidor Java y su prueba de lectura de valores previos. Ante un fallo

transaccional, el migrador revierte el script y conserva la última versión registrada. Un cambio que haya confirmado no se deshace con una edición manual del historial: utiliza una migración correctiva o restaura el conjunto completo bajo mantenimiento. La devolución de binarios a una versión anterior solo ocurre si su contrato sigue siendo compatible con el esquema activo.

Datos Semilla y Datos de Prueba

V003 crea únicamente los roles Propietario y Asistente, las filas de control y los parámetros iniciales. Los estados, monedas y métodos del catálogo están expresados completos mediante CHECK y en la tabla de catálogos del cuerpo, por lo que no requieren filas adicionales de catálogo. La cuenta inicial se crea con Aprovisionar.java del Apéndice C, desde consola local sin eco. La herramienta recibe el secreto de conexión y la contraseña inicial en ejecución, calcula Argon2id y exige confirmación de doce a ciento veintiocho caracteres. La transacción comprueba ausencia de otro Propietario activo, crea la cuenta con cambio_inicial y registra el evento.

Aprovisionamiento y Parámetros Iniciales

La herramienta de aprovisionamiento usa la credencial temporal de instalación o el migrador autorizado, nunca la cuenta cotidiana del Asistente. La configuración externa aporta LATCH_DB_URL y LATCH_PROVISION_USER sin secretos incrustados en el código, y la consola solicita las contraseñas necesarias al ejecutar. El primer acceso permite únicamente cambiar la clave propia y cerrar sesión hasta completar cambio_inicial. Las cuentas sintéticas de T001 están inactivas; la generación y validación de sus hashes de demostración quedan pendientes de reejecución y no acreditan el aprovisionamiento Java.

La Tabla 50 presenta parámetros Iniciales del Sistema.

Tabla 50

Parámetros Iniciales del Sistema

Clave	Valor Adoptado	Regla o Justificación — cambios por INSERT con vigencia y motivo
dias_gracia / mora_tipo / mora_valor	3 / Porcentaje / 0	RN-05: primer despliegue sin mora monetaria hasta configuración del Propietario.
dias_por_vencer / h2_intervalo_minutos	30 / 5	RN-12: revisión periódica de contratos; estados del cargo se calculan en consulta.
max_intentos / bloqueo_minutos	5 / 15	RN-15; hash y longitud de clave son parámetros de seguridad fijados en el código revisado.
fx_intervalo_horas / fx_presupuesto_mes	24 / 100	RN-22 y RNF-040; compartidos en instalación LAN.
fx_concesion_segundos	90	Recupera actualización si la instancia designada deja de responder.
inactividad_minutos	15	Minutos de inactividad antes de bloquear la sesión; rango 5–60. El límite de adjunto de 5 MiB es fijo en código y CHECK, no configurable.
emisor	Latch demostración académica; Managua, datos sintéticos.	Datos de demostración identificados, sin inventar un propietario participante. Logotipo en documento_adjunto.parametro_id del registro emisor; propiedad_id no representa el logotipo.
retencion_finanzas_anios / retencion_logs_dias	5 / 90	Supuestos administrativos del PRD; no habilitan purga automática ni fijan plazos legales.
revision_adjuntos_al_cierre	true	Revisión por finalidad al cerrar, sin un plazo fijo universal de eliminación.

Escenarios Sintéticos y Verificación

El fixture pequeño contiene dos propiedades, ocho unidades de los cuatro tipos, seis personas ficticias y ocho contratos que cubren el catálogo contractual. Los cuatro pagos utilizan los métodos Efectivo, Transferencia, Depósito bancario y Otro, con un pago anulado cuyo

número permanece visible. Una operación de mil cuatrocientos sesenta córdobas usa una tasa ficticia de 36.50000000 para registrar cuarenta dólares de equivalente contractual. Otro pago deja quinientos córdobas de crédito después de cubrir una renta. Las garantías incluyen Retenido, Aplicado, Devuelto parcial y Devuelto; una devolución parcial conserva el contrato pendiente de cierre hasta agotar remanente, según FLU-31 del App Flow.

La Tabla 51 presenta escenarios de Prueba del Esquema.

Tabla 51

Escenarios de Prueba del Esquema

Escenario	Procedimiento y Datos Sintéticos	Resultado Esperado y Criterio	CP equivalente
Contrato duplicado	Dos activaciones de la misma unidad; distintos UUID de contrato y un inquilino existente.	Máximo un contrato activo; segundo intento 23505 o UnidadReservadaException.	CP-052
Duración y día de pago	Inicio 31/01/2026, fin 28/02/2026; día 31 en febrero.	Un mes calendario válido y vencimiento 28/02; fin igual a inicio rechazado.	CP-015
Renta y mora únicas	Repetir H1 para octubre y H2 para la misma renta vencida.	Una renta y a lo sumo una mora; cero mora sobre Mora.	CP-021 / CP-022
Pago parcial y conversión	Cargo USD 100; recibir NIO 1460 a tasa sintética 36.5 del 23/10.	Equivalente USD 40; saldo USD 60; diferencia de conciliación cero centavos.	CP-026 / CP-029
Excedente y siguiente cargo	Pago NIO 3500 contra renta NIO 3000; generar siguiente cargo con crédito.	Crédito inicial 500; aplicación posterior 500; no generar segundo recibo.	CP-028
Anulación dependiente	Pago con aplicación posterior Credito; intentar anular, revertir dependencia y repetir.	Primero rechazo; después una anulación con número conservado y motivo auditado.	CP-032 / CP-033
Garantía y cierre	Aplicación total, devolución total y devolución parcial con concepto.	Ecuación exacta; remanente cero para cerrar; parcial queda pendiente de liquidar.	CP-034 / CP-036

Escenario	Procedimiento y Datos Sintéticos	Resultado Esperado y Criterio	CP equivalente
Consecutivo revertido	Variante a: diez emisores en dos JVM, contador inicial 1;100 pagos confirmados y otras diez solicitudes con rollback antes del commit.	100 recibos y siguiente_numero=101; las diez solicitudes revertidas no consumen número.	CP-041 / CP-042 (variante a)
Emisores y anulaciones	Variante b: diez emisores en dos JVM, contador inicial 1;100 pagos confirmados y diez anulaciones posteriores.	100 recibos,90 pagos Registrado y10 Anulado; siguiente_numero=101, sin renumerar.	CP-041 / CP-032 (variante b)
Permisos SQL	SET ROLE latch_runtime; UPDATE de número, importe o DELETE de bitácora.	Cien por ciento de esos intentos rechazados con 42501.	CP-005 / CP-040 (variante de permisos SQL)
Integración JSON	Base EUR, NIO ausente/negativo, tiempos invertidos, body >256 KiB, 408/429/5xx simulados.	Rechazar inválidos sin sustituir caché;408/red/5xx: único reintento 60 s;429: Retry-After y cuota; banda manual fuera 1–100: revisión del Propietario.	CP-058 / CP-064
Restauración	Clientes detenidos; validar paquete, hashes y GCM; restauración aislada de base 1 GiB y adjuntos 1 GiB antes de sustituir instalación.	Misma historia y próximo número; conciliar recibos posteriores sin reutilizar ni renumerar. Metas 15 min respaldo / 30 min recuperación, pendientes de medición en entorno identificado.	CP-051 / CP-095 (variantes)
Carga V1 y UI	500 cargos y 100000 eventos; H1/H5 en JavaFX con pulsos medidos.	Metas RNF-001 a RNF-004 y RNF-020/021; plan futuro, no ensayo ya ejecutado.	CP-098 / CP-099

Trazabilidad

La trazabilidad conserva los identificadores del PRD, los flujos consultados y las decisiones ADR del TRD, sin inventar una nueva serie de requisitos aprobados. Cada tabla y restricción del diccionario tiene una regla RN o un módulo responsable; los servicios vinculan sus capacidades con los RF correspondientes. Las reglas visuales RN-35 a RN-38 se resuelven

fuera de la base y su ausencia de entidades es una decisión verificable, no una omisión de persistencia. El Apéndice F reúne fuentes y fecha de consulta. Las tablas de resultados distinguen lo ejecutado sobre SQL de los criterios previstos para una aplicación que todavía debe implementarse e integrarse. CP-001 a CP-100 del Plan son los únicos identificadores de ejecución; los escenarios de esta tabla y ESC/PT/PV/EV conservan su función de diseño. Las comprobaciones SQL se registran como variantes de esos CP, sin crear otra serie.

La Tabla 52 presenta requisitos y Reglas contra Persistencia y Servicios.

Tabla 52

Requisitos y Reglas contra Persistencia y Servicios

Requisitos y Reglas	Tablas	Restricciones u Objetos	Servicio y Decisión
RN-01, RN-11; RF-M02-07..11, RF-M04-05	unidad, contrato, propiedad, inquilino	uq_contrato_activo; FK RESTRICT; bloqueo de unidad.	PropiedadUnidadService, ContratoService; ADR-04.
RN-02, RN-17; RF-M04-02/04	contrato, cargo	ck_contrato_duracion; día 1..31; período día 1.	ContratoService, CobranzaService.
RN-03, RN-09, RN-22, RN-32; RF-M06-05/06, RF-M09-09	contrato, pago, tipo_cambio, autorizacion_tasa	CHECK de moneda/tasa; uq_autorizacion_fecha; procedimiento financiero.	PagoService, TipoCambioService; ADR-08, ADR-20.
RN-04, RN-05, RN-20; RF-M05-01..07	cargo, contrato	uq_cargo_renta, uq_cargo_mora; origen y bloqueo común.	CobranzaService; H1/H2.
RN-06, RN-24; RF-M05-08..12, RF-M06-01..04	pago, aplicacion_pago, cargo	FK y suma bajo bloqueo; v_credito_pago, fn_cargos_corte.	PagoService, CobranzaService; RA2/RA4.
RN-07, RN-21; RF-M06-07/08/11/12	consecutivo_recibo, recibo, pago	uq_recibo_numero, uq_recibo_pago, uq_pago_solicitud.	PagoService; ADR-16; H4.
RN-08, RN-13, RN-31, RN-33; RF-M06-09/10	pago, recibo, aplicacion_pago, bitacora	Permisos de columnas; motivo/actor/instante; dependencias.	PagoService, DepositoService; FLU-27.
RN-10, RN-26; RF-	deposito_garantia,	uq_deposito_contrato,	ContratoService,

Requisitos y Reglas	Tablas	Restricciones u Objetos	Servicio y Decisión
M07-01..07	movimiento_deposito, aplicacion_pago	ck_movimiento_destino, uq_aplicacion_movimiento.	DepositoService; FLU-29..31.
RN-12, RN-19, RN-34; RF-M04-08..13	contrato, cargo	Estado y cierre; v_contratos_por_vencer; bloqueos.	ContratoService, CobranzaService, PanelService; H2.
RN-14; RF-M12-04/05	bitacora, usuario; auditoría común en entidades	FK de actor, ix_bitacora_correlacion; INSERT sin UPDATE/DELETE.	ConfiguracionAuditoriaService y servicios financieros.
RN-15, RN-16; RF-M01-01..11	rol, usuario, parametro_sistema	uq_usuario_nombre; hash PHC; version y contador de intentos.	SeguridadService; ADR-07.
RN-18, RN-25; RF-M03-01..09	inquilino, documento_adjunto	ck_documento_dueno, tamaño/ruta/mime/huella.	InquilinoService; metadatos y archivos controlados.
RN-23, RN-27; RF-M08-01..07	solicitud_mantenimiento, gasto_propiedad, cargo	FK de propiedad; estados de cierre; cargo independiente.	MantenimientoService, CobranzaService.
RN-28; RF-M10-01..13, RF-M11-01..08	Vistas, cargo, pago, gasto, garantía	Corte local; moneda separada; snapshot de lectura.	ReporteService, PanelService; ADR-09; H5.
RN-29, RN-30; RF-M12-06..08	puerta_mantenimiento, esquema_migracion	Fila única; versión y checksum; cierre de recursos.	ConfiguracionAuditoriaService; ADR-17; H6.
RN-35..38; RF-M01-12..15, RF-M11-09/10	Ninguna tabla de dominio para tema o efectos.	Preferencias locales por UUID; cero columna visual, RNF-059.	Servicio de presentación ThemeService; ADR-18.

Responsables y Capacidad Propuesta

La responsabilidad nominal, el relevo y la revisión cruzada se rigen por el Plan, Tablas 4 –5 y «Revisión Cruzada y Decisión Técnica» (Cuadra et al., 2026f). La Tabla 53 reproduce la matriz RACI única y no agrega un mapa independiente. Las tareas TAR del Apéndice A del Plan determinan quién ejecuta cada actividad. Cada incremento conserva observaciones identificables, responsable y resolución, clasificadas como defecto, aclaración o cambio de alcance, con

seguimiento de riesgos. La evidencia académica describe lo disponible: revisión documental o actividad prevista no acredita una prueba ejecutada.

La Tabla 53 presenta matriz RACI Propuesta del Backend.

Tabla 53

Matriz RACI del Plan de Implementación

Entregable	Steven Cuadra	Alfredo Montalvan	Caleb Tardencilla	Aris Gavarrete
Alcance y calendario	A/R	C	C	C
Seguridad y roles	A/R	C	C	R
Esquema y migraciones	C	A/R	I	C
Persistencia y transacciones	R	A/R	C	C
FXML y temas	C	I	A/R	R
Concurrencia y cierre	A/R	R	C	C
HTTP y caché	R	C	C	A/R
Reportes y oráculos	C	R	C	A/R
Pruebas y conformidad visual	R	R	A/R	R
Manual y defensa	A/R	R	R	R
App-image ZIP y respaldo de entrega	C	R	C	A/R

Nota. A: responsable final; R: ejecuta; C: consultado; I: informado. A/R corresponde a la misma persona, sin duplicar un segundo responsable final.

La Tabla 54 presenta calendario y Reserva de Capacidad del Backend.

Tabla 54

Calendario y Capacidad del Proyecto según el Plan

Período	TAR del Plan Apéndice A	Carga del Proyecto por Persona / Equipo	Capacidad / Reserva del Equipo
Sprint 0 — 22 –27/10 de 2026	TAR-001 a TAR-016	8 h por persona; 32 h equipo	41.142857 h / 9.142857 h

Período	TAR del Plan Apéndice A	Carga del Proyecto por Persona / Equipo	Capacidad / Reserva del Equipo
Sprint 1 — 28/10–3/11 de 2026	TAR-017 a TAR-032	10 h por persona; 40 h equipo	48.000000 h / 8.000000 h
Sprint 2 — 4–10/11 de 2026	TAR-033 a TAR-048	10 h por persona; 40 h equipo	48.000000 h / 8.000000 h
Sprint 3 — 11–17/11 de 2026	TAR-049 a TAR-064	10 h por persona; 40 h equipo	48.000000 h / 8.000000 h
Sprint 4 — 18–25/11 de 2026	TAR-065 a TAR-080	11 h por persona; 44 h equipo	54.857143 h / 10.857143 h
Cierre — 26/11–1/12 de 2026	TAR-081 a TAR-096	8 h por persona; 32 h equipo	41.142857 h / 9.142857 h
Total — 41 días	TAR-001 a TAR-096; suma del Plan	57 h por persona; 228 h equipo	281.142857 h / 53.142857 h

El calendario único del Plan comprende 41 días del 22 de octubre al 1 de diciembre de 2026. Las 96 tareas TAR-001 a TAR-096 suman 228 horas programadas: 32, 40, 40, 40, 44 y 32 por período. La capacidad es 48 × 41/7 = 281,142857 horas y la reserva 53,142857 horas, 18,902439 % de la capacidad. Cada integrante tiene 57 horas programadas y 70,285714 de capacidad. La Tabla 54 muestra el proyecto completo para evitar una segunda bolsa de backend no sustentada en TAR; no duplica esas horas como carga adicional (Cuadra et al., 2026f, Apéndice A, Tablas A1–A6).

Verificaciones Pendientes y Alcance del Plan

La Tabla 55 queda pendiente de reejecución tras modificar V001, V002, V003, V004, T001 y R001. Se retiran todos los PASS y valores observados del ensayo anterior porque no prueban los scripts corregidos. Las comprobaciones que se logren ejecutar sobre esta revisión se entregan en un registro separado con evidencia reproducible y sus límites. Ninguna prueba SQL

acredita integración JavaFX, Hibernate, JasperReports, cifrado AES-256-GCM o consumo HTTP autenticado.

La Tabla 55 presenta las verificaciones pendientes de reejecución sobre PostgreSQL 18.

Tabla 55

Verificaciones Pendientes de Reejecución sobre PostgreSQL 18

Comprobación	Resultado Obtenido	Evidencia
Ejecutar V001__modelo.sql	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Ejecutar V002__objetos.sql	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Ejecutar V003__catalogos.sql	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Ejecutar T001__demostracion.sql	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Fuentes SQL de los diez reportes y complementos	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Duración mínima de un mes	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Contrato activo único	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Renta única por período	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución

Comprobación	Resultado Obtenido	Evidencia
		nueva.
HALF_EVEN y signo	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Cuatro estados de garantía	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Excedente conservado como crédito	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Rollback conserva número disponible	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Emisión simultánea sin huecos	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Control optimista por versión	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Sobreaplicación rechazada antes de commit	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Recibo inmutable por privilegios	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Auditoría sin DELETE	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Pago sin edición de importe	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución

Comprobación	Resultado Obtenido	Evidencia
		nueva.
Respaldo lógico y restauración aislada	pendiente de reejecución	Resultado anterior retirado tras modificar scripts. Registrar entorno, revisión y evidencia de una ejecución nueva.
Ejecutar V004__privilegios.sql	pendiente de reejecución	Verificar en particular SELECT/INSERT y rechazo de UPDATE/DELETE en parametro_sistema.

Las pruebas futuras de servicio cubrirán autenticación concurrente, veinte intentos de permisos funcionales, diez emisores concurrentes repartidos en dos JVM, con variantes separadas de cien pagos confirmados y diez solicitudes revertidas antes del commit, o cien pagos confirmados y diez anulaciones posteriores y el ciclo de cierre H6 con las tareas activas. El equipo también deberá ejecutar la integración de entidades, consultas nombradas y plantillas con las versiones fijadas, y registrar resultados con revisión y entorno. Esas actividades son criterios de aceptación de la implementación, sin casillas vacías de resultados que aparenten una prueba realizada. La revisión editorial y visual del documento verifica formato, legibilidad y lista de diagramas que requieren redibujo, sin alterar sus imágenes. Ningún resultado documental representa validación con propietarios participantes, observación de campo o aceptación de un cliente externo. El cierre relacionará requisitos, tareas, casos, resultados y evidencias RA1–RA5; los criterios obligatorios requieren evidencia y las excepciones, justificación y aceptación expresa. Se distinguen entrega documental, implementación y validación del producto, con pendientes aceptados, versiones y huellas identificados.

Referencias

Asamblea Nacional de la República de Nicaragua. (29 de marzo de 2012). *Ley N.º 787, Ley de Protección de Datos Personales*. La Gaceta, Diario Oficial, 61. Ejemplar original, p. 2404. https://sajurin.enriquebolanos.org/docs/G-2012-03-29.pdf

AYR Tech (Pty) Ltd. (s. f.-a). *API authentication notes*. https://www.exchangerate-api.com/docs/authentication

AYR Tech (Pty) Ltd. (s. f.-b). *Exchange rate API: Plans & pricing*. https://www.exchangerate-api.com/

AYR Tech (Pty) Ltd. (s. f.-c). *Standard requests*. https://www.exchangerate-api.com/docs/standard-requests

AYR Tech (Pty) Ltd. (s. f.-d). *Supported currency codes and countries*. https://www.exchangerate-api.com/docs/supported-currencies

AYR Tech (Pty) Ltd. (s. f.-e). *Terms of service*. https://www.exchangerate-api.com/terms

Balzarini, J., y Boyd, M. L. (2021). Working with them: Small-scale landlord strategies for avoiding evictions. Housing Policy Debate, 31(3–5), 425–445. https://doi.org/10.1080/10511482.2020.1800779

Banco Central de Nicaragua. (20 de octubre de 2025). *BCN informa que la tasa de deslizamiento del tipo de cambio del córdoba respecto al dólar de los Estados Unidos de América continuará en cero por ciento (0%) anual en el año 2026*. https://www.bcn.gob.ni/divulgacion-prensa/bcn-informa-que-la-tasa-de-deslizamiento-del-tipo-de-cambio-del-cordoba-respecto

Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A. (2026a). Brief de diseño de interfaz y experiencia de usuario para Latch [Documento académico no publicado]. Universidad Americana (UAM).

Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A. (2026b). *Documento de flujo de la aplicación de Latch* [Documento académico no publicado]. Universidad Americana (UAM).

Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A. (2026c). *Documento de requisitos del producto de Latch* [Documento académico no publicado]. Universidad Americana (UAM).

Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A. (2026d). *Documento de requisitos técnicos de Latch* [Documento académico no publicado]. Universidad Americana (UAM).

Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A. (2026f). Plan de implementación de Latch [Documento académico no publicado]. Universidad Americana (UAM).

Hibernate. (s. f.-a). *7.4 series*. https://hibernate.org/orm/releases/7.4/

Hibernate. (s. f.-b). *A guide to Hibernate Query Language*. https://docs.hibernate.org/orm/7.4/querylanguage/html_single/

Hibernate. (s. f.-c). *Hibernate ORM user guide*. https://docs.hibernate.org/orm/7.4/userguide/html_single/

Jaspersoft. (2026). *JasperReports 7.0.8 [Notas de versión]*. https://github.com/Jaspersoft/jasperreports/releases/tag/7.0.8

Jaspersoft. (s. f.). *pom.xml [Código fuente, etiqueta 7.0.8]*. https://github.com/Jaspersoft/jasperreports/blob/7.0.8/pom.xml

Legion of the Bouncy Castle. (s. f.). *Bouncy Castle for Java documentation*. https://www.bouncycastle.org/documentation/documentation-java/

Lemma, S., Janson, A., Persson, L.-Å., Wickremasinghe, D. y Källestål, C. (2020). Improving quality and use of routine health information system data in low- and middle-income countries: A scoping review. *PLOS ONE, 15*(10), Artículo e0239683. https://doi.org/10.1371/journal.pone.0239683

Open Exchange Rates. (s. f.-a). *API introduction*. https://docs.openexchangerates.org/reference/api-introduction

Open Exchange Rates. (s. f.-b). *Sign up: The forever free plan*. https://openexchangerates.org/signup/free

OWASP Foundation. (s. f.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

PostgreSQL Global Development Group. (s. f.-a). *Collation support (PostgreSQL 18)*. https://www.postgresql.org/docs/18/collation.html

PostgreSQL Global Development Group. (s. f.-b). *Constraints (PostgreSQL 18)*. https://www.postgresql.org/docs/18/ddl-constraints.html

PostgreSQL Global Development Group. (s. f.-c). *Continuous archiving and point-in-time recovery (PITR) (PostgreSQL 18)*. https://www.postgresql.org/docs/18/continuous-archiving.html

PostgreSQL Global Development Group. (s. f.-d). *CREATE FUNCTION (PostgreSQL 18)*. https://www.postgresql.org/docs/18/sql-createfunction.html

PostgreSQL Global Development Group. (s. f.-e). *CREATE INDEX (PostgreSQL 18)*. https://www.postgresql.org/docs/18/sql-createindex.html

PostgreSQL Global Development Group. (s. f.-f). *CREATE PROCEDURE (PostgreSQL 18)*. https://www.postgresql.org/docs/18/sql-createprocedure.html

PostgreSQL Global Development Group. (s. f.-g). *CREATE ROLE (PostgreSQL 18)*. https://www.postgresql.org/docs/18/sql-createrole.html

PostgreSQL Global Development Group. (s. f.-h). *Date/time types (PostgreSQL 18)*. https://www.postgresql.org/docs/18/datatype-datetime.html

PostgreSQL Global Development Group. (s. f.-i). *Explicit locking (PostgreSQL 18)*. https://www.postgresql.org/docs/18/explicit-locking.html

PostgreSQL Global Development Group. (s. f.-j). *GRANT (PostgreSQL 18)*. https://www.postgresql.org/docs/18/sql-grant.html

PostgreSQL Global Development Group. (s. f.-k). *Numeric types (PostgreSQL 18)*. https://www.postgresql.org/docs/18/datatype-numeric.html

PostgreSQL Global Development Group. (s. f.-l). *pg_dump (PostgreSQL 18)*. https://www.postgresql.org/docs/18/app-pgdump.html

PostgreSQL Global Development Group. (s. f.-m). *pg_restore (PostgreSQL 18)*. https://www.postgresql.org/docs/18/app-pgrestore.html

PostgreSQL Global Development Group. (s. f.-n). *Routine vacuuming (PostgreSQL 18)*. https://www.postgresql.org/docs/18/routine-vacuuming.html

PostgreSQL Global Development Group. (s. f.-o). *Secure TCP/IP connections with SSL (PostgreSQL 18)*. https://www.postgresql.org/docs/18/ssl-tcp.html

PostgreSQL Global Development Group. (s. f.-p). *Sequence manipulation functions (PostgreSQL 18)*. https://www.postgresql.org/docs/18/functions-sequence.html

PostgreSQL Global Development Group. (s. f.-q). *The pg_hba.conf file (PostgreSQL 18)*. https://www.postgresql.org/docs/18/auth-pg-hba-conf.html

PostgreSQL Global Development Group. (s. f.-r). *Transaction isolation (PostgreSQL 18)*. https://www.postgresql.org/docs/18/transaction-iso.html

PostgreSQL Global Development Group. (s. f.-s). *Using EXPLAIN (PostgreSQL 18)*. https://www.postgresql.org/docs/18/using-explain.html

PostgreSQL Global Development Group. (s. f.-t). *Versioning policy*. https://www.postgresql.org/support/versioning/

PostgreSQL JDBC Driver. (s. f.-a). *Download*. https://jdbc.postgresql.org/download/

PostgreSQL JDBC Driver. (s. f.-b). *Initializing the driver*. https://jdbc.postgresql.org/documentation/use/

PostgreSQL JDBC Driver. (s. f.-c). *Using SSL*. https://jdbc.postgresql.org/documentation/ssl/

Apéndice A

Script DDL Completo

El script V001 crea el esquema latch con sus tablas, claves y restricciones para PostgreSQL 18.6. La base debe existir con codificación UTF8 y locale C antes de ejecutar esta migración inicial. V004 completa la propiedad y los privilegios de las cuentas técnicas y se aplica una vez durante el aprovisionamiento del clúster de Latch. Cada bloque conserva transacción propia para revertir una carga incompleta. Los nombres del diccionario corresponden a estos identificadores físicos, y la instalación no debe repetir V001 o V003 sobre una base que ya haya confirmado pagos o asignado números de recibo.

V001 Modelo Físico

```
-- V001: PostgreSQL 18.6; base UTF8 con locale C.

BEGIN;

CREATE SCHEMA latch;

SET search_path TO latch,pg_catalog;

SET TIME ZONE 'America/Managua';

CREATE TABLE rol (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  nombre VARCHAR(16) NOT NULL,
  CONSTRAINT ck_rol_nombre CHECK (nombre IN ('Propietario','Asistente')),
  CONSTRAINT pk_rol PRIMARY KEY(id),
  CONSTRAINT uq_rol_nombre UNIQUE(nombre)
);

CREATE TABLE usuario (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  rol_id UUID NOT NULL,
  nombre_usuario VARCHAR(40) NOT NULL,
  CONSTRAINT ck_usuario_nombre_usuario CHECK (nombre_usuario ~ '^[A-Za-z0-
9_.]{3,40}$'),
  nombre_visible VARCHAR(120) NOT NULL,
  hash_clave TEXT NOT NULL,
  CONSTRAINT ck_usuario_hash_clave CHECK (hash_clave LIKE '$argon2id$%'),
  cambio_inicial BOOLEAN NOT NULL DEFAULT TRUE,
  intentos_fallidos INTEGER NOT NULL DEFAULT 0,
  CONSTRAINT ck_usuario_intentos_fallidos CHECK (intentos_fallidos >= 0),
  bloqueado_hasta TIMESTAMPTZ,
  activo BOOLEAN NOT NULL DEFAULT TRUE,
```

```
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_usuario_version CHECK (version >= 0),
  CONSTRAINT pk_usuario PRIMARY KEY(id)
);

CREATE TABLE propiedad (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  nombre VARCHAR(120) NOT NULL,
  direccion VARCHAR(300) NOT NULL,
  activa BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_propiedad_version CHECK (version >= 0),
  CONSTRAINT pk_propiedad PRIMARY KEY(id)
);

CREATE TABLE unidad (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  propiedad_id UUID NOT NULL,
  codigo VARCHAR(30) NOT NULL,
  tipo VARCHAR(24) NOT NULL DEFAULT 'Cuarto',
  CONSTRAINT ck_unidad_tipo CHECK (tipo IN
('Casa','Apartamento','Cuarto','Local comercial')),
  estado VARCHAR(24) NOT NULL DEFAULT 'Disponible',
  CONSTRAINT ck_unidad_estado CHECK (estado IN ('Disponible','En
mantenimiento','Inactiva')),
  canon_referencia NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_unidad_canon_referencia CHECK (canon_referencia > 0),
  moneda VARCHAR(3) NOT NULL,
  CONSTRAINT ck_unidad_moneda CHECK (moneda IN ('NIO','USD')),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_unidad_version CHECK (version >= 0),
  CONSTRAINT pk_unidad PRIMARY KEY(id),
  CONSTRAINT uq_unidad_codigo UNIQUE(propiedad_id,codigo)
);

CREATE TABLE inquilino (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  nombre VARCHAR(160) NOT NULL,
  identificacion VARCHAR(40),
  telefono VARCHAR(30),
  correo VARCHAR(254),
  contacto_emergencia VARCHAR(200),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_inquilino_version CHECK (version >= 0),
  CONSTRAINT pk_inquilino PRIMARY KEY(id)
);

CREATE TABLE contrato (
```

```
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  unidad_id UUID NOT NULL,
  inquilino_id UUID NOT NULL,
  predecesor_id UUID,
  inicio DATE NOT NULL,
  fin DATE NOT NULL,
  terminado_el DATE,
  canon NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_contrato_canon CHECK (canon > 0),
  moneda VARCHAR(3) NOT NULL,
  CONSTRAINT ck_contrato_moneda CHECK (moneda IN ('NIO','USD')),
  dia_pago SMALLINT NOT NULL,
  CONSTRAINT ck_contrato_dia_pago CHECK (dia_pago BETWEEN 1 AND 31),
  deposito_pactado NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_contrato_deposito_pactado CHECK (deposito_pactado >= 0),
  estado VARCHAR(24) NOT NULL DEFAULT 'Borrador',
  CONSTRAINT ck_contrato_estado CHECK (estado IN ('Borrador','Vigente','Por
vencer','Vencido','Terminado','Cancelado')),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_contrato_version CHECK (version >= 0),
  CONSTRAINT pk_contrato PRIMARY KEY(id),
  CONSTRAINT ck_contrato_duracion CHECK (fin >= (inicio + INTERVAL '1
month')::date),
  CONSTRAINT ck_contrato_cierre CHECK ((estado = 'Terminado') = (terminado_el
IS NOT NULL))
);

CREATE TABLE cargo (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  contrato_id UUID NOT NULL,
  cargo_origen_id UUID,
  tipo VARCHAR(24) NOT NULL DEFAULT 'Renta',
  CONSTRAINT ck_cargo_tipo CHECK (tipo IN
('Renta','Mora','Reparación','Servicio','Otro')),
  periodo DATE NOT NULL,
  CONSTRAINT ck_cargo_periodo CHECK (EXTRACT(DAY FROM periodo)=1),
  fecha_emision DATE NOT NULL,
  vence_el DATE NOT NULL,
  importe NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_cargo_importe CHECK (importe > 0),
  concepto VARCHAR(300) NOT NULL,
  anulado_en TIMESTAMPTZ,
  anulado_por UUID,
  motivo_anulacion VARCHAR(500),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_cargo_version CHECK (version >= 0),
  CONSTRAINT pk_cargo PRIMARY KEY(id),
  CONSTRAINT ck_cargo_anulacion CHECK ((anulado_en IS NULL AND anulado_por IS
NULL AND motivo_anulacion IS NULL) OR (anulado_en IS NOT NULL AND anulado_por
IS NOT NULL AND length(trim(motivo_anulacion))>0)),
  CONSTRAINT ck_cargo_mora CHECK (tipo <> 'Mora' OR cargo_origen_id IS NOT
NULL)
);
```

```
CREATE TABLE tipo_cambio (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  proveedor VARCHAR(80) NOT NULL,
  base VARCHAR(3) NOT NULL DEFAULT 'USD',
  CONSTRAINT ck_tipo_cambio_base CHECK (base='USD'),
  destino VARCHAR(3) NOT NULL DEFAULT 'NIO',
  CONSTRAINT ck_tipo_cambio_destino CHECK (destino='NIO'),
  valor NUMERIC(19,8) NOT NULL,
  CONSTRAINT ck_tipo_cambio_valor CHECK (valor > 0),
  actualizado_en TIMESTAMPTZ NOT NULL,
  proximo_en TIMESTAMPTZ NOT NULL,
  consultado_en TIMESTAMPTZ NOT NULL,
  huella_respuesta VARCHAR(64) NOT NULL,
  CONSTRAINT ck_tipo_cambio_huella_respuesta CHECK (huella_respuesta ~ '^[0-
9a-f]{64}$'),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_tipo_cambio PRIMARY KEY(id),
  CONSTRAINT ck_tipo_cambio_instantes CHECK (proximo_en > actualizado_en)
);

CREATE TABLE autorizacion_tasa (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  fecha_efectiva DATE NOT NULL,
  valor NUMERIC(19,8) NOT NULL,
  CONSTRAINT ck_autorizacion_tasa_valor CHECK (valor > 0),
  fuente VARCHAR(300) NOT NULL,
  fundamento VARCHAR(500) NOT NULL,
  referencia_id UUID,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_autorizacion_tasa PRIMARY KEY(id),
  CONSTRAINT uq_autorizacion_fecha UNIQUE(fecha_efectiva),
  CONSTRAINT ck_autorizacion_actor CHECK (creado_por IS NOT NULL)
);

CREATE TABLE pago (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  contrato_id UUID NOT NULL,
  solicitud_id UUID NOT NULL,
  solicitud_hash VARCHAR(64) NOT NULL,
  CONSTRAINT ck_pago_solicitud_hash CHECK (solicitud_hash ~ '^[0-9a-f]
{64}$'),
  fecha_pago DATE NOT NULL,
  moneda_original VARCHAR(3) NOT NULL,
  CONSTRAINT ck_pago_moneda_original CHECK (moneda_original IN
('NIO','USD')),
  monto_original NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_pago_monto_original CHECK (monto_original > 0),
  monto_contrato NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_pago_monto_contrato CHECK (monto_contrato > 0),
  tasa_usada NUMERIC(19,8),
  CONSTRAINT ck_pago_tasa_usada CHECK (tasa_usada > 0),
  autorizacion_id UUID,
  metodo VARCHAR(24) NOT NULL DEFAULT 'Efectivo',
  CONSTRAINT ck_pago_metodo CHECK (metodo IN
```

```
('Efectivo','Transferencia','Depósito bancario','Otro')),
  estado VARCHAR(24) NOT NULL DEFAULT 'Registrado',
  CONSTRAINT ck_pago_estado CHECK (estado IN ('Registrado','Anulado')),
  anulado_en TIMESTAMPTZ,
  anulado_por UUID,
  motivo_anulacion VARCHAR(500),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_pago PRIMARY KEY(id),
  CONSTRAINT uq_pago_solicitud UNIQUE(solicitud_id),
  CONSTRAINT ck_pago_tasa CHECK ((tasa_usada IS NULL) = (autorizacion_id IS
NULL)),
  CONSTRAINT ck_pago_anulacion CHECK ((estado='Registrado' AND anulado_en IS
NULL AND anulado_por IS NULL AND motivo_anulacion IS NULL) OR
(estado='Anulado' AND anulado_en IS NOT NULL AND anulado_por IS NOT NULL AND
length(trim(motivo_anulacion))>0))
);

CREATE TABLE deposito_garantia (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  contrato_id UUID NOT NULL,
  importe_inicial NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_deposito_garantia_importe_inicial CHECK (importe_inicial >=
0),
  fecha_recepcion DATE NOT NULL,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_deposito_garantia PRIMARY KEY(id),
  CONSTRAINT uq_deposito_contrato UNIQUE(contrato_id)
);

CREATE TABLE movimiento_deposito (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  deposito_id UUID NOT NULL,
  cargo_id UUID,
  liquidacion_id UUID NOT NULL,
  liquidacion_hash VARCHAR(64) NOT NULL,
  orden SMALLINT NOT NULL,
  CONSTRAINT ck_movimiento_liquidacion_hash CHECK (liquidacion_hash ~ '^[0-
9a-f]{64}$'),
  tipo VARCHAR(24) NOT NULL DEFAULT 'Aplicacion',
  CONSTRAINT ck_movimiento_deposito_tipo CHECK (tipo IN
('Aplicacion','Devolucion')),
  importe NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_movimiento_deposito_importe CHECK (importe > 0),
  fecha_movimiento DATE NOT NULL,
  metodo VARCHAR(24),
  CONSTRAINT ck_movimiento_deposito_metodo CHECK (metodo IN
('Efectivo','Transferencia','Depósito bancario','Otro')),
  concepto VARCHAR(300) NOT NULL,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_movimiento_deposito PRIMARY KEY(id),
  CONSTRAINT uq_movimiento_liquidacion_orden UNIQUE(liquidacion_id,orden),
  CONSTRAINT ck_movimiento_destino CHECK ((tipo='Aplicacion' AND cargo_id IS
NOT NULL AND metodo IS NULL) OR (tipo='Devolucion' AND cargo_id IS NULL AND
metodo IS NOT NULL))
```

```
);

CREATE TABLE aplicacion_pago (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  pago_id UUID,
  movimiento_id UUID,
  cargo_id UUID NOT NULL,
  importe NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_aplicacion_pago_importe CHECK (importe > 0),
  origen VARCHAR(24) NOT NULL DEFAULT 'Inicial',
  CONSTRAINT ck_aplicacion_pago_origen CHECK (origen IN
('Inicial','Credito','Garantia')),
  fecha_aplicacion DATE NOT NULL,
  revertida_en TIMESTAMPTZ,
  revertida_por UUID,
  motivo_reversion VARCHAR(500),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_aplicacion_pago PRIMARY KEY(id),
  CONSTRAINT ck_aplicacion_fuente CHECK ((pago_id IS NOT NULL AND
movimiento_id IS NULL AND origen IN ('Inicial','Credito')) OR (pago_id IS
NULL AND movimiento_id IS NOT NULL AND origen='Garantia')),
  CONSTRAINT uq_aplicacion_movimiento UNIQUE(movimiento_id),
  CONSTRAINT ck_aplicacion_reversion CHECK ((revertida_en IS NULL AND
revertida_por IS NULL AND motivo_reversion IS NULL) OR (revertida_en IS NOT
NULL AND revertida_por IS NOT NULL AND length(trim(motivo_reversion))>0))
);

CREATE TABLE consecutivo_recibo (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  siguiente_numero BIGINT NOT NULL DEFAULT 1,
  CONSTRAINT ck_consecutivo_recibo_siguiente_numero CHECK (siguiente_numero >
0),
  CONSTRAINT pk_consecutivo_recibo PRIMARY KEY(id),
  CONSTRAINT ck_consecutivo_unico CHECK (id='00000000-0000-0000-0000-
000000000001'::uuid)
);

CREATE TABLE recibo (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  pago_id UUID NOT NULL,
  numero BIGINT NOT NULL,
  CONSTRAINT ck_recibo_numero CHECK (numero > 0),
  instantanea JSONB NOT NULL,
  CONSTRAINT ck_recibo_instantanea CHECK
(jsonb_typeof(instantanea)='object'),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_recibo PRIMARY KEY(id),
  CONSTRAINT uq_recibo_pago UNIQUE(pago_id),
  CONSTRAINT uq_recibo_numero UNIQUE(numero)
);

CREATE TABLE solicitud_mantenimiento (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  unidad_id UUID NOT NULL,
  descripcion VARCHAR(1000) NOT NULL,
```

```
  prioridad VARCHAR(24) NOT NULL DEFAULT 'Media',
  CONSTRAINT ck_solicitud_mantenimiento_prioridad CHECK (prioridad IN
('Baja','Media','Alta')),
  estado VARCHAR(24) NOT NULL DEFAULT 'Abierta',
  CONSTRAINT ck_solicitud_mantenimiento_estado CHECK (estado IN
('Abierta','En proceso','Resuelta','Cancelada')),
  responsable VARCHAR(120),
  abierta_el DATE NOT NULL,
  cerrada_el DATE,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_solicitud_mantenimiento_version CHECK (version >= 0),
  CONSTRAINT pk_solicitud_mantenimiento PRIMARY KEY(id),
  CONSTRAINT ck_mantenimiento_cierre CHECK ((estado IN
('Resuelta','Cancelada')) = (cerrada_el IS NOT NULL))
);

CREATE TABLE gasto_propiedad (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  propiedad_id UUID NOT NULL,
  solicitud_id UUID,
  fecha_gasto DATE NOT NULL,
  importe NUMERIC(19,2) NOT NULL,
  CONSTRAINT ck_gasto_propiedad_importe CHECK (importe > 0),
  moneda VARCHAR(3) NOT NULL,
  CONSTRAINT ck_gasto_propiedad_moneda CHECK (moneda IN ('NIO','USD')),
  concepto VARCHAR(300) NOT NULL,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_gasto_propiedad PRIMARY KEY(id)
);

CREATE TABLE parametro_sistema (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  clave VARCHAR(60) NOT NULL,
  valor JSONB NOT NULL,
  vigente_desde TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  motivo VARCHAR(500) NOT NULL,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_parametro_sistema PRIMARY KEY(id),
  CONSTRAINT uq_parametro_clave UNIQUE(clave,vigente_desde)
);

CREATE TABLE bitacora (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  correlacion_id UUID NOT NULL,
  modulo VARCHAR(3) NOT NULL,
  CONSTRAINT ck_bitacora_modulo CHECK (modulo ~ '^M(0[1-9]|1[0-2])$'),
  accion VARCHAR(80) NOT NULL,
  entidad VARCHAR(60) NOT NULL,
  entidad_id UUID,
  detalle JSONB NOT NULL DEFAULT '{}'::jsonb,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_bitacora PRIMARY KEY(id)
```

```
);

CREATE TABLE documento_adjunto (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  propiedad_id UUID,
  unidad_id UUID,
  inquilino_id UUID,
  contrato_id UUID,
  parametro_id UUID,
  nombre_original VARCHAR(180) NOT NULL,
  ruta_interna VARCHAR(80) NOT NULL,
  CONSTRAINT ck_documento_adjunto_ruta_interna CHECK (ruta_interna ~ '^[0-9a-
f-]{36}[.](pdf|png|jpg)$'),
  mime VARCHAR(30) NOT NULL,
  CONSTRAINT ck_documento_adjunto_mime CHECK (mime IN
('application/pdf','image/png','image/jpeg')),
  tamano_bytes BIGINT NOT NULL,
  CONSTRAINT ck_documento_adjunto_tamano_bytes CHECK (tamano_bytes BETWEEN 1
AND 5242880),
  sha256 VARCHAR(64) NOT NULL,
  CONSTRAINT ck_documento_adjunto_sha256 CHECK (sha256 ~ '^[0-9a-f]{64}$'),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  version BIGINT NOT NULL DEFAULT 0,
  CONSTRAINT ck_documento_adjunto_version CHECK (version >= 0),
  CONSTRAINT pk_documento_adjunto PRIMARY KEY(id),
  CONSTRAINT ck_documento_dueno CHECK
(num_nonnulls(propiedad_id,unidad_id,inquilino_id,contrato_id,parametro_id)=1
),
  CONSTRAINT uq_documento_ruta UNIQUE(ruta_interna)
);

CREATE TABLE puerta_mantenimiento (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  descripcion VARCHAR(80) NOT NULL,
  CONSTRAINT pk_puerta_mantenimiento PRIMARY KEY(id),
  CONSTRAINT ck_puerta_unica CHECK (id='00000000-0000-0000-0000-
000000000002'::uuid)
);

CREATE TABLE control_fx (
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  instancia_id UUID,
  concesion_hasta TIMESTAMPTZ,
  proximo_intento TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  mes DATE NOT NULL,
  CONSTRAINT ck_control_fx_mes CHECK (EXTRACT(DAY FROM mes)=1),
  intentos_mes INTEGER NOT NULL DEFAULT 0,
  CONSTRAINT ck_control_fx_intentos_mes CHECK (intentos_mes BETWEEN 0 AND
100),
  CONSTRAINT pk_control_fx PRIMARY KEY(id),
  CONSTRAINT ck_control_fx_unico CHECK (id='00000000-0000-0000-0000-
000000000003'::uuid)
);

CREATE TABLE esquema_migracion (
```

```
  id UUID NOT NULL DEFAULT gen_random_uuid(),
  nombre VARCHAR(80) NOT NULL,
  sha256 VARCHAR(64) NOT NULL,
  CONSTRAINT ck_esquema_migracion_sha256 CHECK (sha256 ~ '^[0-9a-f]{64}$'),
  creado_en TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  creado_por UUID,
  CONSTRAINT pk_esquema_migracion PRIMARY KEY(id),
  CONSTRAINT uq_migracion_nombre UNIQUE(nombre)
);

ALTER TABLE usuario ADD CONSTRAINT fk_usuario_rol_id FOREIGN KEY(rol_id)
REFERENCES rol(id) ON DELETE RESTRICT;

ALTER TABLE usuario ADD CONSTRAINT fk_usuario_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE propiedad ADD CONSTRAINT fk_propiedad_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE unidad ADD CONSTRAINT fk_unidad_propiedad_id FOREIGN
KEY(propiedad_id) REFERENCES propiedad(id) ON DELETE RESTRICT;

ALTER TABLE unidad ADD CONSTRAINT fk_unidad_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE inquilino ADD CONSTRAINT fk_inquilino_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE contrato ADD CONSTRAINT fk_contrato_unidad_id FOREIGN
KEY(unidad_id) REFERENCES unidad(id) ON DELETE RESTRICT;

ALTER TABLE contrato ADD CONSTRAINT fk_contrato_inquilino_id FOREIGN
KEY(inquilino_id) REFERENCES inquilino(id) ON DELETE RESTRICT;

ALTER TABLE contrato ADD CONSTRAINT fk_contrato_predecesor_id FOREIGN
KEY(predecesor_id) REFERENCES contrato(id) ON DELETE RESTRICT;

ALTER TABLE contrato ADD CONSTRAINT fk_contrato_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE cargo ADD CONSTRAINT fk_cargo_contrato_id FOREIGN
KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT;

ALTER TABLE cargo ADD CONSTRAINT fk_cargo_cargo_origen_id FOREIGN
KEY(cargo_origen_id) REFERENCES cargo(id) ON DELETE RESTRICT;

ALTER TABLE cargo ADD CONSTRAINT fk_cargo_anulado_por FOREIGN
KEY(anulado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE cargo ADD CONSTRAINT fk_cargo_creado_por FOREIGN KEY(creado_por)
REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE tipo_cambio ADD CONSTRAINT fk_tipo_cambio_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE autorizacion_tasa ADD CONSTRAINT
fk_autorizacion_tasa_referencia_id FOREIGN KEY(referencia_id) REFERENCES
```

```
tipo_cambio(id) ON DELETE RESTRICT;

ALTER TABLE autorizacion_tasa ADD CONSTRAINT fk_autorizacion_tasa_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE pago ADD CONSTRAINT fk_pago_contrato_id FOREIGN KEY(contrato_id)
REFERENCES contrato(id) ON DELETE RESTRICT;

ALTER TABLE pago ADD CONSTRAINT fk_pago_autorizacion_id FOREIGN
KEY(autorizacion_id) REFERENCES autorizacion_tasa(id) ON DELETE RESTRICT;

ALTER TABLE pago ADD CONSTRAINT fk_pago_anulado_por FOREIGN KEY(anulado_por)
REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE pago ADD CONSTRAINT fk_pago_creado_por FOREIGN KEY(creado_por)
REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE deposito_garantia ADD CONSTRAINT fk_deposito_garantia_contrato_id
FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT;

ALTER TABLE deposito_garantia ADD CONSTRAINT fk_deposito_garantia_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE movimiento_deposito ADD CONSTRAINT
fk_movimiento_deposito_deposito_id FOREIGN KEY(deposito_id) REFERENCES
deposito_garantia(id) ON DELETE RESTRICT;

ALTER TABLE movimiento_deposito ADD CONSTRAINT
fk_movimiento_deposito_cargo_id FOREIGN KEY(cargo_id) REFERENCES cargo(id) ON
DELETE RESTRICT;

ALTER TABLE movimiento_deposito ADD CONSTRAINT
fk_movimiento_deposito_creado_por FOREIGN KEY(creado_por) REFERENCES
usuario(id) ON DELETE RESTRICT;

ALTER TABLE aplicacion_pago ADD CONSTRAINT fk_aplicacion_pago_pago_id FOREIGN
KEY(pago_id) REFERENCES pago(id) ON DELETE RESTRICT;

ALTER TABLE aplicacion_pago ADD CONSTRAINT fk_aplicacion_pago_movimiento_id
FOREIGN KEY(movimiento_id) REFERENCES movimiento_deposito(id) ON DELETE
RESTRICT;

ALTER TABLE aplicacion_pago ADD CONSTRAINT fk_aplicacion_pago_cargo_id
FOREIGN KEY(cargo_id) REFERENCES cargo(id) ON DELETE RESTRICT;

ALTER TABLE aplicacion_pago ADD CONSTRAINT fk_aplicacion_pago_revertida_por
FOREIGN KEY(revertida_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE aplicacion_pago ADD CONSTRAINT fk_aplicacion_pago_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE recibo ADD CONSTRAINT fk_recibo_pago_id FOREIGN KEY(pago_id)
REFERENCES pago(id) ON DELETE RESTRICT;

ALTER TABLE recibo ADD CONSTRAINT fk_recibo_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;
```

```
ALTER TABLE solicitud_mantenimiento ADD CONSTRAINT
fk_solicitud_mantenimiento_unidad_id FOREIGN KEY(unidad_id) REFERENCES
unidad(id) ON DELETE RESTRICT;

ALTER TABLE solicitud_mantenimiento ADD CONSTRAINT
fk_solicitud_mantenimiento_creado_por FOREIGN KEY(creado_por) REFERENCES
usuario(id) ON DELETE RESTRICT;

ALTER TABLE gasto_propiedad ADD CONSTRAINT fk_gasto_propiedad_propiedad_id
FOREIGN KEY(propiedad_id) REFERENCES propiedad(id) ON DELETE RESTRICT;

ALTER TABLE gasto_propiedad ADD CONSTRAINT fk_gasto_propiedad_solicitud_id
FOREIGN KEY(solicitud_id) REFERENCES solicitud_mantenimiento(id) ON DELETE
RESTRICT;

ALTER TABLE gasto_propiedad ADD CONSTRAINT fk_gasto_propiedad_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE parametro_sistema ADD CONSTRAINT fk_parametro_sistema_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE bitacora ADD CONSTRAINT fk_bitacora_creado_por FOREIGN
KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT
fk_documento_adjunto_propiedad_id FOREIGN KEY(propiedad_id) REFERENCES
propiedad(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT fk_documento_adjunto_unidad_id
FOREIGN KEY(unidad_id) REFERENCES unidad(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT
fk_documento_adjunto_inquilino_id FOREIGN KEY(inquilino_id) REFERENCES
inquilino(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT fk_documento_adjunto_contrato_id
FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT
fk_documento_adjunto_parametro_id FOREIGN KEY(parametro_id) REFERENCES
parametro_sistema(id) ON DELETE RESTRICT;

ALTER TABLE documento_adjunto ADD CONSTRAINT fk_documento_adjunto_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

ALTER TABLE esquema_migracion ADD CONSTRAINT fk_esquema_migracion_creado_por
FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT;

CREATE UNIQUE INDEX uq_usuario_nombre ON usuario (lower(nombre_usuario));

CREATE UNIQUE INDEX uq_contrato_activo ON contrato (unidad_id) WHERE estado
IN ('Vigente','Por vencer','Vencido');

CREATE UNIQUE INDEX uq_cargo_renta ON cargo (contrato_id,periodo) WHERE
tipo='Renta';

CREATE UNIQUE INDEX uq_cargo_mora ON cargo (cargo_origen_id) WHERE
```

```
tipo='Mora';

CREATE INDEX ix_unidad_propiedad ON unidad (propiedad_id);

CREATE INDEX ix_contrato_inquilino ON contrato (inquilino_id);

CREATE INDEX ix_contrato_fin ON contrato (fin) WHERE estado IN
('Vigente','Por vencer','Vencido');

CREATE INDEX ix_contrato_predecesor ON contrato (predecesor_id);

CREATE INDEX ix_cargo_cobranza ON cargo (contrato_id,vence_el,id) WHERE
anulado_en IS NULL;

CREATE INDEX ix_pago_historial ON pago (contrato_id,fecha_pago);

CREATE INDEX ix_pago_autorizacion ON pago (autorizacion_id);

CREATE INDEX ix_aplicacion_cargo ON aplicacion_pago
(cargo_id,fecha_aplicacion);

CREATE INDEX ix_aplicacion_pago ON aplicacion_pago (pago_id) WHERE
revertida_en IS NULL;

CREATE INDEX ix_movimiento_deposito ON movimiento_deposito
(deposito_id,fecha_movimiento);

CREATE INDEX ix_mantenimiento_unidad ON solicitud_mantenimiento
(unidad_id,estado);

CREATE INDEX ix_gasto_propiedad ON gasto_propiedad
(propiedad_id,fecha_gasto);

CREATE INDEX ix_bitacora_fecha ON bitacora (creado_en,id);

CREATE INDEX ix_bitacora_correlacion ON bitacora (correlacion_id);

CREATE INDEX ix_tasa_actualizacion ON tipo_cambio (actualizado_en DESC);

COMMIT;
```

V004 Roles y Privilegios

```
-- Ejecuta una cuenta administrativa del clúster; roles LOGIN sin contraseña
fija.
BEGIN;
DO $$ DECLARE n TEXT; esperado_login BOOLEAN; BEGIN
 FOREACH n IN ARRAY
ARRAY['latch_owner','latch_runtime','latch_migrator','latch_backup'] LOOP
  esperado_login := n <> 'latch_owner';
  IF NOT EXISTS(SELECT 1 FROM pg_roles WHERE rolname=n) THEN
   EXECUTE format('CREATE ROLE %I %s NOSUPERUSER NOCREATEDB NOCREATEROLE
NOREPLICATION',
    n,CASE WHEN esperado_login THEN 'LOGIN' ELSE 'NOLOGIN' END);
  ELSIF EXISTS(SELECT 1 FROM pg_roles WHERE rolname=n AND
   (rolcanlogin<>esperado_login OR rolsuper OR rolcreatedb OR rolcreaterole
OR rolreplication)) THEN
```

```
   RAISE EXCEPTION 'Rol existente incompatible: %',n;
  END IF;
 END LOOP;
END $$;
GRANT latch_owner TO latch_migrator;
REVOKE ALL ON SCHEMA latch FROM PUBLIC;
GRANT USAGE ON SCHEMA latch TO latch_runtime,latch_backup;
GRANT ALL ON SCHEMA latch TO latch_owner;
GRANT SELECT ON ALL TABLES IN SCHEMA latch TO latch_runtime,latch_backup;
GRANT INSERT,UPDATE ON
latch.usuario,latch.propiedad,latch.unidad,latch.inquilino,
 latch.contrato,latch.cargo,latch.solicitud_mantenimiento,
 latch.documento_adjunto,latch.control_fx TO latch_runtime;
REVOKE UPDATE,DELETE ON latch.parametro_sistema FROM latch_runtime;
GRANT INSERT ON latch.parametro_sistema TO latch_runtime;
GRANT INSERT ON latch.pago,latch.recibo,latch.aplicacion_pago,
 latch.deposito_garantia,latch.movimiento_deposito,latch.gasto_propiedad,
 latch.tipo_cambio,latch.autorizacion_tasa,latch.bitacora TO latch_runtime;
GRANT UPDATE(estado,anulado_en,anulado_por,motivo_anulacion) ON latch.pago TO
latch_runtime;
GRANT UPDATE(revertida_en,revertida_por,motivo_reversion) ON
latch.aplicacion_pago TO latch_runtime;
GRANT UPDATE(siguiente_numero) ON latch.consecutivo_recibo TO latch_runtime;
-- FOR SHARE requiere privilegio UPDATE en al menos una columna, sin cambiar
datos.
GRANT UPDATE(descripcion) ON latch.puerta_mantenimiento TO
latch_runtime,latch_backup;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA latch TO latch_runtime;
GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA latch TO latch_runtime;
DO $$ DECLARE x RECORD; BEGIN
 FOR x IN SELECT tablename FROM pg_tables WHERE schemaname='latch' LOOP
  EXECUTE format('ALTER TABLE latch.%I OWNER TO latch_owner',x.tablename);
 END LOOP;
 FOR x IN SELECT viewname FROM pg_views WHERE schemaname='latch' LOOP
  EXECUTE format('ALTER VIEW latch.%I OWNER TO latch_owner',x.viewname);
 END LOOP;
 FOR x IN SELECT p.oid::regprocedure AS firma,p.prokind FROM pg_proc p
 JOIN pg_namespace n ON n.oid=p.pronamespace WHERE n.nspname='latch' LOOP
  EXECUTE format('ALTER %s %s OWNER TO latch_owner',
   CASE WHEN x.prokind='p' THEN 'PROCEDURE' ELSE 'FUNCTION' END,x.firma);
 END LOOP;
END $$;
ALTER SCHEMA latch OWNER TO latch_owner;
DO $$ BEGIN
 EXECUTE format('REVOKE CONNECT ON DATABASE %I FROM
PUBLIC',current_database());
 EXECUTE format('GRANT CONNECT ON DATABASE %I TO
latch_runtime,latch_migrator,latch_backup',current_database());
END $$;
ALTER DEFAULT PRIVILEGES FOR ROLE latch_owner IN SCHEMA latch
 REVOKE EXECUTE ON FUNCTIONS FROM PUBLIC;
COMMIT;
-- El DBA asigna contraseñas con \password en psql, fuera del historial SQL.
```

Apéndice B

Script de Vistas y Procedimientos

V002 define las diez vistas y las rutinas utilizadas para lectura financiera, control de la puerta, contador y auditoría. Las funciones se invocan con SELECT y el procedimiento de integridad con CALL, dentro de la transacción coordinada por el servicio. fn_reservar_numero no constituye una operación independiente de emisión y nunca debe confirmarse antes del pago y su recibo. El helper de redondeo reproduce la política HALF_EVEN y el corte usa el límite exclusivo del día local. La base no contiene disparadores; el contrato de servicios exige flush y verificación de integridad antes de cada commit financiero.

```
-- Funciones auxiliares invocadas desde JDBC sobre la conexión de Hibernate.
BEGIN;
SET search_path TO latch,pg_catalog;
SET TIME ZONE 'America/Managua';

CREATE FUNCTION fn_fin_corte(p_fecha DATE) RETURNS TIMESTAMPTZ
LANGUAGE SQL IMMUTABLE STRICT AS $$
 SELECT (p_fecha+1)::timestamp AT TIME ZONE 'America/Managua'
$$;

CREATE FUNCTION fn_redondear_par(p_valor NUMERIC,p_escala INTEGER)
RETURNS NUMERIC LANGUAGE plpgsql IMMUTABLE STRICT AS $$
DECLARE factor NUMERIC; absoluto NUMERIC; entero NUMERIC;
BEGIN
 IF p_escala NOT BETWEEN 0 AND 8 THEN
  RAISE EXCEPTION 'Escala fuera de rango' USING ERRCODE='22023';
 END IF;
 factor:=power(10::numeric,p_escala);
 absoluto:=abs(p_valor)*factor; entero:=floor(absoluto);
 IF absoluto-entero>0.5 OR
    (absoluto-entero=0.5 AND mod(entero,2)=1) THEN
  entero:=entero+1;
 END IF;
 RETURN sign(p_valor)*entero/factor;
END $$;

CREATE FUNCTION fn_cargos_corte(p_corte DATE)
RETURNS TABLE(cargo_id UUID,contrato_id UUID,tipo VARCHAR,
 periodo DATE,vence_el DATE,moneda VARCHAR,importe NUMERIC,
 abonado NUMERIC,saldo NUMERIC,estado TEXT)
LANGUAGE SQL STABLE SET search_path=latch,pg_catalog AS $$
 WITH abonos AS (
  SELECT a.cargo_id,sum(a.importe) AS total
  FROM aplicacion_pago a LEFT JOIN pago p ON p.id=a.pago_id
  WHERE a.fecha_aplicacion<=p_corte
```

```
   AND a.creado_en<fn_fin_corte(p_corte)
   AND (a.revertida_en IS NULL OR a.revertida_en>=fn_fin_corte(p_corte))
   AND (p.id IS NULL OR (p.fecha_pago<=p_corte AND
        (p.anulado_en IS NULL OR p.anulado_en>=fn_fin_corte(p_corte))))
  GROUP BY a.cargo_id
 ), datos AS (
 SELECT c.id,c.contrato_id,c.tipo,c.periodo,c.vence_el,k.moneda,
  c.importe,coalesce(a.total,0) AS abonado,
  c.importe-coalesce(a.total,0) AS saldo,
  c.anulado_en IS NOT NULL AND c.anulado_en<fn_fin_corte(p_corte) AS anulado
 FROM cargo c JOIN contrato k ON k.id=c.contrato_id
 LEFT JOIN abonos a ON a.cargo_id=c.id
 WHERE c.fecha_emision<=p_corte AND c.creado_en<fn_fin_corte(p_corte)
 )
 SELECT id,contrato_id,tipo,periodo,vence_el,moneda,importe,abonado,
  CASE WHEN anulado THEN 0 ELSE saldo END,
  CASE WHEN anulado THEN 'Anulado' WHEN saldo=0 THEN 'Pagado'
   WHEN vence_el<p_corte THEN 'Vencido' WHEN abonado>0 THEN 'Parcial'
   ELSE 'Pendiente' END
 FROM datos
$$;

CREATE VIEW v_estado_cuenta AS
 SELECT * FROM fn_cargos_corte((CURRENT_TIMESTAMP AT TIME ZONE
'America/Managua')::date);
CREATE VIEW v_saldo_contrato AS
 SELECT k.id AS contrato_id,k.moneda,coalesce(sum(e.saldo),0) AS deuda
 FROM contrato k LEFT JOIN v_estado_cuenta e ON e.contrato_id=k.id
 GROUP BY k.id,k.moneda;
CREATE VIEW v_cartera_vencida AS
 SELECT e.*,k.inquilino_id,k.unidad_id,
  (CURRENT_TIMESTAMP AT TIME ZONE 'America/Managua')::date-e.vence_el AS
dias_vencidos
 FROM v_estado_cuenta e JOIN contrato k ON k.id=e.contrato_id
 WHERE e.saldo>0 AND e.estado='Vencido';
CREATE VIEW v_ocupacion AS
 SELECT u.id AS unidad_id,u.propiedad_id,u.codigo,u.tipo,
  CASE WHEN k.id IS NOT NULL THEN 'Ocupada' ELSE u.estado END AS estado,
  k.id AS contrato_id
 FROM unidad u LEFT JOIN contrato k ON k.unidad_id=u.id
  AND k.estado IN ('Vigente','Por vencer','Vencido');
CREATE VIEW v_ingresos_mes AS
 SELECT date_trunc('month',p.fecha_pago)::date AS mes,u.propiedad_id,
 p.moneda_original AS moneda,sum(p.monto_original) AS ingresos
 FROM pago p JOIN contrato k ON k.id=p.contrato_id
 JOIN unidad u ON u.id=k.unidad_id WHERE p.estado='Registrado'
 GROUP BY 1,2,3;
CREATE VIEW v_contratos_por_vencer AS
 SELECT k.* FROM contrato k WHERE k.estado IN ('Vigente','Por vencer')
 AND k.fin BETWEEN (CURRENT_TIMESTAMP AT TIME ZONE 'America/Managua')::date
 AND (CURRENT_TIMESTAMP AT TIME ZONE 'America/Managua')::date+
  (SELECT (valor#>>'{}')::integer FROM parametro_sistema
   WHERE clave='dias_por_vencer' AND vigente_desde<=now()
   ORDER BY vigente_desde DESC LIMIT 1);
CREATE VIEW v_credito_pago AS
 SELECT p.id AS pago_id,p.contrato_id,k.moneda,
 p.monto_contrato-coalesce(sum(a.importe) FILTER(WHERE a.revertida_en IS
```

```
NULL),0) AS disponible
 FROM pago p JOIN contrato k ON k.id=p.contrato_id
 LEFT JOIN aplicacion_pago a ON a.pago_id=p.id
 WHERE p.estado='Registrado' GROUP BY p.id,k.moneda;
CREATE VIEW v_depositos AS
 WITH m AS (
 SELECT deposito_id,sum(importe) AS salidas,
 coalesce(sum(importe) FILTER(WHERE tipo='Devolucion'),0) AS devoluciones
 FROM movimiento_deposito GROUP BY deposito_id)
 SELECT d.id AS deposito_id,d.contrato_id,k.moneda,d.importe_inicial,
 d.importe_inicial-coalesce(m.salidas,0) AS remanente,
 CASE WHEN d.importe_inicial-coalesce(m.salidas,0)>0 AND
coalesce(m.devoluciones,0)>0
 THEN 'Devuelto parcial'
 WHEN d.importe_inicial-coalesce(m.salidas,0)>0 OR d.importe_inicial=0 THEN
'Retenido'
 WHEN coalesce(m.devoluciones,0)>0 THEN 'Devuelto' ELSE 'Aplicado' END AS
estado
 FROM deposito_garantia d JOIN contrato k ON k.id=d.contrato_id
 LEFT JOIN m ON m.deposito_id=d.id;
CREATE VIEW v_recibo AS
 SELECT r.id,r.numero,r.pago_id,r.instantanea,r.creado_en,
 p.estado,p.anulado_en,p.anulado_por,p.motivo_anulacion
 FROM recibo r JOIN pago p ON p.id=r.pago_id;
CREATE VIEW v_gastos_mes AS
 SELECT date_trunc('month',fecha_gasto)::date AS mes,propiedad_id,moneda,
 sum(importe) AS gastos FROM gasto_propiedad GROUP BY 1,2,3;

CREATE FUNCTION fn_puerta_escritura() RETURNS VOID
LANGUAGE plpgsql VOLATILE SET search_path=latch,pg_catalog AS $$
BEGIN
 PERFORM id FROM puerta_mantenimiento
 WHERE id='00000000-0000-0000-0000-000000000002' FOR SHARE;
 IF NOT FOUND THEN RAISE EXCEPTION 'Puerta ausente'; END IF;
END $$;

CREATE FUNCTION fn_reservar_numero() RETURNS BIGINT
LANGUAGE plpgsql VOLATILE SET search_path=latch,pg_catalog AS $$
DECLARE n BIGINT;
BEGIN
 SELECT siguiente_numero INTO STRICT n FROM consecutivo_recibo
 WHERE id='00000000-0000-0000-0000-000000000001' FOR UPDATE;
 UPDATE consecutivo_recibo SET siguiente_numero=n+1
 WHERE id='00000000-0000-0000-0000-000000000001';
 RETURN n;
END $$;

CREATE FUNCTION fn_auditar(p_actor UUID,p_correlacion UUID,p_modulo VARCHAR,
 p_accion VARCHAR,p_entidad VARCHAR,p_id UUID,p_detalle JSONB)
RETURNS UUID LANGUAGE plpgsql VOLATILE
SET search_path=latch,pg_catalog AS $$
DECLARE nuevo UUID;
BEGIN
 IF p_actor IS NULL THEN
  IF ((p_modulo='M01' AND p_accion IN ('ACCESO_FALLIDO','ACCESO_BLOQUEADO'))
OR (p_modulo='M05' AND p_accion IN
('MORA_AUTOMATICA','REVISION_VENCIMIENTO','INCIDENTE_H2') AND p_detalle-
```

```
>>'origen'='H2') OR (p_modulo='M09' AND p_accion IN
('REFERENCIA_ACTUALIZADA','REFERENCIA_EN_CUARENTENA','INCIDENTE_H3') AND
p_detalle->>'origen'='H3')) IS NOT TRUE THEN
   RAISE EXCEPTION 'Actor requerido' USING ERRCODE='42501';
  END IF;
 ELSIF NOT EXISTS(SELECT 1 FROM usuario WHERE id=p_actor AND activo) THEN
  RAISE EXCEPTION 'Actor no activo' USING ERRCODE='42501';
 END IF;
 IF p_modulo='M01' AND p_accion IN ('ACCESO_FALLIDO','ACCESO_BLOQUEADO') THEN
  IF (jsonb_typeof(p_detalle->'nombre_hash')='string'
      AND (p_detalle->>'nombre_hash') ~ '^[0-9a-f]{1,63}$') IS NOT TRUE THEN
   RAISE EXCEPTION 'Se requiere SHA-256 truncado del nombre normalizado'
    USING ERRCODE='22023';
  END IF;
  p_detalle:=jsonb_build_object('nombre_hash',p_detalle->>'nombre_hash');
 END IF;
 INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
 VALUES(p_actor,p_correlacion,p_modulo,p_accion,p_entidad,p_id,p_detalle)
 RETURNING id INTO nuevo;
 RETURN nuevo;
END $$;

CREATE PROCEDURE sp_verificar_integridad(p_contrato UUID)
LANGUAGE plpgsql SET search_path=latch,pg_catalog AS $$
BEGIN
 IF NOT EXISTS(SELECT 1 FROM contrato WHERE id=p_contrato) THEN
  RAISE EXCEPTION 'Contrato inexistente' USING ERRCODE='23503';
 END IF;
 IF EXISTS(SELECT 1 FROM v_credito_pago
           WHERE contrato_id=p_contrato AND disponible<0)
 OR EXISTS(SELECT 1 FROM v_depositos
           WHERE contrato_id=p_contrato AND remanente<0)
 OR EXISTS(SELECT 1 FROM v_estado_cuenta
           WHERE contrato_id=p_contrato AND (saldo<0 OR abonado>importe))
THEN
  RAISE EXCEPTION 'Aplicación o salida excede origen/destino' USING
ERRCODE='23514';
 END IF;
 IF EXISTS(
  SELECT 1 FROM aplicacion_pago a JOIN cargo c ON c.id=a.cargo_id
  LEFT JOIN pago p ON p.id=a.pago_id
  LEFT JOIN movimiento_deposito m ON m.id=a.movimiento_id
  LEFT JOIN deposito_garantia d ON d.id=m.deposito_id
  WHERE (c.contrato_id=p_contrato OR p.contrato_id=p_contrato OR
d.contrato_id=p_contrato)
  AND (coalesce(p.contrato_id,d.contrato_id)<>c.contrato_id
   OR (m.id IS NOT NULL AND (m.tipo<>'Aplicacion' OR m.cargo_id<>c.id OR
m.importe<>a.importe))
   OR (a.revertida_en IS NULL AND (c.anulado_en IS NOT NULL OR
p.estado='Anulado')))
 ) THEN RAISE EXCEPTION 'Origen y destino incompatibles' USING
ERRCODE='23514'; END IF;
 IF EXISTS(SELECT 1 FROM movimiento_deposito m
  JOIN deposito_garantia d ON d.id=m.deposito_id
  LEFT JOIN cargo c ON c.id=m.cargo_id
  WHERE d.contrato_id=p_contrato AND m.tipo='Aplicacion'
```

```
  AND (c.contrato_id<>d.contrato_id OR NOT EXISTS(
   SELECT 1 FROM aplicacion_pago a WHERE a.movimiento_id=m.id AND
a.revertida_en IS NULL))) THEN
  RAISE EXCEPTION 'Garantía sin aplicación equivalente' USING
ERRCODE='23514';
 END IF;
 IF EXISTS(SELECT 1 FROM pago p JOIN contrato k ON k.id=p.contrato_id
  LEFT JOIN autorizacion_tasa t ON t.id=p.autorizacion_id
  WHERE k.id=p_contrato AND (
   (p.moneda_original=k.moneda AND (p.tasa_usada IS NOT NULL OR
p.monto_original<>p.monto_contrato))
   OR (p.moneda_original<>k.moneda AND (t.id IS NULL OR
t.fecha_efectiva<>p.fecha_pago
    OR t.valor<>p.tasa_usada OR p.monto_contrato<>fn_redondear_par(
     CASE WHEN p.moneda_original='NIO' THEN p.monto_original/p.tasa_usada
          ELSE p.monto_original*p.tasa_usada END,2)))
   OR NOT EXISTS(SELECT 1 FROM recibo r WHERE r.pago_id=p.id))) THEN
  RAISE EXCEPTION 'Conversión o recibo incompatible' USING ERRCODE='23514';
 END IF;
 IF EXISTS(SELECT 1 FROM cargo m JOIN cargo r ON r.id=m.cargo_origen_id
  WHERE m.contrato_id=p_contrato AND m.tipo='Mora'
  AND (r.tipo<>'Renta' OR r.contrato_id<>m.contrato_id)) THEN
  RAISE EXCEPTION 'Mora sobre origen inválido' USING ERRCODE='23514';
 END IF;
END $$;
REVOKE EXECUTE ON ALL FUNCTIONS IN SCHEMA latch FROM PUBLIC;
REVOKE EXECUTE ON ALL PROCEDURES IN SCHEMA latch FROM PUBLIC;
COMMIT;
```

R001 conserva las diez consultas de reportes como sentencias PREPARE con tipos explícitos y agrega dos consultas complementarias al estado de cuenta histórico. Los parámetros posicionales tienen el orden declarado en la tabla de cada reporte y se enlazan con PreparedStatement en el adaptador Java. La sesión que ejecute PREPARE puede invocar EXECUTE para comprobarlas sin crear objetos permanentes adicionales en el esquema. El complemento de crédito toma la fecha de aplicación, creación y reversión para reconstruir el remanente al corte. La consulta de pagos conserva referencias de recibo y calcula el estado de anulación a esa misma fecha local.

```
-- R001: fuentes de reportes con parámetros tipados; sin credenciales.

SET search_path=latch,pg_catalog;

-- Recibo de pago

PREPARE reporte_01(BIGINT) AS
SELECT * FROM latch.v_recibo WHERE numero=$1;
```

```
-- Estado de cuenta

PREPARE reporte_02(DATE,UUID) AS
SELECT * FROM latch.fn_cargos_corte($1) WHERE contrato_id=$2 ORDER BY
vence_el,cargo_id;

-- Cartera vencida

PREPARE reporte_03(DATE,DATE) AS
SELECT e.*,k.inquilino_id,k.unidad_id FROM latch.fn_cargos_corte($1) e JOIN
latch.contrato k ON k.id=e.contrato_id WHERE e.saldo>0 AND e.vence_el<$2 AND
e.estado<>'Anulado' ORDER BY e.vence_el,e.cargo_id;

-- Ingresos y gastos

PREPARE reporte_04(DATE,DATE,DATE,DATE,DATE,DATE) AS
WITH i AS (SELECT u.propiedad_id,p.moneda_original
moneda,sum(p.monto_original) ingreso FROM latch.pago p JOIN latch.contrato k
ON k.id=p.contrato_id JOIN latch.unidad u ON u.id=k.unidad_id WHERE
p.fecha_pago BETWEEN $1 AND $2 AND (p.anulado_en IS NULL OR
p.anulado_en>=latch.fn_fin_corte($3)) AND p.creado_en<latch.fn_fin_corte($4)
GROUP BY 1,2), g AS (SELECT propiedad_id,moneda,sum(importe) gasto FROM
latch.gasto_propiedad WHERE fecha_gasto BETWEEN $5 AND $6 GROUP BY 1,2)
SELECT coalesce(i.propiedad_id,g.propiedad_id)
propiedad_id,coalesce(i.moneda,g.moneda) moneda,coalesce(ingreso,0)
ingreso,coalesce(gasto,0) gasto,coalesce(ingreso,0)-coalesce(gasto,0) neto
FROM i FULL JOIN g USING(propiedad_id,moneda);

-- Ocupación actual

PREPARE reporte_05 AS
SELECT propiedad_id,estado,count(*) unidades FROM latch.v_ocupacion GROUP BY
propiedad_id,estado ORDER BY propiedad_id,estado;

-- Contratos por vencer

PREPARE reporte_06(DATE,DATE) AS
SELECT * FROM latch.contrato WHERE estado IN ('Vigente','Por vencer') AND fin
BETWEEN $1 AND $2 ORDER BY fin,id;

-- Historial de pagos

PREPARE reporte_07(DATE,DATE) AS
SELECT p.*,r.numero FROM latch.pago p JOIN latch.recibo r ON r.pago_id=p.id
WHERE p.fecha_pago BETWEEN $1 AND $2 ORDER BY p.fecha_pago,r.numero;

-- Garantías y liquidaciones

PREPARE reporte_08 AS
SELECT d.*,m.id
movimiento_id,m.tipo,m.fecha_movimiento,m.importe,m.concepto,m.metodo FROM
latch.v_depositos d LEFT JOIN latch.movimiento_deposito m ON
m.deposito_id=d.deposito_id ORDER BY d.contrato_id,m.fecha_movimiento,m.id;

-- Gastos de mantenimiento

PREPARE reporte_09(DATE,DATE) AS
```

```
SELECT g.*,s.unidad_id,s.estado FROM latch.gasto_propiedad g LEFT JOIN
latch.solicitud_mantenimiento s ON s.id=g.solicitud_id WHERE g.fecha_gasto
BETWEEN $1 AND $2 ORDER BY g.propiedad_id,g.fecha_gasto,g.id;

-- Tasas utilizadas

PREPARE reporte_10(DATE,DATE) AS
SELECT p.id AS pago_id,p.contrato_id,p.fecha_pago,p.moneda_original,
 p.monto_original,p.monto_contrato,p.tasa_usada,p.estado,
 a.id AS autorizacion_id,a.fecha_efectiva,a.valor AS tasa_autorizada,
 a.fuente AS fuente_autorizacion,a.fundamento,
 t.id AS referencia_id,t.valor AS
tasa_referencia,t.proveedor,t.actualizado_en
FROM latch.pago p
JOIN latch.autorizacion_tasa a ON a.id=p.autorizacion_id
LEFT JOIN latch.tipo_cambio t ON t.id=a.referencia_id
WHERE p.fecha_pago BETWEEN $1 AND $2
ORDER BY p.fecha_pago,p.id;

-- Complementos al estado de cuenta histórico, misma transacción READ ONLY.
PREPARE reporte_credito_corte(DATE,UUID) AS
SELECT p.id AS pago_id,k.moneda,p.monto_contrato-coalesce(sum(a.importe)
 FILTER(WHERE a.fecha_aplicacion<=$1 AND a.creado_en<fn_fin_corte($1)
 AND (a.revertida_en IS NULL OR a.revertida_en>=fn_fin_corte($1))),0) AS
disponible
FROM pago p JOIN contrato k ON k.id=p.contrato_id
LEFT JOIN aplicacion_pago a ON a.pago_id=p.id
WHERE p.contrato_id=$2 AND p.fecha_pago<=$1 AND p.creado_en<fn_fin_corte($1)
AND (p.anulado_en IS NULL OR p.anulado_en>=fn_fin_corte($1))
GROUP BY p.id,k.moneda;

PREPARE reporte_pagos_corte(DATE,UUID) AS
SELECT p.id,r.numero,p.fecha_pago,p.moneda_original,p.monto_original,
p.monto_contrato,p.tasa_usada,
CASE WHEN p.anulado_en<fn_fin_corte($1) THEN 'Anulado' ELSE 'Registrado' END
estado_corte
FROM pago p JOIN recibo r ON r.pago_id=p.id
WHERE p.contrato_id=$2 AND p.fecha_pago<=$1 AND p.creado_en<fn_fin_corte($1)
ORDER BY p.fecha_pago,r.numero;
```

Apéndice C

Script de Datos Semilla

V003 inicializa catálogos de roles, controles y parámetros sin crear una cuenta de acceso con contraseña predeterminada. T001 es un fixture independiente para una base de demostración vacía y declara sintéticos sus nombres, montos y tasas. La generación y validación de los hashes PHC de sus cuentas inactivas quedan pendientes de reejecución; no se reutilizan credenciales operativas. El operador crea su Propietario inicial con la herramienta de consola que sigue a los

guiones. El aprovisionamiento exige contraseña ingresada en ejecución y cambio inicial, mientras la prueba del esquema puede usar datos sintéticos sin reutilizar credenciales operativas.

V003 Catálogos y Parámetros

```
-- V003: catálogos y parámetros, sin cuenta ni contraseña predeterminadas.

BEGIN;

SET search_path=latch,pg_catalog;

SET TIME ZONE 'America/Managua';

INSERT INTO rol(id,nombre) VALUES
('10000000-0000-0000-0000-000000000001','Propietario');

INSERT INTO rol(id,nombre) VALUES
('10000000-0000-0000-0000-000000000002','Asistente');

INSERT INTO consecutivo_recibo(id,siguiente_numero) VALUES
('00000000-0000-0000-0000-000000000001',1);

INSERT INTO puerta_mantenimiento(id,descripcion) VALUES
('00000000-0000-0000-0000-000000000002','Escrituras y respaldo con
adjuntos');

INSERT INTO control_fx(id,mes) VALUES
('00000000-0000-0000-0000-000000000003','2026-10-01');

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'dias_gracia','3'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='dias_gracia' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'mora_tipo','"Porcentaje"'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='mora_tipo' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'mora_valor','0'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='mora_valor' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
```

```
SELECT 'dias_por_vencer','30'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='dias_por_vencer' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'max_intentos','5'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='max_intentos' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'bloqueo_minutos','15'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='bloqueo_minutos' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'h2_intervalo_minutos','5'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='h2_intervalo_minutos' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'fx_intervalo_horas','24'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='fx_intervalo_horas' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'fx_presupuesto_mes','100'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='fx_presupuesto_mes' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'fx_concesion_segundos','90'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='fx_concesion_segundos' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'inactividad_minutos','15'::jsonb,now(),'Configuracion inicial; rango
```

```
5-60 minutos'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='inactividad_minutos' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'emisor','{"nombre": "Latch demostración académica", "direccion":
"Managua, datos sintéticos", "telefono": "No configurado para
contacto"}'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='emisor' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'retencion_finanzas_anios','5'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='retencion_finanzas_anios' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'retencion_logs_dias','90'::jsonb,now(),'Configuracion inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='retencion_logs_dias' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

INSERT INTO parametro_sistema(clave,valor,vigente_desde,motivo)
SELECT 'revision_adjuntos_al_cierre','true'::jsonb,now(),'Configuracion
inicial'
WHERE NOT EXISTS (SELECT 1 FROM (
 SELECT id FROM parametro_sistema
 WHERE clave='revision_adjuntos_al_cierre' AND vigente_desde<=now()
 ORDER BY vigente_desde DESC LIMIT 1
) vigente);

COMMIT;
```

T001 Demostración Sintética

```
-- T001: SOLO DEMOSTRACIÓN. Personas, montos, tasas y datos son sintéticos.

-- No ejecutar en una instalación con datos reales. Las cuentas semilla están
inactivas.

BEGIN;

SET search_path=latch,pg_catalog;

SET TIME ZONE 'America/Managua';
```

```
INSERT INTO
usuario(id,rol_id,nombre_usuario,nombre_visible,hash_clave,activo,cambio_inic
ial,creado_en) VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000001','demo_propietario','Cuenta sintética
demo_propietario','$argon2id$v=19$m=19456,t=2,p=1$SrYrscvAzV6dV/f6ixnF1g$A4g9
PAO3zUQ6H5U881wQmbXvAFGgj4pawGEQnrApFCc',FALSE,TRUE,'2026-10-01T08:00:00-
06:00');

INSERT INTO
usuario(id,rol_id,nombre_usuario,nombre_visible,hash_clave,activo,cambio_inic
ial,creado_en) VALUES
('10000000-0000-0000-0000-000000000011','10000000-0000-0000-0000-
000000000002','demo_asistente','Cuenta sintética
demo_asistente','$argon2id$v=19$m=19456,t=2,p=1$F6NDBEkGW/Sk7dghidEM4A$deoDoU
3W+eFSw0om0W/mACikOWBh5bq4i3zG86UY8XE',FALSE,TRUE,'2026-10-01T08:00:00-
06:00');

INSERT INTO propiedad(id,nombre,direccion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000020','Propiedad sintética 1','Managua,
ubicación ficticia 1','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO propiedad(id,nombre,direccion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000021','Propiedad sintética 2','Managua,
ubicación ficticia 2','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000030','10000000-0000-0000-0000-
000000000020','U01','Casa','Disponible',100,'USD','10000000-0000-0000-0000-
000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000031','10000000-0000-0000-0000-
000000000020','U02','Apartamento','Disponible',3000,'NIO','10000000-0000-
0000-0000-000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000032','10000000-0000-0000-0000-
000000000020','U03','Cuarto','Disponible',100,'USD','10000000-0000-0000-0000-
000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000033','10000000-0000-0000-0000-
000000000020','U04','Local comercial','Disponible',3000,'NIO','10000000-0000-
0000-0000-000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
```

```
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000034','10000000-0000-0000-0000-
000000000021','U05','Casa','Disponible',100,'USD','10000000-0000-0000-0000-
000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000035','10000000-0000-0000-0000-
000000000021','U06','Apartamento','Disponible',3000,'NIO','10000000-0000-
0000-0000-000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000036','10000000-0000-0000-0000-
000000000021','U07','Cuarto','En mantenimiento',100,'USD','10000000-0000-
0000-0000-000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO
unidad(id,propiedad_id,codigo,tipo,estado,canon_referencia,moneda,creado_por,
creado_en) VALUES
('10000000-0000-0000-0000-000000000037','10000000-0000-0000-0000-
000000000021','U08','Local comercial','Disponible',3000,'NIO','10000000-0000-
0000-0000-000000000010','2026-10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000040','Persona sintética
01','persona01@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000041','Persona sintética
02','persona02@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000042','Persona sintética
03','persona03@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000043','Persona sintética
04','persona04@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000044','Persona sintética
05','persona05@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');

INSERT INTO inquilino(id,nombre,correo,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000045','Persona sintética
06','persona06@example.invalid','10000000-0000-0000-0000-000000000010','2026-
10-01T08:00:00-06:00');
```

```
INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000050','10000000-0000-0000-0000-
000000000030','10000000-0000-0000-0000-000000000040','2026-08-01','2027-07-
31',NULL,100,'USD',5,100,'Vigente','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000051','10000000-0000-0000-0000-
000000000031','10000000-0000-0000-0000-000000000041','2026-08-01','2026-11-
15',NULL,3000,'NIO',5,3000,'Por vencer','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000052','10000000-0000-0000-0000-
000000000032','10000000-0000-0000-0000-000000000042','2026-08-01','2026-09-
30',NULL,100,'USD',5,100,'Vencido','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000053','10000000-0000-0000-0000-
000000000033','10000000-0000-0000-0000-000000000043','2026-08-01','2026-09-
30','2026-10-20',3000,'NIO',5,3000,'Terminado','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000054','10000000-0000-0000-0000-
000000000034','10000000-0000-0000-0000-000000000044','2026-08-01','2026-09-
30',NULL,100,'USD',5,100,'Cancelado','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000055','10000000-0000-0000-0000-
000000000035','10000000-0000-0000-0000-000000000045','2026-08-01','2026-09-
30',NULL,3000,'NIO',5,3000,'Borrador','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
cargo(id,contrato_id,tipo,periodo,fecha_emision,vence_el,importe,concepto,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000060','10000000-0000-0000-0000-
000000000050','Renta','2026-10-01','2026-10-01','2026-10-05',100,'Renta
octubre sintética','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO
```

```
cargo(id,contrato_id,tipo,periodo,fecha_emision,vence_el,importe,concepto,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000061','10000000-0000-0000-0000-
000000000051','Renta','2026-10-01','2026-10-01','2026-10-05',3000,'Renta
octubre sintética','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO
cargo(id,contrato_id,tipo,periodo,fecha_emision,vence_el,importe,concepto,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000062','10000000-0000-0000-0000-
000000000052','Renta','2026-10-01','2026-10-01','2026-10-05',100,'Renta
octubre sintética','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO
cargo(id,contrato_id,tipo,periodo,fecha_emision,vence_el,importe,concepto,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000063','10000000-0000-0000-0000-
000000000053','Renta','2026-10-01','2026-10-01','2026-10-05',3000,'Renta
octubre sintética','10000000-0000-0000-0000-000000000010','2026-10-
01T08:00:00-06:00');

INSERT INTO
cargo(id,contrato_id,tipo,cargo_origen_id,periodo,fecha_emision,vence_el,impo
rte,concepto,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000064','10000000-0000-0000-0000-
000000000050','Mora','10000000-0000-0000-0000-000000000060','2026-10-
01','2026-10-09','2026-10-09',5,'Mora sintética: política histórica de
ejemplo','10000000-0000-0000-0000-000000000010','2026-10-09T08:00:00-06:00');

INSERT INTO
tipo_cambio(id,proveedor,valor,actualizado_en,proximo_en,consultado_en,huella
_respuesta,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000070','SIMULADOR, no consulta
real','36.50000000','2026-10-23T00:00:00Z','2026-10-24T00:00:00Z','2026-10-
23T09:00:00-
06:00','0000000000000000000000000000000000000000000000000000000000000000','10
000000-0000-0000-0000-000000000010','2026-10-23T09:00:00-06:00');

INSERT INTO
autorizacion_tasa(id,fecha_efectiva,valor,fuente,fundamento,referencia_id,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000071','2026-10-23','36.50000000','Escenario
sintético, no tasa oficial ni cotización real','Prueba académica de
conversión','10000000-0000-0000-0000-000000000070','10000000-0000-0000-0000-
000000000010','2026-10-23T09:00:00-06:00');

INSERT INTO
pago(id,contrato_id,solicitud_id,solicitud_hash,fecha_pago,moneda_original,mo
nto_original,monto_contrato,tasa_usada,autorizacion_id,metodo,estado,anulado_
en,anulado_por,motivo_anulacion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000080','10000000-0000-0000-0000-
000000000050','10000000-0000-0000-0000-
000000000180','0000000000000000000000000000000000000000000000000000000000000
00','2026-10-23','NIO',1460,40,'36.50000000','10000000-0000-0000-0000-
000000000071','Efectivo','Registrado',NULL,NULL,NULL,'10000000-0000-0000-
```

```
0000-000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO recibo(id,pago_id,numero,instantanea,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000090','10000000-0000-0000-0000-
000000000080',1,'{"emisor": "Latch demostración académica", "inquilino":
"Persona sintética 01", "monto_original": "1460", "moneda_original": "NIO",
"monto_contrato": "40", "tasa_usada": "36.50000000", "leyenda": "Recibo
administrativo, datos sintéticos"}','10000000-0000-0000-0000-
000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
pago(id,contrato_id,solicitud_id,solicitud_hash,fecha_pago,moneda_original,mo
nto_original,monto_contrato,tasa_usada,autorizacion_id,metodo,estado,anulado_
en,anulado_por,motivo_anulacion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000081','10000000-0000-0000-0000-
000000000051','10000000-0000-0000-0000-
000000000181','11111111111111111111111111111111111111111111111111111111111
11','2026-10-
23','NIO',3500,3500,NULL,NULL,'Transferencia','Registrado',NULL,NULL,NULL,'10
000000-0000-0000-0000-000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO recibo(id,pago_id,numero,instantanea,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000091','10000000-0000-0000-0000-
000000000081',2,'{"emisor": "Latch demostración académica", "inquilino":
"Persona sintética 02", "monto_original": "3500", "moneda_original": "NIO",
"monto_contrato": "3500", "tasa_usada": null, "leyenda": "Recibo
administrativo, datos sintéticos"}','10000000-0000-0000-0000-
000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
pago(id,contrato_id,solicitud_id,solicitud_hash,fecha_pago,moneda_original,mo
nto_original,monto_contrato,tasa_usada,autorizacion_id,metodo,estado,anulado_
en,anulado_por,motivo_anulacion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000082','10000000-0000-0000-0000-
000000000052','10000000-0000-0000-0000-
000000000182','22222222222222222222222222222222222222222222222222222222222
22','2026-10-23','USD',50,50,NULL,NULL,'Depósito
bancario','Registrado',NULL,NULL,NULL,'10000000-0000-0000-0000-
000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO recibo(id,pago_id,numero,instantanea,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000092','10000000-0000-0000-0000-
000000000082',3,'{"emisor": "Latch demostración académica", "inquilino":
"Persona sintética 03", "monto_original": "50", "moneda_original": "USD",
"monto_contrato": "50", "tasa_usada": null, "leyenda": "Recibo
administrativo, datos sintéticos"}','10000000-0000-0000-0000-
000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
pago(id,contrato_id,solicitud_id,solicitud_hash,fecha_pago,moneda_original,mo
nto_original,monto_contrato,tasa_usada,autorizacion_id,metodo,estado,anulado_
en,anulado_por,motivo_anulacion,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000083','10000000-0000-0000-0000-
000000000050','10000000-0000-0000-0000-
000000000183','33333333333333333333333333333333333333333333333333333333333
33','2026-10-23','USD',10,10,NULL,NULL,'Otro','Anulado','2026-10-23T10:00:00-
06:00','10000000-0000-0000-0000-000000000010','Corrección sintética de
```

```
captura','10000000-0000-0000-0000-000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO recibo(id,pago_id,numero,instantanea,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000093','10000000-0000-0000-0000-
000000000083',4,'{"emisor": "Latch demostración académica", "inquilino":
"Persona sintética 01", "monto_original": "10", "moneda_original": "USD",
"monto_contrato": "10", "tasa_usada": null, "leyenda": "Recibo
administrativo, datos sintéticos"}','10000000-0000-0000-0000-
000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
aplicacion_pago(id,pago_id,cargo_id,importe,origen,fecha_aplicacion,creado_po
r,creado_en) VALUES
('10000000-0000-0000-0000-000000000100','10000000-0000-0000-0000-
000000000080','10000000-0000-0000-0000-000000000060',40,'Inicial','2026-10-
23','10000000-0000-0000-0000-000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
aplicacion_pago(id,pago_id,cargo_id,importe,origen,fecha_aplicacion,creado_po
r,creado_en) VALUES
('10000000-0000-0000-0000-000000000101','10000000-0000-0000-0000-
000000000081','10000000-0000-0000-0000-000000000061',3000,'Inicial','2026-10-
23','10000000-0000-0000-0000-000000000010','2026-10-23T09:10:00-06:00');

INSERT INTO
aplicacion_pago(id,pago_id,cargo_id,importe,origen,fecha_aplicacion,creado_po
r,creado_en) VALUES
('10000000-0000-0000-0000-000000000102','10000000-0000-0000-0000-
000000000082','10000000-0000-0000-0000-000000000062',50,'Inicial','2026-10-
23','10000000-0000-0000-0000-000000000010','2026-10-23T09:10:00-06:00');

UPDATE consecutivo_recibo SET siguiente_numero=5;

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
('10000000-0000-0000-0000-000000000110','10000000-0000-0000-0000-
000000000050',100,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
('10000000-0000-0000-0000-000000000111','10000000-0000-0000-0000-
000000000051',3000,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
('10000000-0000-0000-0000-000000000112','10000000-0000-0000-0000-
000000000052',100,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
```

```
('10000000-0000-0000-0000-000000000113','10000000-0000-0000-0000-
000000000053',3000,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000056','10000000-0000-0000-0000-
000000000036','10000000-0000-0000-0000-000000000040','2026-08-01','2026-09-
30','2026-10-20',100,'USD',5,100,'Terminado','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
('10000000-0000-0000-0000-000000000114','10000000-0000-0000-0000-
000000000056',100,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
contrato(id,unidad_id,inquilino_id,inicio,fin,terminado_el,canon,moneda,dia_p
ago,deposito_pactado,estado,creado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000057','10000000-0000-0000-0000-
000000000037','10000000-0000-0000-0000-000000000041','2026-08-01','2026-09-
30',NULL,100,'USD',5,100,'Vencido','10000000-0000-0000-0000-
000000000010','2026-08-01T08:00:00-06:00');

INSERT INTO
deposito_garantia(id,contrato_id,importe_inicial,fecha_recepcion,creado_por,c
reado_en) VALUES
('10000000-0000-0000-0000-000000000115','10000000-0000-0000-0000-
000000000057',100,'2026-08-01','10000000-0000-0000-0000-000000000010','2026-
08-01T08:00:00-06:00');

INSERT INTO
cargo(id,contrato_id,tipo,periodo,fecha_emision,vence_el,importe,concepto,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000065','10000000-0000-0000-0000-
000000000056','Reparación','2026-10-01','2026-10-20','2026-10-
20',100,'Reparación sintética autorizada al cierre','10000000-0000-0000-0000-
000000000010','2026-10-20T09:15:00-06:00');

INSERT INTO
movimiento_deposito(id,deposito_id,cargo_id,liquidacion_id,liquidacion_hash,o
rden,tipo,importe,fecha_movimiento,metodo,concepto,creado_por,creado_en)
VALUES
('10000000-0000-0000-0000-000000000120','10000000-0000-0000-0000-
000000000114','10000000-0000-0000-0000-000000000065','10000000-0000-0000-
0000-
000000000220','2fa009171f8bac9fb30c8717facc5b2b614fdbfb7d631a61657283cf4e6c98
dc',1,'Aplicacion',100,'2026-10-23',NULL,'Liquidación sintética:
Aplicación','10000000-0000-0000-0000-000000000010','2026-10-23T09:20:00-
06:00');

INSERT INTO
movimiento_deposito(id,deposito_id,cargo_id,liquidacion_id,liquidacion_hash,o
rden,tipo,importe,fecha_movimiento,metodo,concepto,creado_por,creado_en)
```

```
VALUES
('10000000-0000-0000-0000-000000000121','10000000-0000-0000-0000-
000000000115',NULL,'10000000-0000-0000-0000-
000000000221','9f4dc59afa280a900a82d139264e6f71f85087487cd222025e28308f57d8db
71',1,'Devolucion',20,'2026-10-23','Efectivo','Liquidación sintética:
Devolución','10000000-0000-0000-0000-000000000010','2026-10-23T09:20:00-
06:00');

INSERT INTO
movimiento_deposito(id,deposito_id,cargo_id,liquidacion_id,liquidacion_hash,o
rden,tipo,importe,fecha_movimiento,metodo,concepto,creado_por,creado_en)
VALUES
('10000000-0000-0000-0000-000000000122','10000000-0000-0000-0000-
000000000113',NULL,'10000000-0000-0000-0000-
000000000222','d369641fef9ff8f240347ad3099097af38377bf051d3d77f4f7e847099f953
37',1,'Devolucion',3000,'2026-10-23','Efectivo','Liquidación sintética:
Devolución','10000000-0000-0000-0000-000000000010','2026-10-23T09:20:00-
06:00');

INSERT INTO
aplicacion_pago(id,movimiento_id,cargo_id,importe,origen,fecha_aplicacion,cre
ado_por,creado_en) VALUES
('10000000-0000-0000-0000-000000000103','10000000-0000-0000-0000-
000000000120','10000000-0000-0000-0000-000000000065',100,'Garantia','2026-10-
23','10000000-0000-0000-0000-000000000010','2026-10-23T09:20:00-06:00');

INSERT INTO
solicitud_mantenimiento(id,unidad_id,descripcion,prioridad,estado,abierta_el,
cerrada_el,creado_por) VALUES
('10000000-0000-0000-0000-000000000130','10000000-0000-0000-0000-
000000000030','Incidencia sintética 1','Baja','Abierta','2026-10-
20',NULL,'10000000-0000-0000-0000-000000000010');

INSERT INTO
solicitud_mantenimiento(id,unidad_id,descripcion,prioridad,estado,abierta_el,
cerrada_el,creado_por) VALUES
('10000000-0000-0000-0000-000000000131','10000000-0000-0000-0000-
000000000031','Incidencia sintética 2','Media','En proceso','2026-10-
20',NULL,'10000000-0000-0000-0000-000000000010');

INSERT INTO
solicitud_mantenimiento(id,unidad_id,descripcion,prioridad,estado,abierta_el,
cerrada_el,creado_por) VALUES
('10000000-0000-0000-0000-000000000132','10000000-0000-0000-0000-
000000000032','Incidencia sintética 3','Alta','Resuelta','2026-10-20','2026-
10-22','10000000-0000-0000-0000-000000000010');

INSERT INTO
solicitud_mantenimiento(id,unidad_id,descripcion,prioridad,estado,abierta_el,
cerrada_el,creado_por) VALUES
('10000000-0000-0000-0000-000000000133','10000000-0000-0000-0000-
000000000033','Incidencia sintética 4','Media','Cancelada','2026-10-
20','2026-10-22','10000000-0000-0000-0000-000000000010');

INSERT INTO
gasto_propiedad(id,propiedad_id,solicitud_id,fecha_gasto,importe,moneda,conce
pto,creado_por) VALUES
```

```
('10000000-0000-0000-0000-000000000140','10000000-0000-0000-0000-
000000000020','10000000-0000-0000-0000-000000000132','2026-10-
22',600,'NIO','Gasto de mantenimiento sintético','10000000-0000-0000-0000-
000000000010');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000180','M06','IMPORTAR_DEMO','pago','10000000-0000-0000-0000-
000000000080','{"sintetico":true,"origen":"T001"}');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000050');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000051');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000052');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000053');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000054');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000055');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000056');

CALL sp_verificar_integridad('10000000-0000-0000-0000-000000000057');


INSERT INTO
documento_adjunto(id,contrato_id,nombre_original,ruta_interna,mime,tamano_byt
es,sha256,creado_por) VALUES
('10000000-0000-0000-0000-000000000300','10000000-0000-0000-0000-
000000000050','Contrato sintetico de prueba.pdf','10000000-0000-0000-0000-
000000000300.pdf','application/
pdf',1435,'cba768919446e6609bc5f634dfb213ca5fcffd72173d80819de9568e6450a887',
'10000000-0000-0000-0000-000000000010');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000180','M06','PAGO_SINTETICO','pago','10000000-0000-0000-0000-
000000000080','{"sintetico": true, "numero": 1}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000181','M06','PAGO_SINTETICO','pago','10000000-0000-0000-0000-
000000000081','{"sintetico": true, "numero": 2}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000182','M06','PAGO_SINTETICO','pago','10000000-0000-0000-0000-
```

```
000000000082','{"sintetico": true, "numero": 3}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000183','M06','PAGO_SINTETICO','pago','10000000-0000-0000-0000-
000000000083','{"sintetico": true, "numero": 4}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000220','M07','LIQUIDACION_SINTETICA','movimiento_deposito','10000000-
0000-0000-0000-000000000120','{"sintetico":true}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000221','M07','LIQUIDACION_SINTETICA','movimiento_deposito','10000000-
0000-0000-0000-000000000121','{"sintetico":true}');

INSERT INTO
bitacora(creado_por,correlacion_id,modulo,accion,entidad,entidad_id,detalle)
VALUES
('10000000-0000-0000-0000-000000000010','10000000-0000-0000-0000-
000000000222','M07','LIQUIDACION_SINTETICA','movimiento_deposito','10000000-
0000-0000-0000-000000000122','{"sintetico":true}');

COMMIT;
```

Aprovisionamiento Seguro en Java

Aprovisionar.java usa Bouncy Castle para calcular Argon2id y pgJDBC para insertar la cuenta y su evento en una conexión. LATCH_DB_URL contiene la conexión configurada con TLS y LATCH_PROVISION_USER identifica la cuenta técnica autorizada; ambos valores provienen del proceso de instalación y no representan campos editoriales pendientes. La consola solicita el secreto técnico y la contraseña funcional sin eco, y el programa rechaza una ejecución sin consola interactiva. El POM de ejemplos del paquete declara las dependencias fijadas por el TRD. Su código ilustra el procedimiento completo de alta inicial; SQL, hashes de fixture y esta herramienta Java quedan pendientes de reejecución y validación.

```
package ni.edu.uam.latch.herramientas;
import java.io.Console;
import java.security.SecureRandom;
import java.sql.*;
```

```
import java.util.*;
import org.bouncycastle.crypto.generators.Argon2BytesGenerator;
import org.bouncycastle.crypto.params.Argon2Parameters;

public final class Aprovisionar {
 public static void main(String[] args) throws Exception {
  Console consola=System.console();
  if(consola==null) throw new IllegalStateException("Requiere consola local
interactiva");
  String url=Objects.requireNonNull(System.getenv("LATCH_DB_URL"));
  String
dbUser=Objects.requireNonNull(System.getenv("LATCH_PROVISION_USER"));
  char[] dbPass=consola.readPassword("Secreto de conexión de
aprovisionamiento: ");
  char[] clave=consola.readPassword("Contraseña inicial de Latch: ");
  char[] segunda=consola.readPassword("Repita la contraseña inicial: ");
  try {
   if(!Arrays.equals(clave,segunda) || clave.length<12 || clave.length>128)
    throw new IllegalArgumentException("Longitud de 12 a 128 y confirmación
idéntica requeridas");
   String nombre=consola.readLine("Cuenta inicial (ASCII, 3 a 40 caracteres):
");
   if(!nombre.matches("[A-Za-z0-9_.]{3,40}"))
    throw new IllegalArgumentException("Nombre de cuenta inválido");
   byte[] sal=new byte[16],derivado=new byte[32];new
SecureRandom().nextBytes(sal);
   var parametros=new Argon2Parameters.Builder(Argon2Parameters.ARGON2_id)
    .withVersion(Argon2Parameters.ARGON2_VERSION_13).withSalt(sal)
    .withMemoryAsKB(19456).withIterations(2).withParallelism(1).build();
   var argon=new
Argon2BytesGenerator();argon.init(parametros);argon.generateBytes(clave,deriv
ado);
   var b64=Base64.getEncoder().withoutPadding();
   String phc="$argon2id$v=19$m=19456,t=2,p=1$"+b64.encodeToString(sal)
+"$"+b64.encodeToString(derivado);
   try(Connection c=DriverManager.getConnection(url,dbUser,new
String(dbPass))) {
    c.setAutoCommit(false);
    try {
     c.createStatement().execute("SET search_path=latch,pg_catalog");
     c.createStatement().execute("SELECT id FROM puerta_mantenimiento FOR
UPDATE");
     try(var s=c.prepareStatement("SELECT count(*) FROM usuario u JOIN rol r
ON r.id=u.rol_id WHERE r.nombre='Propietario' AND u.activo")) {
      try(var r=s.executeQuery()) {r.next();if(r.getLong(1)>0)
       throw new IllegalStateException("Ya existe Propietario activo; use
M01");}
     }
     UUID id=UUID.randomUUID();
     try(var s=c.prepareStatement("INSERT INTO
usuario(id,rol_id,nombre_usuario,nombre_visible,hash_clave,cambio_inicial,act
ivo) SELECT ?,id,?,?,?,true,true FROM rol WHERE nombre='Propietario'")) {

s.setObject(1,id);s.setString(2,nombre);s.setString(3,nombre);s.setString(4,p
hc);
      if(s.executeUpdate()!=1)throw new SQLException("Catálogo Propietario
ausente");
```

```
     }
     try(var s=c.prepareStatement("SELECT
fn_auditar(?,?,'M01','APROVISIONAR','usuario',?,'{}'::jsonb)")) {

s.setObject(1,id);s.setObject(2,UUID.randomUUID());s.setObject(3,id);s.execut
e();
     }
     c.commit();consola.printf("Propietario creado. Cambio de contraseña
obligatorio en el primer acceso.%n");
    } catch(Exception e) {c.rollback();throw e;}
   } finally {Arrays.fill(sal,(byte)0);Arrays.fill(derivado,(byte)0);}
  } finally
{Arrays.fill(clave,'\0');Arrays.fill(segunda,'\0');Arrays.fill(dbPass,'\0');}
 }
}
```

Apéndice D

Diccionario de Datos Completo

Cada tabla del diccionario enumera las columnas que aparecen en V001 y conserva su nombre físico sin traducciones. Nulo indica si SQL permite NULL y Clave distingue PK, FK o atributo ordinario; las unicidades compuestas constan en el catálogo de restricciones. Ninguno en Valor por defecto significa que el comando debe aportar el valor o que una columna opcional queda NULL, según su nulabilidad. Las reglas entre filas permanecen en la capa de servicios y el procedimiento financiero. Los valores visibles de catálogos conservan las etiquetas compartidas, y los instantes se muestran en America/Managua aunque su representación de auditoría sea temporal y no texto local.

La Tabla D1 presenta diccionario de rol.

Tabla D1

Diccionario de rol

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
nombre	VARCHAR (16)	No	No	Ninguno	nombre IN ('Propietario','Asistente')	Propietario o Asistente.	RN-13

La Tabla D2 presenta diccionario de usuario.

Tabla D2

Diccionario de usuario

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
rol_id	UUID	No	FK	Ninguno	→ rol.id; RESTRICT	Rol funcional asignado.	RN-13
nombre_usuario	VARCHAR (40)	No	No	Ninguno	nombre_usuario ~ '^[A-Za-z0-9_.]{3,40}$'	Cuenta ASCII de 3 a 40 caracteres; comparación sin mayúsculas.	RN-16

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
nombre_visible	VARCHAR (120)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre presentado en auditoría.	RN-16
hash_clave	TEXT	No	No	Ninguno	hash_clave LIKE '$argon2id$%'	PHC Argon2id con sal y parámetros, sin secreto en claro. La contraseña de entrada admite de 12 a 128 caracteres. Usuario ORM es @Immutable; seguridad por JDBC.	RN-15
cambio_inicial	BOOLEAN	No	No	TRUE	Tipo SQL; restricciones en Tabla D24.	Obliga a cambiar la contraseña antes de operar.	RN-15
intentos_fallidos	INTEGER	No	No	0	intentos_fallidos >= 0	Fallos consecutivos confirmados.	RN-15
bloqueado_hasta	TIMESTAMPTZ	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fin del bloqueo temporal.	RN-15
activo	BOOLEAN	No	No	TRUE	Tipo SQL; restricciones en Tabla D24.	Baja lógica de la cuenta.	RN-13
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D3 presenta diccionario de propiedad.

Tabla D3

Diccionario de propiedad

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
nombre	VARCHAR (120)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre administrativo.	RN-16
direccion	VARCHAR (300)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Ubicación declarada; sin geolocalización automática.	RN-18
activa	BOOLEAN	No	No	TRUE	Tipo SQL; restricciones en Tabla D24.	Baja lógica; exige ausencia de unidades activas.	RN-11
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D4 presenta diccionario de unidad.

Tabla D4

Diccionario de unidad

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
propiedad_id	UUID	No	FK	Ninguno	→ propiedad.id; RESTRICT	Propiedad contenedora.	RN-11
codigo	VARCHAR (30)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Código único dentro de su propiedad.	RN-16
tipo	VARCHAR (24)	No	No	'Cuarto'	tipo IN ('Casa','Apartamento','Cuarto','Local comercial')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-16
estado	VARCHAR (24)	No	No	'Disponible'	estado IN ('Disponible','En mantenimiento','Inactiva')	EstadoUnidad persistido. Ocupada se deriva en v_ocupacion y en EstadoUnidadVisible de UnidadDto.	RN-11
canon_referencia	NUMERIC(19,2)	No	No	Ninguno	canon_referencia > 0	Importe orientativo; el contrato conserva el suyo.	RN-03
moneda	VARCHAR (3)	No	No	Ninguno	moneda IN ('NIO','USD')	Moneda del importe.	RN-03

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D5 presenta diccionario de inquilino.

Tabla D5

Diccionario de inquilino

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
nombre	VARCHAR(160)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre para contrato y recibo.	RN-18
identificacion	VARCHAR(40)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Dato opcional sin validación de documento oficial.	RN-18
telefono	VARCHAR(30)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Contacto opcional.	RN-18

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
correo	VARCHAR (254)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Dirección opcional; no habilita envíos.	RN-18
contacto_emergencia	VARCHAR (200)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre y medio de contacto opcionales.	RN-18
activo	BOOLEAN	No	No	TRUE	Tipo SQL; restricciones en Tabla D24.	Baja lógica sin contrato activo.	RN-11
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D6 presenta diccionario de contrato.

Tabla D6

Diccionario de contrato

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
unidad_id	UUID	No	FK	Ninguno	→ unidad.id; RESTRICT	Unidad reservada.	RN-01
inquilino_id	UUID	No	FK	Ninguno	→ inquilino.id; RESTRICT	Titular del contrato.	RN-18
predecesor_id	UUID	Sí	FK	Ninguno	→ contrato.id; RESTRICT	Contrato anterior de una renovación.	RN-19
inicio	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local inicial.	RN-02
fin	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local final inclusiva; se conserva fin >= inicio + 1 mes. Un mes natural completo del día 1 al 31 exige fin >= día 1 del mes siguiente.	RN-02
terminado_el	DATE	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Cierre explícito; no borra deuda.	RN-19
canon	NUMERIC(19,2)	No	No	Ninguno	canon > 0	Renta completa del período mensual.	RN-03
moneda	VARCHAR(3)	No	No	Ninguno	moneda IN ('NIO','USD')	Moneda del importe.	RN-03
dia_pago	SMALLINT	No	No	Ninguno	dia_pago BETWEEN 1 AND 31	Día 1 a 31; ajuste al último día del mes.	RN-17
deposito_pactado	NUMERIC(19,2)	No	No	Ninguno	deposito_pactado >= 0	Garantía separada de renta.	RN-10

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
estado	VARCHAR (24)	No	No	'Borrador'	estado IN ('Borrador','Vigente', 'Por vencer','Vencido','Terminado','Cancelado')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-19
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D7 presenta diccionario de cargo.

Tabla D7

Diccionario de cargo

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
contrato_id	UUID	No	FK	Ninguno	→ contrato.id; RESTRICT	Contrato de la obligación.	RN-04

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
cargo_origen_id	UUID	Sí	FK	Ninguno	→ cargo.id; RESTRICT	Renta que originó mora o cargo corregido.	RN-05
tipo	VARCHAR (24)	No	No	'Renta'	tipo IN ('Renta','Mora','Reparación','Servicio','Otro')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-16
periodo	DATE	No	No	Ninguno	EXTRACT(DAY FROM periodo)=1	Primer día del mes facturado.	RN-04
fecha_emision	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha de nacimiento del cargo.	RN-34
vence_el	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local de exigibilidad.	RN-17
importe	NUMERIC(19,2)	No	No	Ninguno	importe > 0	Principal inmutable del cargo.	RN-03
concepto	VARCHAR (300)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Detalle administrativo; obligatorio.	RN-16
anulado_en	TIMESTAMPTZ	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Instante de anulación; se conserva el cargo.	RN-13
anulado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Propietario que anuló.	RN-13
motivo_anulacion	VARCHAR (500)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Motivo de anulación.	RN-13

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D8 presenta diccionario de tipo_cambio.

Tabla D8

Diccionario de tipo_cambio

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
proveedor	VARCHAR(80)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Organización que suministró la referencia.	RN-22
base	VARCHAR(3)	No	No	'USD'	base='USD'	Código USD.	RN-22
destino	VARCHAR(3)	No	No	'NIO'	destino='NIO'	Código NIO.	RN-22
valor	NUMERIC(19,8)	No	No	Ninguno	valor > 0	NIO por un USD; solo referencia.	RN-22

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
actualizado_en	TIMESTAMPTZ	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	time_last_update_unix convertido a Instant.	RN-22
proximo_en	TIMESTAMPTZ	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	time_next_update_unix convertido a Instant.	RN-22
consultado_en	TIMESTAMPTZ	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Reloj local al recibir y validar.	RN-22
huella_respuesta	VARCHAR(64)	No	No	Ninguno	huella_respuesta ~ '^[0-9a-f]{64}$'	SHA-256 hexadecimal del cuerpo validado.	RN-22
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D9 presenta diccionario de autorizacion_tasa.

Tabla D9

Diccionario de autorizacion_tasa

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
fecha_efectiva	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local del pago autorizado.	RN-09
valor	NUMERIC(19,8)	No	No	Ninguno	valor > 0	NIO por USD autorizado.	RN-32
fuente	VARCHAR(300)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Referencia documental admisible o fundamento manual.	RN-22
fundamento	VARCHAR(500)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Explicación de la autorización; no basta promover la caché.	RN-22
referencia_id	UUID	Sí	FK	Ninguno	→ tipo_cambio.id; RESTRICT	Referencia informativa relacionada, si existe.	RN-22
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D10 presenta diccionario de pago.

Tabla D10

Diccionario de pago

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
contrato_id	UUID	No	FK	Ninguno	→ contrato.id; RESTRICT	Contrato al que pertenece el pago.	RN-06
solicitud_id	UUID	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	UUID de idempotencia único.	RN-21
solicitud_hash	VARCHAR (64)	No	No	Ninguno	solicitud_hash ~ '^[0-9a-f]{64}$'	SHA-256 del comando canónico; detecta UUID reutilizado con otros datos.	RN-21
fecha_pago	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha efectiva ingresada por el usuario; no futura respecto a la fecha operativa (creado_en en America/Managua). La autorización de tasa se busca por esta fecha.	RN-09
moneda_original	VARCHAR (3)	No	No	Ninguno	moneda_original IN ('NIO','USD')	Moneda entregada por el inquilino.	RN-03
monto_original	NUMERIC(19,2)	No	No	Ninguno	monto_original > 0	Dinero recibido en moneda original.	RN-06

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
monto_contrato	NUMERIC(19,2)	No	No	Ninguno	monto_contrato > 0	Total convertido una vez a moneda del contrato.	RN-32
tasa_usada	NUMERIC(19,8)	Sí	No	Ninguno	tasa_usada > 0	NIO por USD; NULL si monedas iguales.	RN-09
autorizacion_id	UUID	Sí	FK	Ninguno	→ autorizacion_tasa.id; RESTRICT	Autorización usada para fecha y tasa.	RN-22
metodo	VARCHAR(24)	No	No	'Efectivo'	metodo IN ('Efectivo','Transferencia','Depósito bancario','Otro')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-16
estado	VARCHAR(24)	No	No	'Registrado'	estado IN ('Registrado','Anulado')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-08
anulado_en	TIMESTAMPTZ	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Instante de reversión administrativa.	RN-08
anulado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Actor de la anulación.	RN-08
motivo_anulacion	VARCHAR(500)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Razón obligatoria para anular.	RN-08
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D11 presenta diccionario de deposito_garantia.

Tabla D11

Diccionario de deposito_garantia

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
contrato_id	UUID	No	FK	Ninguno	→ contrato.id; RESTRICT	Contrato que origina una sola garantía.	RN-10
importe_inicial	NUMERIC(19,2)	No	No	Ninguno	importe_inicial >= 0	Garantía recibida al activar, incluso cero.	RN-10
fecha_recepcion	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local de recepción.	RN-10
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D12 presenta diccionario de movimiento_deposito.

Tabla D12

Diccionario de movimiento_deposito

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
deposito_id	UUID	No	FK	Ninguno	→ deposito_garantia.id; RESTRICT	Garantía afectada.	RN-26
cargo_id	UUID	Sí	FK	Ninguno	→ cargo.id; RESTRICT	Cargo pagado por aplicación.	RN-10
liquidacion_id	UUID	No	No	Ninguno	UNIQUE(liquidacion_id,orden); paquete identificado por liquidacion_id.	UUID compartido por todos los movimientos del paquete; la idempotencia consulta por liquidacion_id.	RN-21
liquidacion_hash	VARCHAR(64)	No	No	Ninguno	liquidacion_hash ~ '^[0-9a-f]{64}$'	SHA-256 compartido por todos los movimientos del paquete; una repetición debe conservar el hash.	RN-21
orden	SMALLINT	No	No	Ninguno	UNIQUE(liquidacion_id,orden)	Posición del movimiento dentro del paquete.	RN-21
tipo	VARCHAR(24)	No	No	'Aplicacion'	tipo IN ('Aplicacion','Devolucion')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-26

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
importe	NUMERIC(19,2)	No	No	Ninguno	importe > 0	Salida positiva en moneda contractual.	RN-26
fecha_movimiento	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local del movimiento.	RN-10
metodo	VARCHAR(24)	Sí	No	Ninguno	metodo IN ('Efectivo','Transferencia','Depósito bancario','Otro')	Método de devolución; NULL en aplicación sin salida de caja.	RN-10
concepto	VARCHAR(300)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Concepto obligatorio de cada deducción.	RN-10
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D13 presenta diccionario de aplicacion_pago.

Tabla D13

Diccionario de aplicacion_pago

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
pago_id	UUID	Sí	FK	Ninguno	→ pago.id; RESTRICT	Pago de origen; NULL si proviene de garantía.	RN-06
movimiento_id	UUID	Sí	FK	Ninguno	→ movimiento_deposito.id; RESTRICT	Aplicación de garantía de origen.	RN-10
cargo_id	UUID	No	FK	Ninguno	→ cargo.id; RESTRICT	Obligación que recibe el abono.	RN-06
importe	NUMERIC(19,2)	No	No	Ninguno	importe > 0	Abono en moneda del contrato destino.	RN-06
origen	VARCHAR(24)	No	No	'Inicial'	origen IN ('Inicial','Credito','Garantia')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-31
fecha_aplicacion	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha local para corte de reportes.	RN-28
revertida_en	TIMESTAMPTZ	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Reversión autorizada; conserva el movimiento.	RN-31

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
revertida_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Propietario que revirtió.	RN-31
motivo_reversion	VARCHAR(500)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Explicación de reversión.	RN-31
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D14 presenta diccionario de consecutivo_recibo.

Tabla D14

Diccionario de consecutivo_recibo

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
siguiente_numero	BIGINT	No	No	1	siguiente_numero > 0	Próximo número comercial sin reinicio anual.	RN-07

La Tabla D15 presenta diccionario de recibo.

Tabla D15

Diccionario de recibo

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
pago_id	UUID	No	FK	Ninguno	→ pago.id; RESTRICT	Pago único documentado.	RN-07
numero	BIGINT	No	No	Ninguno	numero > 0	Número comercial contiguo, visible con al menos seis dígitos.	RN-07
instantanea	JSONB	No	No	Ninguno	jsonb_typeof(instantanea)='object'	Contenido administrativo de emisión sin credenciales.	RN-08
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D16 presenta diccionario de solicitud_mantenimiento.

Tabla D16

Diccionario de solicitud_mantenimiento

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
unidad_id	UUID	No	FK	Ninguno	→ unidad.id; RESTRICT	Unidad donde ocurre la solicitud.	RN-23
descripcion	VARCHAR (1000)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Descripción de la incidencia.	RN-16
prioridad	VARCHAR (24)	No	No	'Media'	prioridad IN ('Baja','Media','Alta')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-27
estado	VARCHAR (24)	No	No	'Abierta'	estado IN ('Abierta','En proceso','Resuelta','Cancelada')	Valor exacto del catálogo. Los estados derivados se calculan en vistas.	RN-27
responsable	VARCHAR (120)	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Persona o empresa designada, dato opcional.	RN-18
abierta_el	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha de apertura.	RN-16

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
cerrada_el	DATE	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha de resolución o cancelación.	RN-27
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

La Tabla D17 presenta diccionario de gasto_propiedad.

Tabla D17

Diccionario de gasto_propiedad

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
propiedad_id	UUID	No	FK	Ninguno	→ propiedad.id; RESTRICT	Propiedad que soporta el gasto.	RN-23
solicitud_id	UUID	Sí	FK	Ninguno	→ solicitud_mantenimiento.id; RESTRICT	Solicitud relacionada, opcional.	RN-23

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
fecha_gasto	DATE	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Fecha de egreso.	RN-16
importe	NUMERIC(19,2)	No	No	Ninguno	importe > 0	Valor del egreso.	RN-03
moneda	VARCHAR(3)	No	No	Ninguno	moneda IN ('NIO','USD')	Moneda del importe.	RN-03
concepto	VARCHAR(300)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Explicación del gasto.	RN-23
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D18 presenta diccionario de parametro_sistema.

Tabla D18

Diccionario de parametro_sistema

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
clave	VARCHAR (60)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Clave estable del parámetro.	RN-16
valor	JSONB	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Valor tipado y validado por la capa de servicios.	RN-13
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
vigente_desde	TIMESTAMPTZ	No	UNIQUE compuesta	CURRENT_TIMESTAMP	UNIQUE(clave,vigente_desde)	Inicio de vigencia; vigente = última fila por clave con fecha ≤ now().	RN-13
motivo	VARCHAR (500)	No	No	Ninguno	Límite de 500 caracteres	Motivo del nuevo registro, conservado sin actualización.	RN-13 / RN-14

La Tabla D19 presenta diccionario de bitacora.

Tabla D19

Diccionario de bitacora

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
correlacion_id	UUID	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Identifica el caso de uso completo.	RN-14
modulo	VARCHAR (3)	No	No	Ninguno	modulo ~ '^M(0[1-9]|1[0-2])$'	Código M01 a M12.	RN-14
accion	VARCHAR (80)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Acción realizada, sin secretos.	RN-14
entidad	VARCHAR (60)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Tabla o agregado afectado.	RN-14
entidad_id	UUID	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Identificador del objeto, si existe.	RN-14
detalle	JSONB	No	No	'{}'::jsonb	Tipo SQL; restricciones en Tabla D24.	Cambios financieros y motivo; excluye identificación innecesaria.	RN-14

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

La Tabla D20 presenta diccionario de documento_adjunto.

Tabla D20

Diccionario de documento_adjunto

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
propiedad_id	UUID	Sí	FK	Ninguno	→ propiedad.id; RESTRICT	Documento de propiedad; el logotipo corresponde al registro emisor mediante parametro_id.	RN-25
unidad_id	UUID	Sí	FK	Ninguno	→ unidad.id; RESTRICT	Fotografía de unidad.	RN-25
inquilino_id	UUID	Sí	FK	Ninguno	→ inquilino.id; RESTRICT	Documento del inquilino.	RN-25
contrato_id	UUID	Sí	FK	Ninguno	→ contrato.id; RESTRICT	Documento contractual.	RN-25

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
nombre_original	VARCHAR (180)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre visible sin usarlo como ruta.	RN-25
ruta_interna	VARCHAR (80)	No	No	Ninguno	ruta_interna ~ '^[0-9a-f-]{36}[.](pdf|png|jpg)$'	UUID y extensión relativa, sin recorridos de directorio.	RN-25
mime	VARCHAR (30)	No	No	Ninguno	mime IN ('application/pdf','image/png','image/jpeg')	Tipo contrastado con firma.	RN-25
tamano_bytes	BIGINT	No	No	Ninguno	tamano_bytes BETWEEN 1 AND 5242880	Tamaño de 1 a 5242880 bytes.	RN-25
sha256	VARCHAR (64)	No	No	Ninguno	sha256 ~ '^[0-9a-f]{64}$'	Huella del archivo publicado.	RN-25
activo	BOOLEAN	No	No	TRUE	Tipo SQL; restricciones en Tabla D24.	Baja lógica del metadato.	RN-11
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14
version	BIGINT	No	No	0	version >= 0	Versión optimista; cada edición suma uno.	RN-21

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
parametro_id	UUID	Sí	FK	Ninguno	→ parametro_sistema.id; RESTRICT; ck_documento_dueno cuenta cinco columnas	Logotipo asociado al UUID de la versión del parámetro emisor.	RN-25

La Tabla D21 presenta diccionario de puerta_mantenimiento.

Tabla D21

Diccionario de puerta_mantenimiento

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
descripcion	VARCHAR(80)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Identifica la puerta de mantenimiento.	RN-16

La Tabla D22 presenta diccionario de control_fx.

Tabla D22

Diccionario de control_fx

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
instancia_id	UUID	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Proceso designado que posee la concesión.	RN-22
concesion_hasta	TIMESTAMPTZ	Sí	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Expiración a 90 segundos.	RN-22
proximo_intento	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	No se consulta antes de este instante.	RN-22
mes	DATE	No	No	Ninguno	EXTRACT(DAY FROM mes)=1	Primer día del mes local del presupuesto.	RN-22
intentos_mes	INTEGER	No	No	0	intentos_mes BETWEEN 0 AND 100	Incluye errores y reintentos, límite interno 100.	RN-22

La Tabla D23 presenta diccionario de esquema_migracion.

Tabla D23

Diccionario de esquema_migracion

Columna	Tipo	Nulo	Clave	Valor por Defecto	Restricciones	Descripción	Regla de Negocio
id	UUID	No	PK	gen_random_uuid()	Tipo SQL; restricciones en Tabla D24.	Identificador técnico opaco; no es número de recibo.	RN-16
nombre	VARCHAR(80)	No	No	Ninguno	Tipo SQL; restricciones en Tabla D24.	Nombre Vnnn del script aplicado.	RN-16
sha256	VARCHAR(64)	No	No	Ninguno	sha256 ~ '^[0-9a-f]{64}$'	Huella del script aplicado.	RN-16
creado_en	TIMESTAMPTZ	No	No	CURRENT_TIMESTAMP	Tipo SQL; restricciones en Tabla D24.	Instante de creación; presentación en America/Managua.	RN-14
creado_por	UUID	Sí	FK	Ninguno	→ usuario.id; RESTRICT	Usuario que creó el registro.	RN-14

Catálogo Íntegro de Restricciones

La Tabla D24 presenta claves y Restricciones Declaradas en V001.

Tabla D24

Claves y Restricciones Declaradas en V001

Nombre	Tipo	Tabla	Expresión	Regla
ck_rol_nombre	CHECK	rol	nombre IN ('Propietario','Asistente')	RN-13
pk_rol	PK	rol	id	RN-13
uq_rol_nombre	UNIQUE	rol	UNIQUE(nombre)	RN-13
ck_usuario_nombre_usuario	CHECK	usuario	nombre_usuario ~ '^[A-Za-z0-9_.]{3,40}$'	RN-16
ck_usuario_hash_clave	CHECK	usuario	hash_clave LIKE '$argon2id$%'	RN-15
ck_usuario_intentos_fallidos	CHECK	usuario	intentos_fallidos >= 0	RN-15
ck_usuario_version	CHECK	usuario	version >= 0	RN-21
pk_usuario	PK	usuario	id	RN-15
ck_propiedad_version	CHECK	propiedad	version >= 0	RN-21
pk_propiedad	PK	propiedad	id	RN-11
ck_unidad_tipo	CHECK	unidad	tipo IN ('Casa','Apartamento','Cuarto','Local comercial')	RN-16
ck_unidad_estado	CHECK	unidad	estado IN ('Disponible','En mantenimiento','Inactiva')	RN-11
ck_unidad_canon_referencia	CHECK	unidad	canon_referencia > 0	RN-03
ck_unidad_moneda	CHECK	unidad	moneda IN ('NIO','USD')	RN-03

Nombre	Tipo	Tabla	Expresión	Regla
ck_unidad_version	CHECK	unidad	version >= 0	RN-21
pk_unidad	PK	unidad	id	RN-01
uq_unidad_codigo	UNIQUE	unidad	UNIQUE(propiedad_id,codigo)	RN-16
ck_inquilino_version	CHECK	inquilino	version >= 0	RN-21
pk_inquilino	PK	inquilino	id	RN-18
ck_contrato_canon	CHECK	contrato	canon > 0	RN-03
ck_contrato_moneda	CHECK	contrato	moneda IN ('NIO','USD')	RN-03
ck_contrato_dia_pago	CHECK	contrato	dia_pago BETWEEN 1 AND 31	RN-17
ck_contrato_deposito_pactado	CHECK	contrato	deposito_pactado >= 0	RN-10
ck_contrato_estado	CHECK	contrato	estado IN ('Borrador','Vigente','Por vencer','Vencido','Terminado','Cancelado')	RN-19
ck_contrato_version	CHECK	contrato	version >= 0	RN-21
pk_contrato	PK	contrato	id	RN-01
ck_contrato_duracion	CHECK	contrato	fin >= (inicio + INTERVAL '1 month')::date	RN-02
ck_contrato_cierre	CHECK	contrato	(estado = 'Terminado') = (terminado_el IS NOT NULL)	RN-19
ck_cargo_tipo	CHECK	cargo	tipo IN ('Renta','Mora','Reparación','Servicio','Otro')	RN-16
ck_cargo_periodo	CHECK	cargo	EXTRACT(DAY FROM periodo)=1	RN-04
ck_cargo_importe	CHECK	cargo	importe > 0	RN-03
ck_cargo_version	CHECK	cargo	version >= 0	RN-21
pk_cargo	PK	cargo	id	RN-04

Nombre	Tipo	Tabla	Expresión	Regla
ck_cargo_anulacion	CHECK	cargo	(anulado_en IS NULL AND anulado_por IS NULL AND motivo_anulacion IS NULL) OR (anulado_en IS NOT NULL AND anulado_por IS NOT NULL AND length(trim(motivo_anulacion))>0)	RN-13
ck_cargo_mora	CHECK	cargo	tipo <> 'Mora' OR cargo_origen_id IS NOT NULL	RN-05
ck_tipo_cambio_base	CHECK	tipo_cambio	base='USD'	RN-22
ck_tipo_cambio_destino	CHECK	tipo_cambio	destino='NIO'	RN-22
ck_tipo_cambio_valor	CHECK	tipo_cambio	valor > 0	RN-22
ck_tipo_cambio_huella_respuesta	CHECK	tipo_cambio	huella_respuesta ~ '^[0-9a-f]{64}$'	RN-22
pk_tipo_cambio	PK	tipo_cambio	id	RN-22
ck_tipo_cambio_instantes	CHECK	tipo_cambio	proximo_en > actualizado_en	RN-22
ck_autorizacion_tasa_valor	CHECK	autorizacion_tasa	valor > 0	RN-32
pk_autorizacion_tasa	PK	autorizacion_tasa	id	RN-09
uq_autorizacion_fecha	UNIQUE	autorizacion_tasa	UNIQUE(fecha_efectiva)	RN-22
ck_autorizacion_actor	CHECK	autorizacion_tasa	creado_por IS NOT NULL	RN-14
ck_pago_solicitud_hash	CHECK	pago	solicitud_hash ~ '^[0-9a-f]{64}$'	RN-21
ck_pago_moneda_original	CHECK	pago	moneda_original IN ('NIO','USD')	RN-03
ck_pago_monto_original	CHECK	pago	monto_original > 0	RN-06
ck_pago_monto_contrato	CHECK	pago	monto_contrato > 0	RN-32
ck_pago_tasa_usada	CHECK	pago	tasa_usada > 0	RN-09
ck_pago_metodo	CHECK	pago	metodo IN ('Efectivo','Transferencia','Depósito bancario','Otro')	RN-16

Nombre	Tipo	Tabla	Expresión	Regla
ck_pago_estado	CHECK	pago	estado IN ('Registrado','Anulado')	RN-08
pk_pago	PK	pago	id	RN-06
uq_pago_solicitud	UNIQUE	pago	UNIQUE(solicitud_id)	RN-21
ck_pago_tasa	CHECK	pago	(tasa_usada IS NULL) = (autorizacion_id IS NULL)	RN-09
ck_pago_anulacion	CHECK	pago	(estado='Registrado' AND anulado_en IS NULL AND anulado_por IS NULL AND motivo_anulacion IS NULL) OR (estado='Anulado' AND anulado_en IS NOT NULL AND anulado_por IS NOT NULL AND length(trim(motivo_anulacion))>0)	RN-08
ck_deposito_garantia_importe_inicial	CHECK	deposito_garantia	importe_inicial >= 0	RN-10
pk_deposito_garantia	PK	deposito_garantia	id	RN-10
uq_deposito_contrato	UNIQUE	deposito_garantia	UNIQUE(contrato_id)	RN-10
ck_movimiento_deposito_tipo	CHECK	movimiento_deposito	tipo IN ('Aplicacion','Devolucion')	RN-26
ck_movimiento_deposito_importe	CHECK	movimiento_deposito	importe > 0	RN-26
ck_movimiento_deposito_metodo	CHECK	movimiento_deposito	metodo IN ('Efectivo','Transferencia','Depósito bancario','Otro')	RN-10
pk_movimiento_deposito	PK	movimiento_deposito	id	RN-26
uq_movimiento_liquidacion_orden	UNIQUE	movimiento_deposito	UNIQUE(liquidacion_id,orden)	RN-21

Nombre	Tipo	Tabla	Expresión	Regla
ck_movimiento_liquidacion_hash	CHECK	movimiento_deposito	liquidacion_hash ~ '^[0-9a-f]{64}$'	RN-21
ck_movimiento_destino	CHECK	movimiento_deposito	(tipo='Aplicacion' AND cargo_id IS NOT NULL AND metodo IS NULL) OR (tipo='Devolucion' AND cargo_id IS NULL AND metodo IS NOT NULL)	RN-10
ck_aplicacion_pago_importe	CHECK	aplicacion_pago	importe > 0	RN-06
ck_aplicacion_pago_origen	CHECK	aplicacion_pago	origen IN ('Inicial','Credito','Garantia')	RN-31
pk_aplicacion_pago	PK	aplicacion_pago	id	RN-06
ck_aplicacion_fuente	CHECK	aplicacion_pago	(pago_id IS NOT NULL AND movimiento_id IS NULL AND origen IN ('Inicial','Credito')) OR (pago_id IS NULL AND movimiento_id IS NOT NULL AND origen='Garantia')	RN-06
uq_aplicacion_movimiento	UNIQUE	aplicacion_pago	UNIQUE(movimiento_id)	RN-10
ck_aplicacion_reversion	CHECK	aplicacion_pago	(revertida_en IS NULL AND revertida_por IS NULL AND motivo_reversion IS NULL) OR (revertida_en IS NOT NULL AND revertida_por IS NOT NULL AND length(trim(motivo_reversion))>0)	RN-31
ck_consecutivo_recibo_siguiente_numero	CHECK	consecutivo_recibo	siguiente_numero > 0	RN-07
pk_consecutivo_recibo	PK	consecutivo_recibo	id	RN-07
ck_consecutivo_unico	CHECK	consecutivo_recibo	id='00000000-0000-0000-0000-000000000001'::uuid	RN-07
ck_recibo_numero	CHECK	recibo	numero > 0	RN-07

Nombre	Tipo	Tabla	Expresión	Regla
ck_recibo_instantanea	CHECK	recibo	jsonb_typeof(instantanea)='object'	RN-08
pk_recibo	PK	recibo	id	RN-08
uq_recibo_pago	UNIQUE	recibo	UNIQUE(pago_id)	RN-07
uq_recibo_numero	UNIQUE	recibo	UNIQUE(numero)	RN-07
ck_solicitud_mantenimiento_prioridad	CHECK	solicitud_mantenimiento	prioridad IN ('Baja','Media','Alta')	RN-27
ck_solicitud_mantenimiento_estado	CHECK	solicitud_mantenimiento	estado IN ('Abierta','En proceso','Resuelta','Cancelada')	RN-27
ck_solicitud_mantenimiento_version	CHECK	solicitud_mantenimiento	version >= 0	RN-21
pk_solicitud_mantenimiento	PK	solicitud_mantenimiento	id	RN-27
ck_mantenimiento_cierre	CHECK	solicitud_mantenimiento	(estado IN ('Resuelta','Cancelada')) = (cerrada_el IS NOT NULL)	RN-27
ck_gasto_propiedad_importe	CHECK	gasto_propiedad	importe > 0	RN-03
ck_gasto_propiedad_moneda	CHECK	gasto_propiedad	moneda IN ('NIO','USD')	RN-03
pk_gasto_propiedad	PK	gasto_propiedad	id	RN-23
pk_parametro_sistema	PK	parametro_sistema	id	RN-13
uq_parametro_clave	UNIQUE	parametro_sistema	UNIQUE(clave,vigente_desde)	RN-13
ck_bitacora_modulo	CHECK	bitacora	modulo ~ '^M(0[1-9]|1[0-2])$'	RN-14
pk_bitacora	PK	bitacora	id	RN-14

Nombre	Tipo	Tabla	Expresión	Regla
ck_documento_adjunto_ruta_interna	CHECK	documento_adjunto	ruta_interna ~ '^[0-9a-f-]{36}[.](pdf|png|jpg)$'	RN-25
ck_documento_adjunto_mime	CHECK	documento_adjunto	mime IN ('application/pdf','image/png','image/jpeg')	RN-25
ck_documento_adjunto_tamano_bytes	CHECK	documento_adjunto	tamano_bytes BETWEEN 1 AND 5242880	RN-25
ck_documento_adjunto_sha256	CHECK	documento_adjunto	sha256 ~ '^[0-9a-f]{64}$'	RN-25
ck_documento_adjunto_version	CHECK	documento_adjunto	version >= 0	RN-21
pk_documento_adjunto	PK	documento_adjunto	id	RN-25
ck_documento_dueno	CHECK	documento_adjunto	num_nonnulls(propiedad_id,unidad_id,inquilino_id,contrato_id,parametro_id)=1	RN-25
uq_documento_ruta	UNIQUE	documento_adjunto	UNIQUE(ruta_interna)	RN-25
pk_puerta_mantenimiento	PK	puerta_mantenimiento	id	RN-29
ck_puerta_unica	CHECK	puerta_mantenimiento	id='00000000-0000-0000-0000-000000000002'::uuid	RN-29
ck_control_fx_mes	CHECK	control_fx	EXTRACT(DAY FROM mes)=1	RN-22
ck_control_fx_intentos_mes	CHECK	control_fx	intentos_mes BETWEEN 0 AND 100	RN-22
pk_control_fx	PK	control_fx	id	RN-22
ck_control_fx_unico	CHECK	control_fx	id='00000000-0000-0000-0000-000000000003'::uuid	RN-22

Nombre	Tipo	Tabla	Expresión	Regla
ck_esquema_migracion_sha256	CHECK	esquema_migracion	sha256 ~ '^[0-9a-f]{64}$'	RN-16
pk_esquema_migracion	PK	esquema_migracion	id	RN-29
uq_migracion_nombre	UNIQUE	esquema_migracion	UNIQUE(nombre)	RN-29
fk_usuario_rol_id	FK	usuario	FOREIGN KEY(rol_id) REFERENCES rol(id) ON DELETE RESTRICT	RN-13
fk_usuario_creado_por	FK	usuario	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_propiedad_creado_por	FK	propiedad	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_unidad_propiedad_id	FK	unidad	FOREIGN KEY(propiedad_id) REFERENCES propiedad(id) ON DELETE RESTRICT	RN-11
fk_unidad_creado_por	FK	unidad	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_inquilino_creado_por	FK	inquilino	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_contrato_unidad_id	FK	contrato	FOREIGN KEY(unidad_id) REFERENCES unidad(id) ON DELETE RESTRICT	RN-01
fk_contrato_inquilino_id	FK	contrato	FOREIGN KEY(inquilino_id) REFERENCES inquilino(id) ON DELETE RESTRICT	RN-18
fk_contrato_predecesor_id	FK	contrato	FOREIGN KEY(predecesor_id) REFERENCES contrato(id) ON DELETE RESTRICT	RN-19
fk_contrato_creado_por	FK	contrato	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14

Nombre	Tipo	Tabla	Expresión	Regla
fk_cargo_contrato_id	FK	cargo	FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT	RN-04
fk_cargo_cargo_origen_id	FK	cargo	FOREIGN KEY(cargo_origen_id) REFERENCES cargo(id) ON DELETE RESTRICT	RN-05
fk_cargo_anulado_por	FK	cargo	FOREIGN KEY(anulado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-13
fk_cargo_creado_por	FK	cargo	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_tipo_cambio_creado_por	FK	tipo_cambio	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_autorizacion_tasa_referencia_id	FK	autorizacion_tasa	FOREIGN KEY(referencia_id) REFERENCES tipo_cambio(id) ON DELETE RESTRICT	RN-22
fk_autorizacion_tasa_creado_por	FK	autorizacion_tasa	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_pago_contrato_id	FK	pago	FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT	RN-06
fk_pago_autorizacion_id	FK	pago	FOREIGN KEY(autorizacion_id) REFERENCES autorizacion_tasa(id) ON DELETE RESTRICT	RN-22
fk_pago_anulado_por	FK	pago	FOREIGN KEY(anulado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-08
fk_pago_creado_por	FK	pago	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_deposito_garantia_contrato_id	FK	deposito_garantia	FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT	RN-10
fk_deposito_garantia_creado_por	FK	deposito_garantia	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14

Nombre	Tipo	Tabla	Expresión	Regla
fk_movimiento_deposito_deposito_id	FK	movimiento_deposito	FOREIGN KEY(deposito_id) REFERENCES deposito_garantia(id) ON DELETE RESTRICT	RN-26
fk_movimiento_deposito_cargo_id	FK	movimiento_deposito	FOREIGN KEY(cargo_id) REFERENCES cargo(id) ON DELETE RESTRICT	RN-10
fk_movimiento_deposito_creado_por	FK	movimiento_deposito	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_aplicacion_pago_pago_id	FK	aplicacion_pago	FOREIGN KEY(pago_id) REFERENCES pago(id) ON DELETE RESTRICT	RN-06
fk_aplicacion_pago_movimiento_id	FK	aplicacion_pago	FOREIGN KEY(movimiento_id) REFERENCES movimiento_deposito(id) ON DELETE RESTRICT	RN-10
fk_aplicacion_pago_cargo_id	FK	aplicacion_pago	FOREIGN KEY(cargo_id) REFERENCES cargo(id) ON DELETE RESTRICT	RN-06
fk_aplicacion_pago_revertida_por	FK	aplicacion_pago	FOREIGN KEY(revertida_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-31
fk_aplicacion_pago_creado_por	FK	aplicacion_pago	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_recibo_pago_id	FK	recibo	FOREIGN KEY(pago_id) REFERENCES pago(id) ON DELETE RESTRICT	RN-07
fk_recibo_creado_por	FK	recibo	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_solicitud_mantenimiento_unidad_id	FK	solicitud_mantenimiento	FOREIGN KEY(unidad_id) REFERENCES unidad(id) ON DELETE RESTRICT	RN-23
fk_solicitud_mantenimiento_creado_por	FK	solicitud_mantenimiento	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14

Nombre	Tipo	Tabla	Expresión	Regla
fk_gasto_propiedad_propiedad_id	FK	gasto_propiedad	FOREIGN KEY(propiedad_id) REFERENCES propiedad(id) ON DELETE RESTRICT	RN-23
fk_gasto_propiedad_solicitud_id	FK	gasto_propiedad	FOREIGN KEY(solicitud_id) REFERENCES solicitud_mantenimiento(id) ON DELETE RESTRICT	RN-23
fk_gasto_propiedad_creado_por	FK	gasto_propiedad	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_parametro_sistema_creado_por	FK	parametro_sistema	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_bitacora_creado_por	FK	bitacora	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_documento_adjunto_propiedad_id	FK	documento_adjunto	FOREIGN KEY(propiedad_id) REFERENCES propiedad(id) ON DELETE RESTRICT	RN-25
fk_documento_adjunto_unidad_id	FK	documento_adjunto	FOREIGN KEY(unidad_id) REFERENCES unidad(id) ON DELETE RESTRICT	RN-25
fk_documento_adjunto_inquilino_id	FK	documento_adjunto	FOREIGN KEY(inquilino_id) REFERENCES inquilino(id) ON DELETE RESTRICT	RN-25
fk_documento_adjunto_contrato_id	FK	documento_adjunto	FOREIGN KEY(contrato_id) REFERENCES contrato(id) ON DELETE RESTRICT	RN-25
fk_documento_adjunto_creado_por	FK	documento_adjunto	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
fk_esquema_migracion_creado_por	FK	esquema_migracion	FOREIGN KEY(creado_por) REFERENCES usuario(id) ON DELETE RESTRICT	RN-14
uq_usuario_nombre	Índice único	usuario	lower(nombre_usuario)	RN-16
uq_contrato_activo	Índice único parcial	contrato	unidad_id WHERE estado IN ('Vigente','Por vencer','Vencido')	RN-01

Nombre	Tipo	Tabla	Expresión	Regla
uq_cargo_renta	Índice único parcial	cargo	contrato_id,periodo WHERE tipo='Renta'	RN-04
uq_cargo_mora	Índice único parcial	cargo	cargo_origen_id WHERE tipo='Mora'	RN-05
fk_documento_adjunto_parametro_id	FK	documento_adjunto	FOREIGN KEY(parametro_id) REFERENCES parametro_sistema(id) ON DELETE RESTRICT	RN-25

Glosario del Modelo

La Tabla D25 presenta términos Compartidos del Dominio.

Tabla D25

Términos Compartidos del Dominio

Término	Definición de Uso en Latch
Unidad	Espacio alquilable de tipo Casa, Apartamento, Cuarto o Local comercial, identificado dentro de una propiedad.
Canon	Monto positivo de renta pactado en NIO o USD en el contrato, independiente del canon de referencia de unidad.
Cargo	Obligación de pago con concepto, importe y fecha de vencimiento en la moneda del contrato.
Mora	Cargo por pago tardío calculado después de la gracia sobre principal pendiente, sin mora sobre mora.
Depósito de garantía	Importe retenido al activar el contrato, separado de renta y sujeto a aplicación o devolución con concepto.
Recibo	Documento administrativo de un pago confirmado con número comercial único y conservado aun al anular.
Estado de cuenta	Reporte de obligaciones, aplicaciones, pagos y saldos de un contrato con fecha de corte y moneda.
Tasa de cambio	Cantidad de NIO por un USD; se distingue referencia informativa de autorización de cobro por fecha.
Período de facturación	Mes del cargo, representado por su primer día; visualización AAAA-MM, sin prorrateo automático.
Crédito o saldo a favor	Remanente de un pago registrado aún no aplicado, en moneda del mismo contrato y trazable al pago de origen.

Apéndice E

Ejemplos de Entidades JPA

Los ejemplos usan acceso por campos, UUID preasignado, relaciones LAZY y columnas exactas del esquema. Contrato incluye @Version y convierte EstadoContrato mediante una etiqueta estable; las entidades históricas se marcan @Immutable para impedir cambios ordinarios de contenido. La anulación de Pago y la reversión de aplicaciones se ejecutan mediante el adaptador de columnas permitidas, bajo bloqueo de contrato, y la sesión se limpia o refresca después de esa escritura. El paquete adjunto incluye los mapeos de todas las tablas como referencia del diccionario. La operación real conserva la distribución JDBC/Hibernate descrita en el cuerpo, aunque exista una clase ilustrativa para una tabla de administración.

Contrato.java

```
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.UUID;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name="contrato", schema="latch")
public class Contrato {
  @Id
  @Column(nullable=false, updatable=false)
  private UUID id = UUID.randomUUID();
  @ManyToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="unidad_id", nullable=false)
  private Unidad unidad;
  @ManyToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="inquilino_id", nullable=false)
  private Inquilino inquilino;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="predecesor_id", nullable=true)
  private Contrato predecesor;
  @Column(name="inicio", nullable=false)
  private LocalDate inicio;
  @Column(name="fin", nullable=false)
  private LocalDate fin;
  @Column(name="terminado_el", nullable=true)
  private LocalDate terminadoEl;
```

```
  @Column(name="canon", nullable=false, precision=19, scale=2)
  private BigDecimal canon;
  @Column(name="moneda", nullable=false, length=3)
  @Convert(converter=MonedaConverter.class)
  private Moneda moneda;
  @Column(name="dia_pago", nullable=false)
  private Short diaPago;
  @Column(name="deposito_pactado", nullable=false, precision=19, scale=2)
  private BigDecimal depositoPactado;
  @Convert(converter=EstadoContratoConverter.class)
  @Column(name="estado", nullable=false, length=24)
  private EstadoContrato estado;
  @Column(name="creado_en", nullable=false, insertable=false,
updatable=false)
  private Instant creadoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="creado_por", nullable=true)
  private Usuario creadoPor;
  @Version
  @Column(name="version", nullable=false)
  private Long version;

  protected Contrato() {}
  public UUID getId() { return id; }
  @Override public final int hashCode() { return id.hashCode(); }
  @Override public final boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || org.hibernate.Hibernate.getClass(this)
        != org.hibernate.Hibernate.getClass(other)) return false;
    return getId().equals(((Contrato) other).getId());
  }
}
```

EstadoContrato.java

```
package ni.edu.uam.latch.dominio;
public enum EstadoContrato {
 BORRADOR("Borrador"), VIGENTE("Vigente"), POR_VENCER("Por vencer"),
 VENCIDO("Vencido"), TERMINADO("Terminado"), CANCELADO("Cancelado");
 public final String etiqueta;
 EstadoContrato(String etiqueta) { this.etiqueta=etiqueta; }
}
```

EstadoContratoConverter.java

```
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter
public class EstadoContratoConverter
 implements AttributeConverter<EstadoContrato,String> {
 public String convertToDatabaseColumn(EstadoContrato valor) {
  return valor==null ? null : valor.etiqueta;
 }
 public EstadoContrato convertToEntityAttribute(String valor) {
  if(valor==null) return null;
  for(var estado:EstadoContrato.values())
   if(estado.etiqueta.equals(valor)) return estado;
```

```
  throw new IllegalArgumentException("Estado contractual inválido");
 }
}
```

Cargo.java

```
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.UUID;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name="cargo", schema="latch")
public class Cargo {
  @Id
  @Column(nullable=false, updatable=false)
  private UUID id = UUID.randomUUID();
  @ManyToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="contrato_id", nullable=false)
  private Contrato contrato;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="cargo_origen_id", nullable=true)
  private Cargo cargoOrigen;
  @Column(name="tipo", nullable=false, length=24)
  @Convert(converter=TipoCargoConverter.class)
  private TipoCargo tipo;
  @Column(name="periodo", nullable=false)
  private LocalDate periodo;
  @Column(name="fecha_emision", nullable=false)
  private LocalDate fechaEmision;
  @Column(name="vence_el", nullable=false)
  private LocalDate venceEl;
  @Column(name="importe", nullable=false, precision=19, scale=2)
  private BigDecimal importe;
  @Column(name="concepto", nullable=false, length=300)
  private String concepto;
  @Column(name="anulado_en", nullable=true)
  private Instant anuladoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="anulado_por", nullable=true)
  private Usuario anuladoPor;
  @Column(name="motivo_anulacion", nullable=true, length=500)
  private String motivoAnulacion;
  @Column(name="creado_en", nullable=false, insertable=false,
updatable=false)
  private Instant creadoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="creado_por", nullable=true)
  private Usuario creadoPor;
  @Version
  @Column(name="version", nullable=false)
  private Long version;
```

```
  protected Cargo() {}
  public UUID getId() { return id; }
  @Override public final int hashCode() { return id.hashCode(); }
  @Override public final boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || org.hibernate.Hibernate.getClass(this)
        != org.hibernate.Hibernate.getClass(other)) return false;
    return getId().equals(((Cargo) other).getId());
  }
}
```

Pago.java

```
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.UUID;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name="pago", schema="latch")
@org.hibernate.annotations.Immutable
public class Pago {
  @Id
  @Column(nullable=false, updatable=false)
  private UUID id = UUID.randomUUID();
  @ManyToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="contrato_id", nullable=false)
  private Contrato contrato;
  @Column(name="solicitud_id", nullable=false)
  private UUID solicitudId;
  @Column(name="solicitud_hash", nullable=false, length=64)
  private String solicitudHash;
  @Column(name="fecha_pago", nullable=false)
  private LocalDate fechaPago;
  @Column(name="moneda_original", nullable=false, length=3)
  @Convert(converter=MonedaConverter.class)
  private Moneda monedaOriginal;
  @Column(name="monto_original", nullable=false, precision=19, scale=2)
  private BigDecimal montoOriginal;
  @Column(name="monto_contrato", nullable=false, precision=19, scale=2)
  private BigDecimal montoContrato;
  @Column(name="tasa_usada", nullable=true, precision=19, scale=8)
  private BigDecimal tasaUsada;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="autorizacion_id", nullable=true)
  private AutorizacionTasa autorizacion;
  @Column(name="metodo", nullable=false, length=24)
  @Convert(converter=MetodoPagoConverter.class)
  private MetodoPago metodo;
  @Column(name="estado", nullable=false, length=24)
  @Convert(converter=EstadoPagoConverter.class)
  private EstadoPago estado;
  @Column(name="anulado_en", nullable=true)
```

```
  private Instant anuladoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="anulado_por", nullable=true)
  private Usuario anuladoPor;
  @Column(name="motivo_anulacion", nullable=true, length=500)
  private String motivoAnulacion;
  @Column(name="creado_en", nullable=false, insertable=false,
updatable=false)
  private Instant creadoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="creado_por", nullable=true)
  private Usuario creadoPor;

  protected Pago() {}
  public UUID getId() { return id; }
  @Override public final int hashCode() { return id.hashCode(); }
  @Override public final boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || org.hibernate.Hibernate.getClass(this)
        != org.hibernate.Hibernate.getClass(other)) return false;
    return getId().equals(((Pago) other).getId());
  }
}
```

Recibo.java

```
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.UUID;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name="recibo", schema="latch")
@org.hibernate.annotations.Immutable
public class Recibo {
  @Id
  @Column(nullable=false, updatable=false)
  private UUID id = UUID.randomUUID();
  @OneToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="pago_id", nullable=false)
  private Pago pago;
  @Column(name="numero", nullable=false)
  private Long numero;
  @JdbcTypeCode(SqlTypes.JSON)
  @Column(name="instantanea", nullable=false, columnDefinition="jsonb")
  private String instantanea;
  @Column(name="creado_en", nullable=false, insertable=false,
updatable=false)
  private Instant creadoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="creado_por", nullable=true)
  private Usuario creadoPor;
```

```
  protected Recibo() {}
  public UUID getId() { return id; }
  @Override public final int hashCode() { return id.hashCode(); }
  @Override public final boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || org.hibernate.Hibernate.getClass(this)
        != org.hibernate.Hibernate.getClass(other)) return false;
    return getId().equals(((Recibo) other).getId());
  }
}
```

AplicacionPago.java

```
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.*;
import java.util.UUID;
import org.hibernate.annotations.JdbcTypeCode;
import org.hibernate.type.SqlTypes;

@Entity
@Table(name="aplicacion_pago", schema="latch")
@org.hibernate.annotations.Immutable
public class AplicacionPago {
  @Id
  @Column(nullable=false, updatable=false)
  private UUID id = UUID.randomUUID();
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="pago_id", nullable=true)
  private Pago pago;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="movimiento_id", nullable=true)
  private MovimientoDeposito movimiento;
  @ManyToOne(fetch=FetchType.LAZY, optional=false)
  @JoinColumn(name="cargo_id", nullable=false)
  private Cargo cargo;
  @Column(name="importe", nullable=false, precision=19, scale=2)
  private BigDecimal importe;
  @Column(name="origen", nullable=false, length=24)
  @Convert(converter=OrigenAplicacionConverter.class)
  private OrigenAplicacion origen;
  @Column(name="fecha_aplicacion", nullable=false)
  private LocalDate fechaAplicacion;
  @Column(name="revertida_en", nullable=true)
  private Instant revertidaEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="revertida_por", nullable=true)
  private Usuario revertidaPor;
  @Column(name="motivo_reversion", nullable=true, length=500)
  private String motivoReversion;
  @Column(name="creado_en", nullable=false, insertable=false,
updatable=false)
  private Instant creadoEn;
  @ManyToOne(fetch=FetchType.LAZY, optional=true)
  @JoinColumn(name="creado_por", nullable=true)
```

```
  private Usuario creadoPor;

  protected AplicacionPago() {}
  public UUID getId() { return id; }
  @Override public final int hashCode() { return id.hashCode(); }
  @Override public final boolean equals(Object other) {
    if (this == other) return true;
    if (other == null || org.hibernate.Hibernate.getClass(this)
        != org.hibernate.Hibernate.getClass(other)) return false;
    return getId().equals(((AplicacionPago) other).getId());
  }
}
```

Consultas Nombradas en orm.xml

El archivo de consultas se ubica como META-INF/orm.xml en los recursos del proyecto Java y se registra una vez en la unidad de persistencia. Sus dieciocho nombres corresponden a la tabla de consultas del cuerpo y usan parámetros enlazados por tipo. Las consultas que devuelven entidades no cruzan la frontera de presentación sin materializar el DTO necesario. El contrato de bloqueo se establece en el repositorio mediante LockModeType.PESSIMISTIC_WRITE cuando el servicio requiere modificar el agregado, no dentro de la cadena JPQL. El paquete conserva este archivo completo y no atribuye una prueba de compilación ORM a las verificaciones ejecutadas únicamente sobre PostgreSQL.

```
<?xml version='1.0' encoding='utf-8'?>
<entity-mappings xmlns="https://jakarta.ee/xml/ns/persistence/orm"
version="3.2">
  <named-query name="Propiedad.activas">
    <query>select p from Propiedad p where p.activa=true order by
p.nombre,p.id</query>
  </named-query>
  <named-query name="Unidad.porPropiedad">
    <query>select u from Unidad u where u.propiedad.id=:propiedad order by
u.codigo,u.id</query>
  </named-query>
  <named-query name="Unidad.disponibles">
    <query>select u from Unidad u where u.estado=:disponible and not exists
(select c.id from Contrato c where c.unidad=u and c.estado in :activos) order
by u.codigo,u.id</query>
  </named-query>
  <named-query name="Inquilino.buscar">
    <query>select i from Inquilino i where i.activo=true and lower(i.nombre)
like :patron order by i.nombre,i.id</query>
  </named-query>
  <named-query name="Contrato.porInquilino">
    <query>select c from Contrato c where c.inquilino.id=:inquilino order by
c.inicio desc,c.id</query>
```

```
  </named-query>
  <named-query name="Contrato.activoUnidad">
    <query>select c from Contrato c where c.unidad.id=:unidad and c.estado in
:activos</query>
  </named-query>
  <named-query name="Contrato.porVencer">
    <query>select c from Contrato c where c.estado in :activos and c.fin
between :desde and :hasta order by c.fin,c.id</query>
  </named-query>
  <named-query name="Cargo.porPeriodo">
    <query>select c from Cargo c where c.contrato.id=:contrato and
c.periodo=:periodo and c.tipo=:tipo</query>
  </named-query>
  <named-query name="Cargo.ordenCobro">
    <query>select c from Cargo c where c.contrato.id=:contrato and
c.anuladoEn is null and c.venceEl&lt;:fecha order by c.venceEl,c.id</query>
  </named-query>
  <named-query name="Pago.porSolicitud">
    <query>select p from Pago p where p.solicitudId=:solicitud</query>
  </named-query>
  <named-query name="Pago.historial">
    <query>select p from Pago p where p.contrato.id=:contrato and p.fechaPago
between :desde and :hasta order by p.fechaPago,p.id</query>
  </named-query>
  <named-query name="Recibo.porNumero">
    <query>select r from Recibo r join fetch r.pago where
r.numero=:numero</query>
  </named-query>
  <named-query name="AplicacionPago.porPago">
    <query>select a from AplicacionPago a where a.pago.id=:pago and
a.revertidaEn is null order by a.fechaAplicacion,a.id</query>
  </named-query>
  <named-query name="DepositoGarantia.porContrato">
    <query>select d from DepositoGarantia d where
d.contrato.id=:contrato</query>
  </named-query>
  <named-query name="SolicitudMantenimiento.pendientes">
    <query>select s from SolicitudMantenimiento s where s.unidad.id=:unidad
and s.estado in :estados order by s.abiertaEl,s.id</query>
  </named-query>
  <named-query name="GastoPropiedad.porFecha">
    <query>select g from GastoPropiedad g where g.propiedad.id=:propiedad and
g.fechaGasto between :desde and :hasta order by g.fechaGasto,g.id</query>
  </named-query>
  <named-query name="DocumentoAdjunto.porContrato">
    <query>select d from DocumentoAdjunto d where d.contrato.id=:contrato and
d.activo=true order by d.nombreOriginal,d.id</query>
  </named-query>
  <named-query name="AutorizacionTasa.porFecha">
    <query>select t from AutorizacionTasa t where
t.fechaEfectiva=:fecha</query>
  </named-query>
</entity-mappings>
```

```
Usuario.java
package ni.edu.uam.latch.dominio;

import jakarta.persistence.*;
import java.util.UUID;
import org.hibernate.annotations.Immutable;

@Entity
@Table(name="usuario", schema="latch")
@Immutable
public class Usuario {
    @Id
    @Column(nullable=false, updatable=false)
    private UUID id;
    @Column(name="rol_id", nullable=false, updatable=false)
    private UUID rolId;
    @Column(name="nombre_usuario", nullable=false, updatable=false)
    private String nombreUsuario;
    @Column(name="nombre_visible", nullable=false, updatable=false)
    private String nombreVisible;
    @Column(nullable=false, updatable=false)
    private boolean activo;
    protected Usuario() {}
    public UUID getId() { return id; }
}
UnidadDto.java
package ni.edu.uam.latch.aplicacion;
import java.util.UUID;
import ni.edu.uam.latch.dominio.EstadoUnidadVisible;
import ni.edu.uam.latch.dominio.EstadoUnidadVisibleConverter;
public record UnidadDto(UUID id, EstadoUnidadVisible estado) {
    public static UnidadDto desdeFila(UUID id, String estadoCalculado) {
        var converter = new EstadoUnidadVisibleConverter();
        return new UnidadDto(id,
converter.convertToEntityAttribute(estadoCalculado));
    }
}

Moneda.java
package ni.edu.uam.latch.dominio;
public enum Moneda {
    NIO("NIO"), USD("USD");
    public final String etiqueta;
    Moneda(String etiqueta) { this.etiqueta = etiqueta; }
}
MonedaConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class MonedaConverter implements AttributeConverter<Moneda,String> {
    public String convertToDatabaseColumn(Moneda valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public Moneda convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : Moneda.values()) {
```

```
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de Moneda");
    }
}

TipoCargo.java
package ni.edu.uam.latch.dominio;
public enum TipoCargo {
    RENTA("Renta"), MORA("Mora"), REPARACION("Reparación"),
SERVICIO("Servicio"), OTRO("Otro");
    public final String etiqueta;
    TipoCargo(String etiqueta) { this.etiqueta = etiqueta; }
}
TipoCargoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class TipoCargoConverter implements
AttributeConverter<TipoCargo,String> {
    public String convertToDatabaseColumn(TipoCargo valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public TipoCargo convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : TipoCargo.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
TipoCargo");
    }
}

MetodoPago.java
package ni.edu.uam.latch.dominio;
public enum MetodoPago {
    EFECTIVO("Efectivo"), TRANSFERENCIA("Transferencia"),
DEPOSITO_BANCARIO("Depósito bancario"), OTRO("Otro");
    public final String etiqueta;
    MetodoPago(String etiqueta) { this.etiqueta = etiqueta; }
}
MetodoPagoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class MetodoPagoConverter implements
AttributeConverter<MetodoPago,String> {
    public String convertToDatabaseColumn(MetodoPago valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public MetodoPago convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : MetodoPago.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
```

```
        throw new IllegalArgumentException("Etiqueta desconocida de
MetodoPago");
    }
}

EstadoPago.java
package ni.edu.uam.latch.dominio;
public enum EstadoPago {
    REGISTRADO("Registrado"), ANULADO("Anulado");
    public final String etiqueta;
    EstadoPago(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoPagoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoPagoConverter implements
AttributeConverter<EstadoPago,String> {
    public String convertToDatabaseColumn(EstadoPago valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoPago convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoPago.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoPago");
    }
}

OrigenAplicacion.java
package ni.edu.uam.latch.dominio;
public enum OrigenAplicacion {
    INICIAL("Inicial"), CREDITO("Credito"), GARANTIA("Garantia");
    public final String etiqueta;
    OrigenAplicacion(String etiqueta) { this.etiqueta = etiqueta; }
}
OrigenAplicacionConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class OrigenAplicacionConverter implements
AttributeConverter<OrigenAplicacion,String> {
    public String convertToDatabaseColumn(OrigenAplicacion valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public OrigenAplicacion convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : OrigenAplicacion.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
OrigenAplicacion");
    }
```

```
}

TipoMovimientoDeposito.java
package ni.edu.uam.latch.dominio;
public enum TipoMovimientoDeposito {
    APLICACION("Aplicacion"), DEVOLUCION("Devolucion");
    public final String etiqueta;
    TipoMovimientoDeposito(String etiqueta) { this.etiqueta = etiqueta; }
}
TipoMovimientoDepositoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class TipoMovimientoDepositoConverter implements
AttributeConverter<TipoMovimientoDeposito,String> {
    public String convertToDatabaseColumn(TipoMovimientoDeposito valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public TipoMovimientoDeposito convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : TipoMovimientoDeposito.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
TipoMovimientoDeposito");
    }
}

EstadoUnidad.java
package ni.edu.uam.latch.dominio;
public enum EstadoUnidad {
    DISPONIBLE("Disponible"), EN_MANTENIMIENTO("En mantenimiento"),
INACTIVA("Inactiva");
    public final String etiqueta;
    EstadoUnidad(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoUnidadConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoUnidadConverter implements
AttributeConverter<EstadoUnidad,String> {
    public String convertToDatabaseColumn(EstadoUnidad valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoUnidad convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoUnidad.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoUnidad");
    }
}
```

```
EstadoUnidadVisible.java
package ni.edu.uam.latch.dominio;
public enum EstadoUnidadVisible {
    DISPONIBLE("Disponible"), OCUPADA("Ocupada"), EN_MANTENIMIENTO("En
mantenimiento"), INACTIVA("Inactiva");
    public final String etiqueta;
    EstadoUnidadVisible(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoUnidadVisibleConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoUnidadVisibleConverter implements
AttributeConverter<EstadoUnidadVisible,String> {
    public String convertToDatabaseColumn(EstadoUnidadVisible valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoUnidadVisible convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoUnidadVisible.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoUnidadVisible");
    }
}

EstadoMantenimiento.java
package ni.edu.uam.latch.dominio;
public enum EstadoMantenimiento {
    ABIERTA("Abierta"), EN_PROCESO("En proceso"), RESUELTA("Resuelta"),
CANCELADA("Cancelada");
    public final String etiqueta;
    EstadoMantenimiento(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoMantenimientoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoMantenimientoConverter implements
AttributeConverter<EstadoMantenimiento,String> {
    public String convertToDatabaseColumn(EstadoMantenimiento valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoMantenimiento convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoMantenimiento.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoMantenimiento");
    }
}

EstadoCargo.java
```

```
package ni.edu.uam.latch.dominio;
public enum EstadoCargo {
    PENDIENTE("Pendiente"), PARCIAL("Parcial"), PAGADO("Pagado"),
VENCIDO("Vencido"), ANULADO("Anulado");
    public final String etiqueta;
    EstadoCargo(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoCargoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoCargoConverter implements
AttributeConverter<EstadoCargo,String> {
    public String convertToDatabaseColumn(EstadoCargo valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoCargo convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoCargo.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoCargo");
    }
}

EstadoDeposito.java
package ni.edu.uam.latch.dominio;
public enum EstadoDeposito {
    RETENIDO("Retenido"), APLICADO("Aplicado"), DEVUELTO_PARCIAL("Devuelto
parcial"), DEVUELTO("Devuelto");
    public final String etiqueta;
    EstadoDeposito(String etiqueta) { this.etiqueta = etiqueta; }
}
EstadoDepositoConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class EstadoDepositoConverter implements
AttributeConverter<EstadoDeposito,String> {
    public String convertToDatabaseColumn(EstadoDeposito valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public EstadoDeposito convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : EstadoDeposito.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
EstadoDeposito");
    }
}

TipoUnidad.java
package ni.edu.uam.latch.dominio;
```

```
public enum TipoUnidad {
    CASA("Casa"), APARTAMENTO("Apartamento"), CUARTO("Cuarto"),
LOCAL_COMERCIAL("Local comercial");
    public final String etiqueta;
    TipoUnidad(String etiqueta) { this.etiqueta = etiqueta; }
}
TipoUnidadConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class TipoUnidadConverter implements
AttributeConverter<TipoUnidad,String> {
    public String convertToDatabaseColumn(TipoUnidad valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public TipoUnidad convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : TipoUnidad.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de
TipoUnidad");
    }
}

Rol.java
package ni.edu.uam.latch.dominio;
public enum Rol {
    PROPIETARIO("Propietario"), ASISTENTE("Asistente");
    public final String etiqueta;
    Rol(String etiqueta) { this.etiqueta = etiqueta; }
}
RolConverter.java
package ni.edu.uam.latch.dominio;
import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
@Converter(autoApply=true)
public class RolConverter implements AttributeConverter<Rol,String> {
    public String convertToDatabaseColumn(Rol valor) {
        return valor == null ? null : valor.etiqueta;
    }
    public Rol convertToEntityAttribute(String valor) {
        if (valor == null) return null;
        for (var elemento : Rol.values()) {
            if (elemento.etiqueta.equals(valor)) return elemento;
        }
        throw new IllegalArgumentException("Etiqueta desconocida de Rol");
    }
}
```

Apéndice F

Fuentes y Trazabilidad Documental

La tabla registra las fuentes abiertas durante la elaboración, su alcance y la razón para usarlas en una afirmación concreta. La fecha real de consulta fue el 23 de octubre de 2026 y no se hereda la fecha de consulta que aparece en los documentos hermanos. Los manuales dinámicos pueden mostrar un parche posterior al fijado por el proyecto, circunstancia que se declara cuando afecta la lectura. Las fuentes institucionales respaldan sus propios contratos y disposiciones; los criterios de diseño siguen siendo decisiones del equipo. Los documentos adjuntos aportan requisitos y no sustituyen la consulta de documentación oficial para versiones, privilegios o servicios externos.

La Tabla F1 presenta fuentes Consultadas y Afirmaciones Sustentadas.

Tabla F1

Fuentes Consultadas y Afirmaciones Sustentadas

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Releases; versión 18.6 y soporte hasta 2030	PostgreSQL Global Development Group	Documentación oficial	Versioning policy	s. f.	https://www.postgresql.org/support/versioning/	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
23.2; elección explícita de locale C	PostgreSQL Global Development Group	Documentación oficial	Collation support (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/collation.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
8.1; decimal exacto y diferencias de redondeo	PostgreSQL Global Development Group	Documentación oficial	Numeric types (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/datatype-numeric.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
8.5; DATE y TIMESTAMPTZ	PostgreSQL Global Development Group	Documentación oficial	Date/time types (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/datatype-datetime.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
5.5; CHECK, FK, UNIQUE y límites entre filas	PostgreSQL Global Development Group	Documentación oficial	Constraints (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/ddl-constraints.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Índices parciales y unicidad	PostgreSQL Global Development Group	Documentación oficial	CREATE INDEX (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/sql-createindex.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
24.1; autovacuum y versiones obsoletas	PostgreSQL Global Development Group	Documentación oficial	Routine vacuuming (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/routine-vacuuming.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
14.1; plan y ejecución con ANALYZE	PostgreSQL Global Development Group	Documentación oficial	Using EXPLAIN (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/using-explain.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Funciones, retorno, volatilidad e invocación	PostgreSQL Global Development Group	Documentación oficial	CREATE FUNCTION (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/sql-createfunction.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Procedimientos invocados con CALL	PostgreSQL Global Development Group	Documentación oficial	CREATE PROCEDURE (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/sql-createprocedure.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
13.2; READ COMMITTED y REPEATABLE READ	PostgreSQL Global Development Group	Documentación oficial	Transaction isolation (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/transaction-iso.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
9.17; nextval y rollback	PostgreSQL Global Development Group	Documentación oficial	Sequence manipulation functions (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/functions-sequence.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
13.3; bloqueos de fila, orden e interbloqueo	PostgreSQL Global Development Group	Documentación oficial	Explicit locking (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/explicit-locking.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
LOGIN, NOLOGIN y atributos de rol	PostgreSQL Global Development Group	Documentación oficial	CREATE ROLE (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/sql-createrole.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Privilegios por objeto y columna	PostgreSQL Global Development Group	Documentación oficial	GRANT (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/sql-grant.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
18.9; certificados y cifrado del servidor	PostgreSQL Global Development Group	Documentación oficial	Secure TCP/IP connections with SSL (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/ssl-tcp.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
20.1; hostssl y autenticación	PostgreSQL Global Development Group	Documentación oficial	The pg_hba.conf file (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/auth-pg-hba-conf.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Formato custom y copia lógica consistente	PostgreSQL Global Development Group	Documentación oficial	pg_dump (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/app-pgdump.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
Restauración de archivos no SQL plano	PostgreSQL Global Development Group	Documentación oficial	pg_restore (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/app-pgrestore.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
25.3; copia base y WAL para alcance futuro	PostgreSQL Global Development Group	Documentación oficial	Continuous archiving and point-in-time recovery (PITR) (PostgreSQL 18)	s. f.	https://www.postgresql.org/docs/18/continuous-archiving.html	23/10/2026	Responsable del motor y manual de la versión adoptada; se abrió el texto original.
42.7.14, Java 8 o posterior	PostgreSQL JDBC Driver	Documentación oficial	Download	s. f.	https://jdbc.postgresql.org/download/	23/10/2026	Proyecto oficial del controlador seleccionado.
sslmode=verify-full y raíz confiable	PostgreSQL JDBC Driver	Documentación oficial	Using SSL	s. f.	https://jdbc.postgresql.org/documentation/ssl/	23/10/2026	Proyecto oficial del controlador seleccionado.
URL y Properties	PostgreSQL JDBC Driver	Documentación oficial	Initializing the driver	s. f.	https://jdbc.postgresql.org/documentation/use/	23/10/2026	Proyecto oficial del controlador seleccionado.
Compatibilidad Java 21, JPA 3.2; publicación 7.4.6.Final	Hibernate	Documentación oficial	7.4 series	s. f.	https://hibernate.org/orm/releases/7.4/	23/10/2026	Equipo oficial de Hibernate; documentación de la serie 7.4. La guía dinámica consultada muestra 7.4.13.Final; no se atribuye ese parche al proyecto fijado en 7.4.6.Final.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Identificadores, asociaciones, versiones y transacciones	Hibernate	Documentación oficial	Hibernate ORM user guide	s. f.	https://docs.hibernate.org/orm/7.4/userguide/html_single/	23/10/2026	Equipo oficial de Hibernate; documentación de la serie 7.4. La guía dinámica consultada muestra 7.4.13.Final; no se atribuye ese parche al proyecto fijado en 7.4.6.Final.
Parámetros y proyecciones	Hibernate	Documentación oficial	A guide to Hibernate Query Language	s. f.	https://docs.hibernate.org/orm/7.4/querylanguage/html_single/	23/10/2026	Equipo oficial de Hibernate; documentación de la serie 7.4. La guía dinámica consultada muestra 7.4.13.Final; no se atribuye ese parche al proyecto fijado en 7.4.6.Final.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Bearer y URL sin llave	AYR Tech (Pty) Ltd.	Documentación oficial	API authentication notes	s. f.	https://www.exchangerate-api.com/docs/authentication	23/10/2026	Organización identificada en sus términos; fuente primaria para su propio contrato, sin tratar publicidad como evidencia de beneficios.
Plan Free: USD 0, 1500 solicitudes y actualización diaria	AYR Tech (Pty) Ltd.	Documentación oficial	Exchange rate API: Plans & pricing	s. f.	https://www.exchangerate-api.com/	23/10/2026	Organización identificada en sus términos; fuente primaria para su propio contrato, sin tratar publicidad como evidencia de beneficios.
Estructura JSON y errores	AYR Tech (Pty) Ltd.	Documentación oficial	Standard requests	s. f.	https://www.exchangerate-api.com/docs/standard-requests	23/10/2026	Organización identificada en sus términos; fuente primaria para su propio contrato, sin tratar publicidad como evidencia de beneficios.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Fila NIO	AYR Tech (Pty) Ltd.	Documentación oficial	Supported currency codes and countries	s. f.	https://www.exchangerate-api.com/docs/supported-currencies	23/10/2026	Organización identificada en sus términos; fuente primaria para su propio contrato, sin tratar publicidad como evidencia de beneficios.
License, Disclaimer, Data caching policy y Free plans	AYR Tech (Pty) Ltd.	Documentación oficial	Terms of service	s. f.	https://www.exchangerate-api.com/terms	23/10/2026	Organización identificada en sus términos; fuente primaria para su propio contrato, sin tratar publicidad como evidencia de beneficios.
Comunicado; cifra 36.6243 NIO/USD y deslizamiento 0 % para 2026	Banco Central de Nicaragua	Comunicado institucional	BCN informa que la tasa de deslizamiento del tipo de cambio del córdoba respecto al dólar de los Estados Unidos de América continuará en cero por ciento (0%) anual en el año 2026	20 de octubre de 2025	https://www.bcn.gob.ni/divulgacion-prensa/bcn-informa-que-la-tasa-de-deslizamiento-del-tipo-de-cambio-del-cordoba-respecto	23/10/2026	Organismo competente en la referencia oficial; no atribuye oficialidad a tasas comerciales.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Argon2id; m=19456 KiB, t=2 y p=1	OWASP Foundation	Documentación oficial	Password storage cheat sheet	s. f.	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html	23/10/2026	Proyecto reconocido de seguridad de aplicaciones; documento técnico primario.
API y documentación Java; se conserva versión del TRD	Legion of the Bouncy Castle	Documentación oficial	Bouncy Castle for Java documentation	s. f.	https://www.bouncycastle.org/documentation/documentation-java/	23/10/2026	Proyecto oficial de la biblioteca; no se afirma ejecución del aprovisionamiento Java.
Etiqueta 7.0.8 y cambios publicados	Jaspersoft	Notas de versión	JasperReports 7.0.8 [Notas de versión]	2026	https://github.com/Jaspersoft/jasperreports/releases/tag/7.0.8	23/10/2026	Repositorio oficial del responsable del producto.
Identidad y configuración de la versión etiquetada	Jaspersoft	Código oficial	pom.xml [Código fuente, etiqueta 7.0.8]	s. f.	https://github.com/Jaspersoft/jasperreports/blob/7.0.8/pom.xml	23/10/2026	Código oficial con etiqueta explícita.
JSON base/rates/timestamp y URL API	Open Exchange Rates	Documentación oficial	API introduction	s. f.	https://docs.openexchangerates.org/reference/api-introduction	23/10/2026	Documentación del proveedor alternativo, sin afirmar consumo autenticado.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
1000 solicitudes/mes y actualización horaria	Open Exchange Rates	Documentación oficial	Sign up: The forever free plan	s. f.	https://openexchangerates.org/signup/free	23/10/2026	Condiciones publicadas por el propio proveedor.
Resumen, métodos y resultados; contexto sanitario y límites de transferencia	Lemma, S., Janson, A., Persson, L.-Å., Wickremasinghe, D. y Källestål, C.	Artículo científico	Improving quality and use of routine health information system data in low- and middle-income countries: A scoping review	2020	https://doi.org/10.1371/journal.pone.0239683	23/10/2026	Artículo revisado por pares; página original de PLOS consultada, con autores y método identificables. No describe arrendadores de Managua.
Artículos 11 y 12; La Gaceta N.º 61, 29/03/2012, p. 2404	Asamblea Nacional de la República de Nicaragua	Texto normativo	Ley N.º 787, Ley de Protección de Datos Personales	29 de marzo de 2012	https://sajurin.enriquebolanos.org/docs/G-2012-03-29.pdf	23/10/2026	Texto normativo reproducido en repositorio institucional de Naciones Unidas con enlace al emisor. El sitio de Asamblea no respondió; no se certifica consolidación de reformas.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
RF, RN, RNF, flujos o ADR citados según el cuerpo	Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A.	Documento académico no publicado	Documento de flujo de la aplicación de Latch	2026b	Archivo adjunto: Latch_App_Flow_APA7_v3.pdf	23/10/2026	Documento del proyecto suministrado y consultado; fuente de requisitos, no evidencia empírica externa.
RF, RN, RNF, flujos o ADR citados según el cuerpo	Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A.	Documento académico no publicado	Documento de requisitos del producto de Latch	2026c	Archivo local: Latch_PRD_APA7_v3.pdf	23/10/2026	Documento del proyecto suministrado y consultado; fuente de requisitos, no evidencia empírica externa.
RF, RN, RNF, flujos o ADR citados según el cuerpo	Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A.	Documento académico no publicado	Documento de requisitos técnicos de Latch	2026d	Archivo adjunto: Latch_TRD_APA7_v3.pdf	23/10/2026	Documento del proyecto suministrado y consultado; fuente de requisitos, no evidencia empírica externa.

Afirmación o Sección Utilizada	Autor u Organización	Tipo de Fuente	Título	Publicación	URL o DOI	Consulta	Motivo de Confiabilidad
Pantallas, catálogos, mensajes y UI	Cuadra et al.	Documento académico del proyecto	Brief de diseño de interfaz y experiencia de usuario para Latch	2026a	Archivo local: Latch_Brief_UIUX_APA7_v3.docx	23/10/2026	Fuente primaria interna; las decisiones canónicas prevalecen sobre redacciones anteriores.
Calendario, RACI, revisión cruzada, TAR y CP	Cuadra, S., Montalvan, A., Tardencilla, C. y Gavarrete, A.	Documento académico del proyecto	Plan de implementación de Latch	2026f	Archivo local: Latch_Plan_Implementacion_APA7_v3.docx	23/10/2026	Fuente única de planificación y ejecución CP; Apéndice A TAR-001 a TAR-096.
Antecedente documental citado en el PRD	Balzarini, J., y Boyd, M. L.	Artículo científico	Working with them: Small-scale landlord strategies for avoiding evictions	2021	https://doi.org/10.1080/10511482.2020.1800779	Consulta de coherencia del PRD: 23/10/2026	Housing Policy Debate, 31(3–5), 425–445. Versión en línea anticipada: 2020. Referencia cruzada desde el PRD; no se atribuye consulta nueva del original.